Git Practice Assignments — Tasks 1 to 22

Repository: git-practice-21tasks

GitHub URL: https://github.com/naveenlingampelli456-cmyk/git-practice-21tasks

Environment: Windows 11, Git Bash, GitHub

Task 1: Install Git

Outcome

Git is installed on the Windows system and is available through Git Bash for version control operations.

Objective

To install Git and configure the local environment so that files can be tracked, committed, branched, merged, and synchronized with GitHub repositories.

Prerequisites

- Windows 11 computer.
- Internet connectivity.
- Permission to install software.
- GitHub account.

Step-by-Step Implementation

Step 1: Download Git

1. Open a web browser.
2. Navigate to https://git-scm.com/downloads/win.
3. Download the appropriate Git for Windows installer.
4. Run the downloaded installer.

Step 2: Install Git

1. Follow the installation wizard.
2. Keep the recommended settings unless a specific configuration is required.
3. Complete the installation.

Step 3: Verify Git Installation

Open Git Bash and execute:

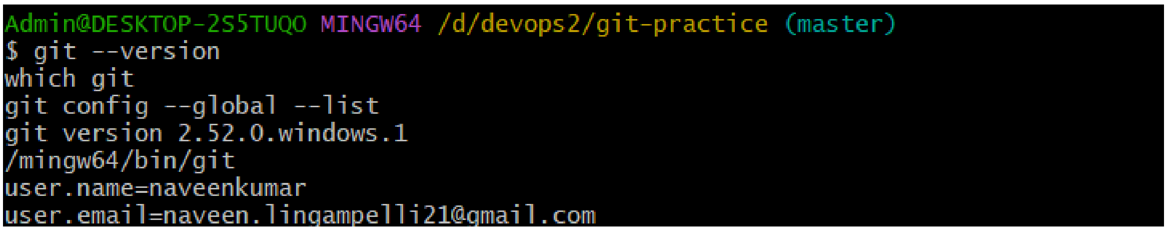
```
git --version
```

Example output:

```
git version 2.x.x.windows.x
```

The exact version depends on the installed release.

Screenshot Evidence



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git --version
which git
git config --global --list
git version 2.52.0.windows.1
/mingw64/bin/git
user.name=naveenkumar
user.email=naveen.lingampelli21@gmail.com
```

Validation Steps

Run:

git --version

Verify that Git returns a version number without errors.

Explanation

Git is a distributed version control system that tracks changes to files and allows developers to maintain project history. It supports commits, branches, merges, and collaboration.

Git Bash provides a command-line environment on Windows where Git commands and many Linux-style commands can be executed.

Real-Time Use Case

Development teams use Git to track source-code changes, maintain feature branches, review changes, and collaborate on application development.

Common Errors and Troubleshooting

Error	**Cause**	**Resolution**
git is not recognized	Git is not installed or PATH is not configured	Reinstall Git and enable the appropriate PATH option
Git Bash does not open	Installation issue or shortcut problem	Verify installation and reopen Git Bash
Incorrect Git version displayed	Older Git version installed	Update Git if required

---

Task 2: Create a GitHub Repository with README.md and .gitignore

Outcome

A GitHub repository named git-practice-21tasks was created with a README file. A .gitignore file was subsequently created and committed to the repository.

Objective

To create a central Git repository for storing project files and practicing version control operations. A README describes the project, while .gitignore specifies files Git should not track.

Prerequisites

- GitHub account.
- Internet connectivity.
- Git installed.
- Access to GitHub repositories.

Step-by-Step Implementation

Step 1: Open GitHub

Navigate to:

https://github.com

Sign in to your GitHub account.

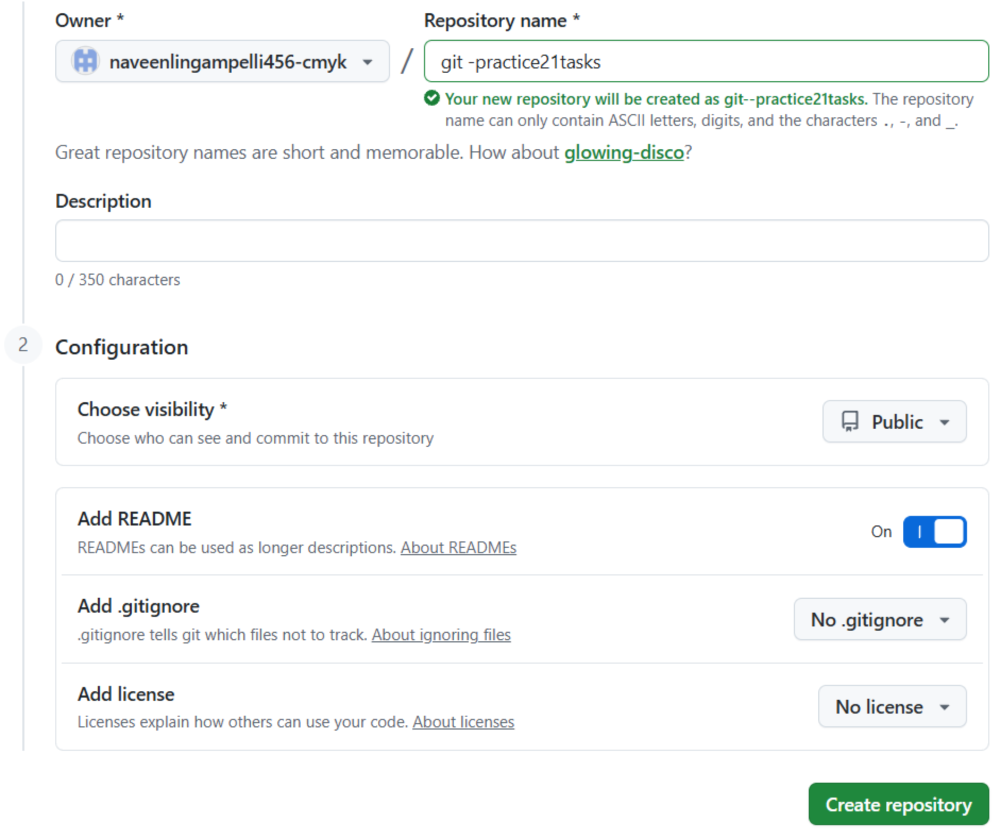
Step 2: Create the Repository

1. Click the **+** icon or **New repository**.
2. Enter the repository name.
3. Select the visibility as required.
4. Select the option to add a README file.
5. Create the repository.

Configuration:

Parameter	**Value**
Repository name	git-practice-21tasks
Owner	naveenlingampelli456-cmyk
README	Enabled
Default branch	Main
Visibility	As selected during creation

Screenshot Evidence



Owner *

naveenlingampelli456-cmyk /

Repository name *

git -practice21tasks

✓ Your new repository will be created as git--practice21tasks. The repository name can only contain ASCII letters, digits, and the characters ., -, and _.

Great repository names are short and memorable. How about glowing-disco?

Description

0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository — Public

Add README
READMEs can be used as longer descriptions. About READMEs — On

Add .gitignore
.gitignore tells git which files not to track. About ignoring files — No .gitignore

Add license
Licenses explain how others can use your code. About licenses — No license

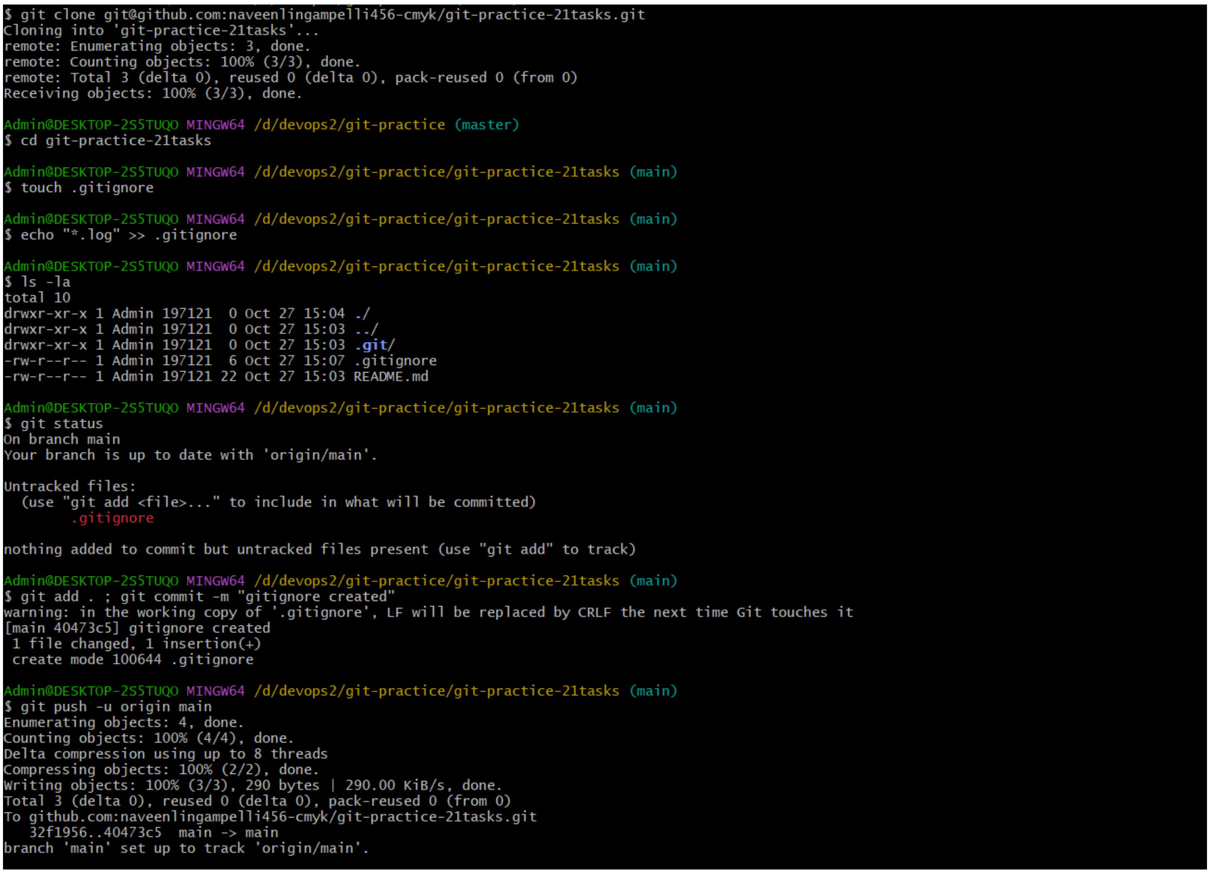
Step 3: Create the .gitignore File

The .gitignore file was created locally after cloning the repository.

```
touch .gitignore

echo "*.log" >> .gitignore

git add .

git commit -m "gitignore created"
```

This adds a rule to ignore files ending in .log.

Screenshot Evidence

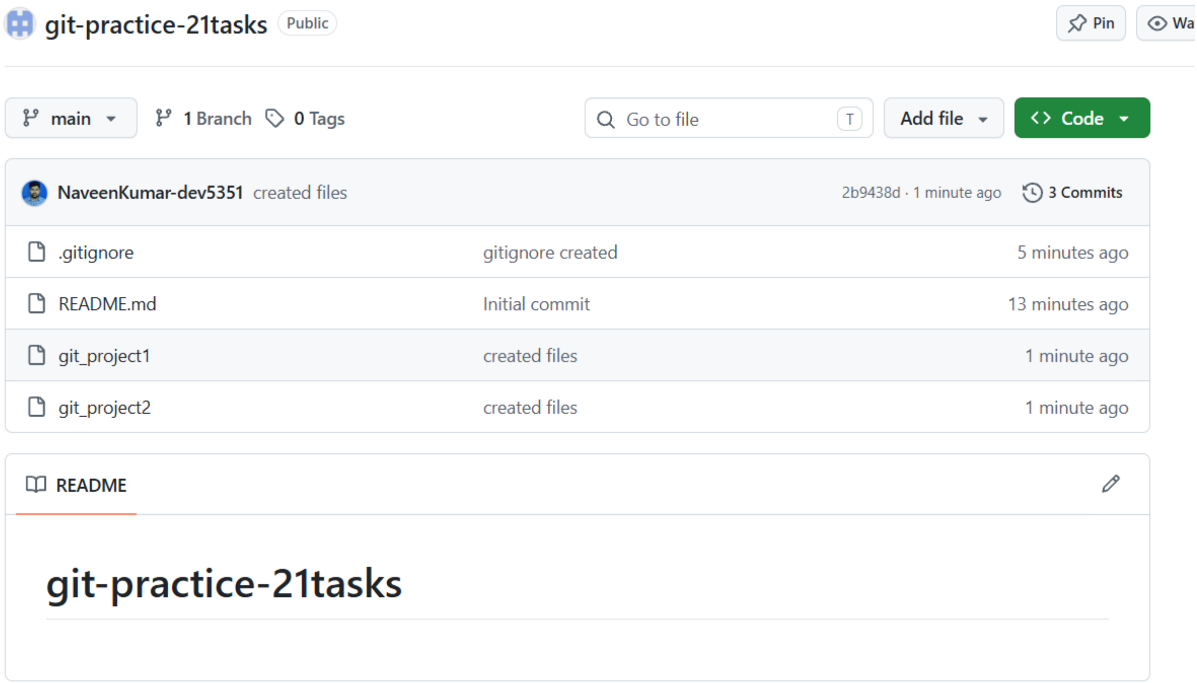


```
$ git clone git@github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git
Cloning into 'git-practice-21tasks'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ cd git-practice-21tasks

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ touch .gitignore

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ echo "*.log" >> .gitignore

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ ls -la
total 10
drwxr-xr-x 1 Admin 197121  0 Oct 27 15:04 ./
drwxr-xr-x 1 Admin 197121  0 Oct 27 15:03 ../
drwxr-xr-x 1 Admin 197121  0 Oct 27 15:03 .git/
-rw-r--r-- 1 Admin 197121  6 Oct 27 15:07 .gitignore
-rw-r--r-- 1 Admin 197121 22 Oct 27 15:03 README.md

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore

nothing added to commit but untracked files present (use "git add" to track)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git add . ; git commit -m "gitignore created"
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
[main 40473c5] gitignore created
 1 file changed, 1 insertion(+)
 create mode 100644 .gitignore

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 290 bytes | 290.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git
   32f1956..40473c5  main -> main
branch 'main' set up to track 'origin/main'.
```

Validation Steps

Open the repository on GitHub and verify that README.md is present. Confirm that .gitignore appears in the repository after its changes are pushed.

Validation Screenshot



git-practice-21tasks

NaveenKumar-dev5351 created files 2b9438d · 1 minute ago

.gitignore	gitignore created	5 minutes ago
README.md	Initial commit	13 minutes ago
git_project1	created files	1 minute ago
git_project2	created files	1 minute ago

git-practice-21tasks

Explanation

A GitHub repository acts as a central location for storing project files and sharing changes. The README introduces the project, and .gitignore prevents specified files from being added to version control.

The rule *.log matches files with the .log extension. It does not remove files that are already tracked by Git.

Real-Time Use Case

Teams maintain application code in GitHub repositories and use .gitignore to exclude generated logs, build artifacts, temporary files, and local configuration files from commits.

Common Errors and Troubleshooting

Error	Cause	Resolution
.gitignore not visible on GitHub	File was not committed or pushed	Run git add .gitignore, commit, and push
Log files still appear in Git status	Files may already be tracked	Remove them from the index using git rm --cached if appropriate
touch command not recognized	Command run in an unsupported terminal	Use Git Bash or create the file through an editor

---

Task 3: Clone the GitHub Repository Locally

Outcome

The git-practice-21tasks repository was available locally at:

/d/devops2/git-practice/git-practice-21tasks

Objective

To create a local copy of a remote repository so that project files can be edited, committed, and pushed using Git.

Prerequisites

- Git installed.
- GitHub repository created.
- Repository URL available.
- Network access to GitHub.

Step-by-Step Implementation

Step 1: Open Git Bash

Open Git Bash and navigate to the directory where you want to store the repository.

```
cd /d/devops2/git-practice
```

Step 2: Clone the Repository

Execute:

```
git clone git@github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git
```

The SSH URL requires SSH authentication to be configured. An HTTPS repository URL can also be used when appropriate.

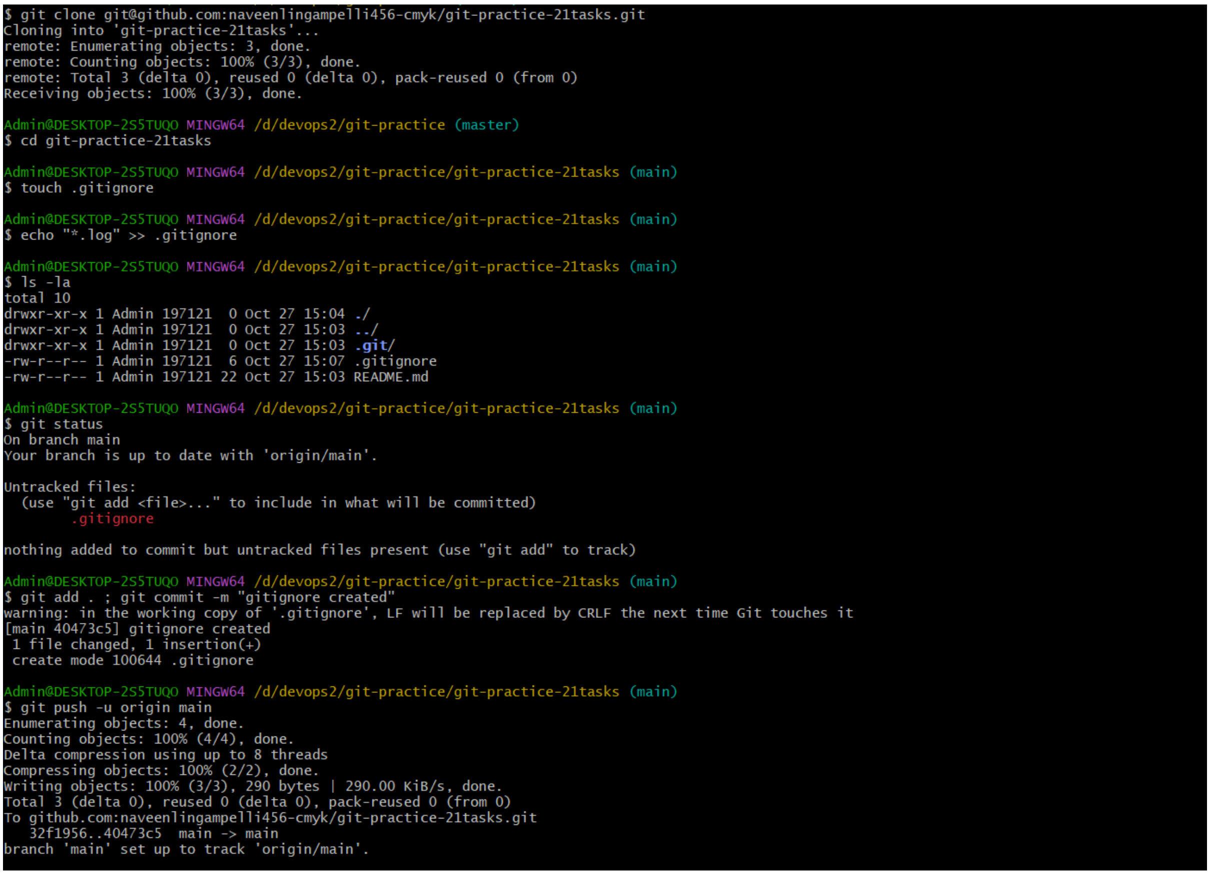
Step 3: Enter the Repository

```
cd git-practice-21tasks
```

Check the current directory and files:

```
pwd
```

```
ls -la
```



```
$ git clone git@github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git
Cloning into 'git-practice-21tasks'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ cd git-practice-21tasks

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ touch .gitignore

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ echo "*.log" >> .gitignore

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ ls -la
total 10
drwxr-xr-x 1 Admin 197121  0 Oct 27 15:04 ./
drwxr-xr-x 1 Admin 197121  0 Oct 27 15:03 ../
drwxr-xr-x 1 Admin 197121  0 Oct 27 15:03 .git/
-rw-r--r-- 1 Admin 197121  6 Oct 27 15:07 .gitignore
-rw-r--r-- 1 Admin 197121 22 Oct 27 15:03 README.md

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore

nothing added to commit but untracked files present (use "git add" to track)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git add . ; git commit -m "gitignore created"
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
[main 40473c5] gitignore created
 1 file changed, 1 insertion(+)
 create mode 100644 .gitignore

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 290 bytes | 290.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git
   32f1956..40473c5  main -> main
branch 'main' set up to track 'origin/main'.
```

Validation Steps

Run:

```
git status
```

```
git remote -v
```

Verify that Git recognizes the repository and that the remote points to the correct GitHub URL.

Explanation

Cloning downloads a repository's files and Git history to the local machine. It also configures a remote, usually named origin, so local changes can be synchronized with GitHub.

A clone is different from downloading a ZIP file because it includes the Git history and branch information needed for version control operations.

Real-Time Use Case

A new developer clones a company's repository to a workstation before implementing a feature or fixing a bug.

Common Errors and Troubleshooting

Error	Cause	Resolution
Permission denied (publickey)	SSH key is missing or not accepted	Configure an SSH key in GitHub and test authentication
Repository not found	Incorrect URL or insufficient access	Verify the repository URL and permissions
Destination path already exists	Directory already exists	Enter the existing repository or clone into a different directory

---

Task 4 and 5: Create Two Files Locally, Commit, and Push to GitHub

Outcome

Two local files were created and the changes were prepared for committing and pushing to the central repository.

Objective

To understand the Git workflow of creating files, staging changes, recording a commit, and pushing the commit to GitHub.

Prerequisites

- Local repository cloned.
- Git configured with a user name and email.
- Remote repository accessible.
- Working directory available.

Step-by-Step Implementation

Step 1: Open the Repository

```
cd /d/devops2/git-practice/git-practice-21tasks
```

Step 2: Create Two Files

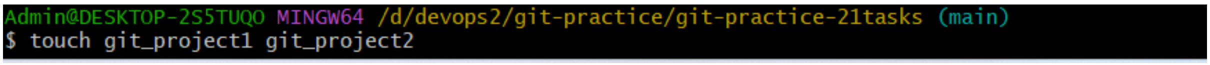
For example:

Touch git_project1 git_project2

Check the files:

ls

Screenshot Evidence



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ touch git_project1 git_project2
```

Step 3: Stage and Commit

git status

git add .

git commit -m "Add two sample files"

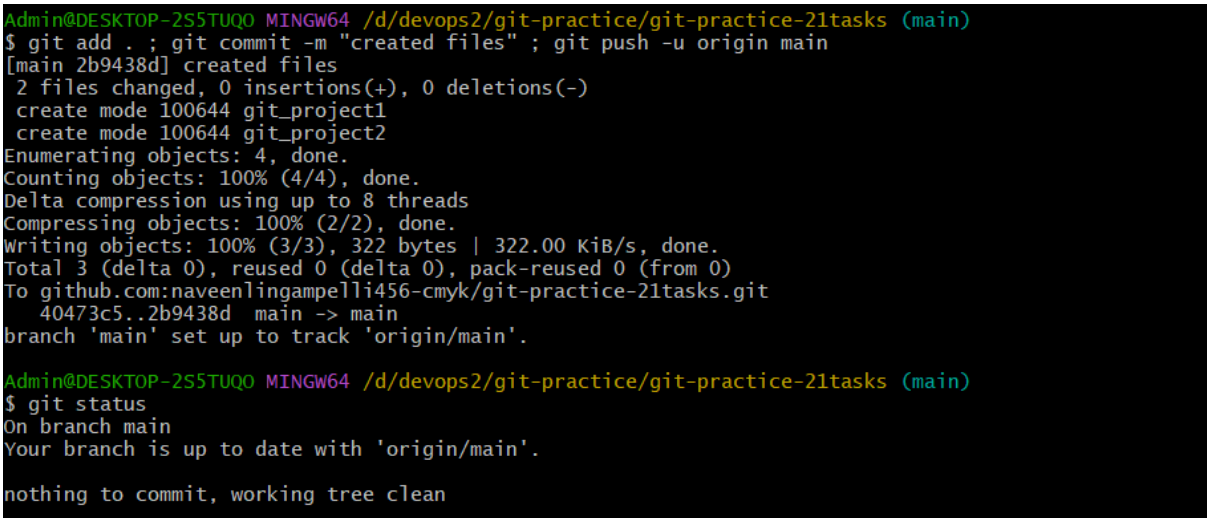
Staging selects the changes to include in the commit.

Step 4: Push to GitHub

git push origin main

If the files were created on a different branch, push that branch instead.

Screenshot Evidence



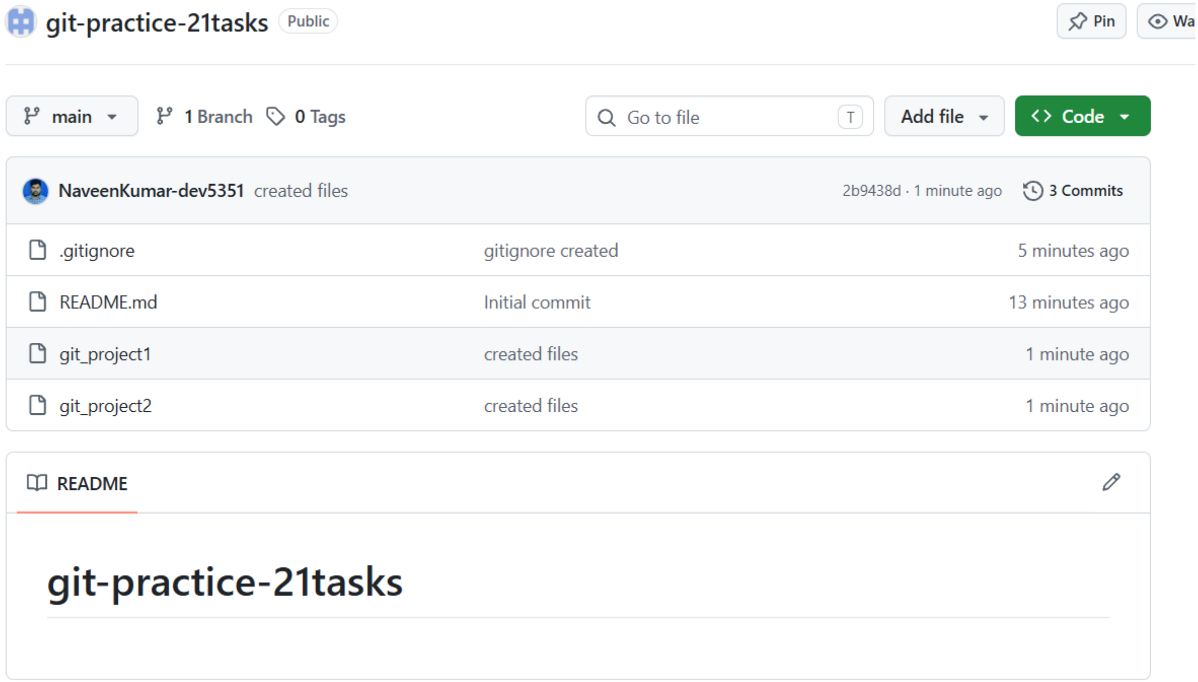
```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git add . ; git commit -m "created files" ; git push -u origin main
[main 2b9438d] created files
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 git_project1
 create mode 100644 git_project2
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 322 bytes | 322.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git
   40473c5..2b9438d  main -> main
branch 'main' set up to track 'origin/main'.

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Validation Steps

Run:

git status

Open the GitHub repository and confirm that both files appear on the branch to which they were pushed.



NaveenKumar-dev5351 created files 2b9438d · 1 minute ago 3 Commits

.gitignore	gitignore created	5 minutes ago
README.md	Initial commit	13 minutes ago
git_project1	created files	1 minute ago
git_project2	created files	1 minute ago

git-practice-21tasks

Explanation

git add stages the selected files, and git commit saves a snapshot in the local repository's history. git push transfers local commits to the remote repository.

A commit does not automatically update GitHub. A push is needed to publish the commit to the remote branch.

Real-Time Use Case

Developers commit and push source-code files, configuration changes, and documentation so teammates can access the latest project version.

Common Errors and Troubleshooting

Error	Cause	Resolution
Nothing to commit	No new changes or changes already committed	Run git status and verify file contents
Push rejected	Remote branch contains newer commits	Fetch or pull the remote changes and resolve conflicts if needed
Permission denied	Authentication or access issue	Verify SSH or HTTPS credentials
File not included in commit	File was not staged	Run git add for the required file

---

Task 6: Create a Local Branch, Add a File, and Push the Branch

Outcome

A local branch named feature-branch was created. The file branch-demo.txt was committed and the branch was pushed to GitHub.

Objective

To understand branch-based development, which allows developers to work on changes separately from the main branch.

Prerequisites

- Local Git repository.
- GitHub remote configured.
- Git authentication working.
- Existing main branch.

Step-by-Step Implementation

Step 1: Create and Switch to the Branch

git switch -c feature-branch

This creates feature-branch and switches to it.

Step 2: Create a Sample File

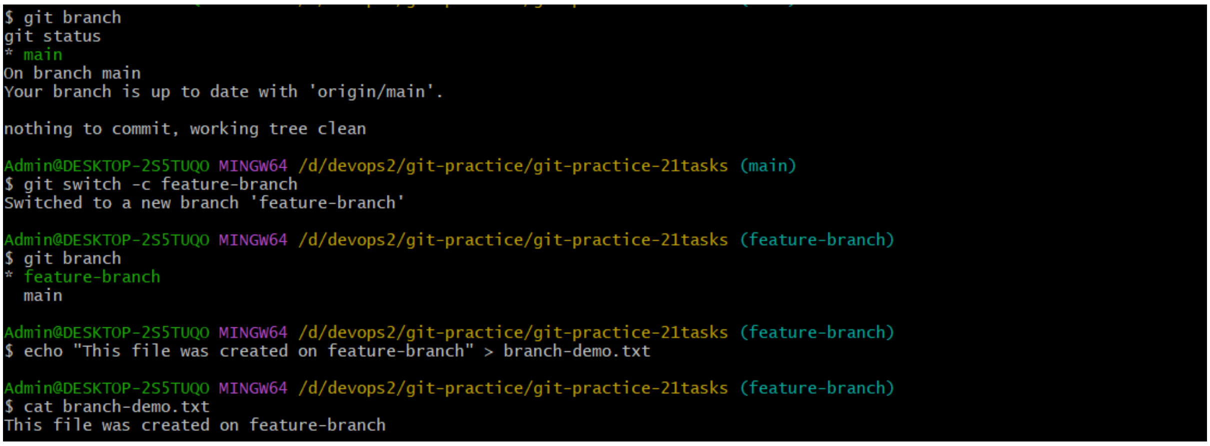
echo "Sample content for feature branch" > branch-demo.txt

Verify:

cat branch-demo.txt

git status

Screenshot Evidence



```
$ git branch
git status
* main
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git switch -c feature-branch
Switched to a new branch 'feature-branch'

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git branch
* feature-branch
  main

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ echo "This file was created on feature-branch" > branch-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ cat branch-demo.txt
This file was created on feature-branch
```

Step 3: Commit the File

git add branch-demo.txt

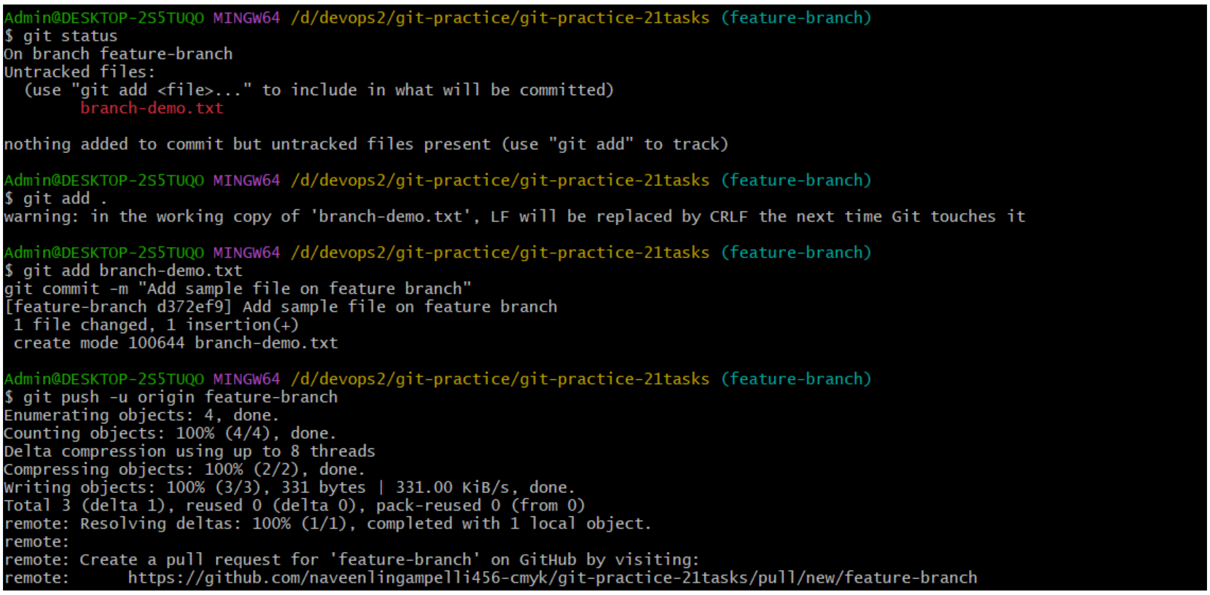
git commit -m "Add sample file on feature branch"

Step 4: Push the Branch to GitHub

git push -u origin feature-branch

The -u option sets the upstream tracking branch.

Screenshot Evidence



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git status
On branch feature-branch
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        branch-demo.txt

nothing added to commit but untracked files present (use "git add" to track)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git add .
warning: in the working copy of 'branch-demo.txt', LF will be replaced by CRLF the next time Git touches it

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git add branch-demo.txt
git commit -m "Add sample file on feature branch"
[feature-branch d372ef9] Add sample file on feature branch
 1 file changed, 1 insertion(+)
 create mode 100644 branch-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git push -u origin feature-branch
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 331 bytes | 331.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature-branch' on GitHub by visiting:
remote:      https://github.com/naveenlingampelli456-cmyk/git-practice-21tasks/pull/new/feature-branch
```

Validation Steps

Run:

git branch -a

git log -1 --oneline

Verify that feature-branch and remotes/origin/feature-branch appear in the branch list.

Explanation

Branches provide independent lines of development. Changes can be committed to a feature branch without immediately modifying main.

Pushing the branch publishes it to GitHub, where it can be reviewed or merged through a pull request.

Real-Time Use Case

A developer creates a feature branch to implement a new login page while other developers continue working on the main branch.

Common Errors and Troubleshooting

Error	**Cause**	**Resolution**
Branch already exists	Branch name is already in use	Switch to the existing branch or choose another name

Push rejected	Remote branch has different commits	Fetch and inspect remote changes
Branch not visible on GitHub	Branch was not pushed	Run git push -u origin feature-branch

---

Task 7: Create a Branch on GitHub and Check It Out Locally

Outcome

A branch named github-created-branch was created on GitHub and then fetched and checked out locally as a tracking branch.

Objective

To learn how to retrieve a branch created remotely and work with it on the local machine.

Prerequisites

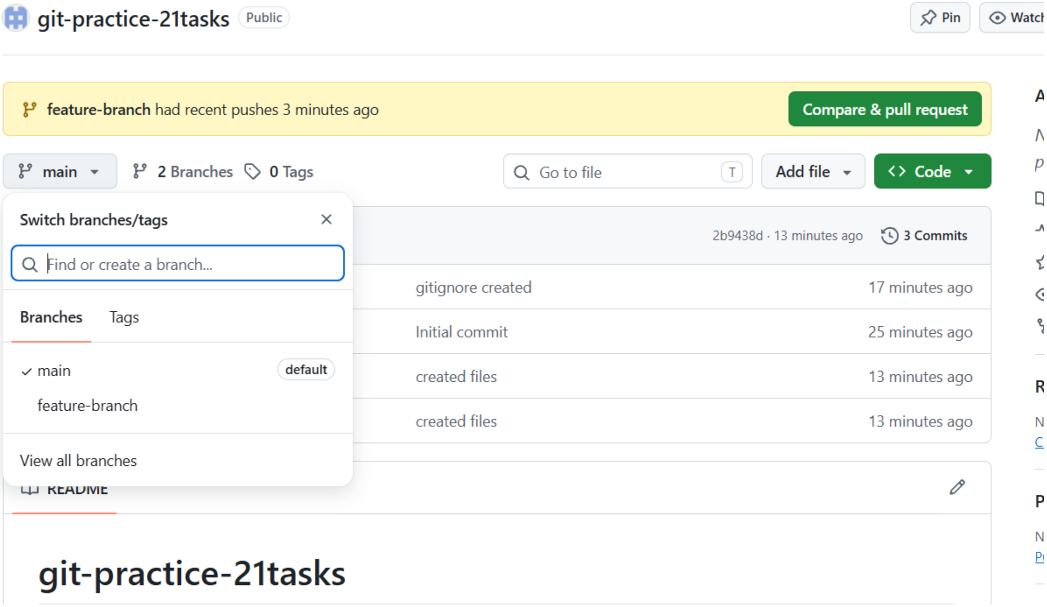
- GitHub repository available.
- Local clone configured with origin.
- Network access.
- Git authentication configured.

Step-by-Step Implementation

Step 1: Create a Branch on GitHub

1. Open the repository on GitHub.
2. Click the branch selector.
3. Enter github-created-branch.
4. Create the branch from main.

Screenshot Evidence


git-practice-21tasks
Public
Pin
Watch
Compare & pull request
main
2 Branches
0 Tags
Go to file
T
Add file
Code
Switch branches/tags
Find or create a branch...
Branches
Tags
main
default
feature-branch
View all branches
2b9438d · 13 minutes ago
3 Commits
README
git-practice-21tasks

feature-branch had recent pushes 3 minutes ago

gitignore created 17 minutes ago

Initial commit 25 minutes ago

created files 13 minutes ago

created files 13 minutes ago

Step 2: Fetch Remote Branch Information

In Git Bash, navigate to the local repository and run:

```
git fetch origin
```

This retrieves remote branch information and commits without automatically merging changes into the current branch.

Step 3: Check Out the Branch Locally

```
git switch --track origin/github-created-branch
```

This creates a local branch that tracks the remote branch and switches to it.

Validation Steps

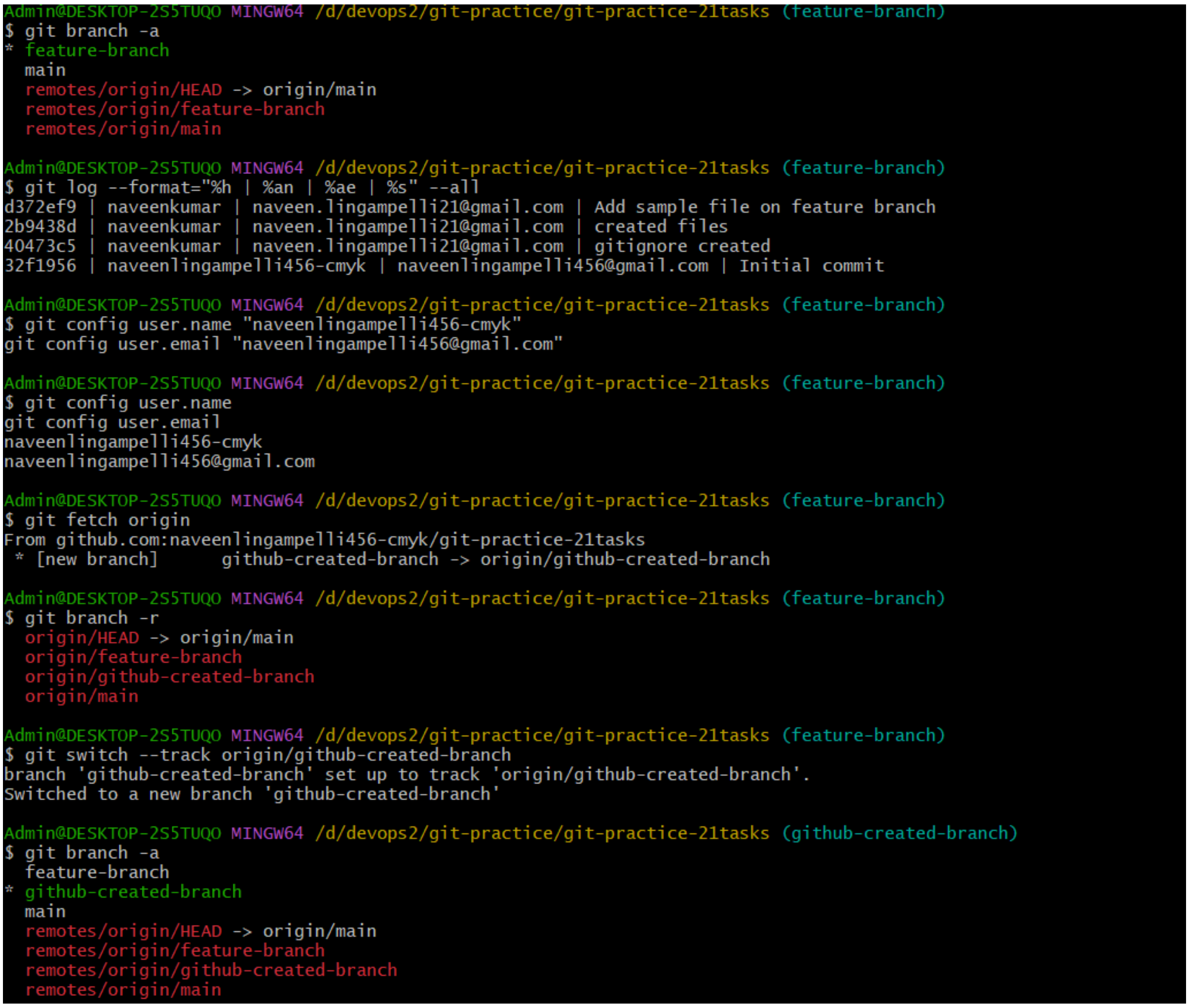
Run:

```
git branch -a
```

```
git status
```

Verify that both github-created-branch and remotes/origin/github-created-branch are listed.

Validation Screenshot



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git branch -a
* feature-branch
  main
  remotes/origin/HEAD -> origin/main
  remotes/origin/feature-branch
  remotes/origin/main

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git log --format="%h | %an | %ae | %s" --all
d372ef9 | naveenkumar | naveen.lingampelli21@gmail.com | Add sample file on feature branch
2b9438d | naveenkumar | naveen.lingampelli21@gmail.com | created files
40473c5 | naveenkumar | naveen.lingampelli21@gmail.com | gitignore created
32f1956 | naveenlingampelli456-cmyk | naveenlingampelli456@gmail.com | Initial commit

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git config user.name "naveenlingampelli456-cmyk"
git config user.email "naveenlingampelli456@gmail.com"

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git config user.name
git config user.email
naveenlingampelli456-cmyk
naveenlingampelli456@gmail.com

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git fetch origin
From github.com:naveenlingampelli456-cmyk/git-practice-21tasks
 * [new branch]      github-created-branch -> origin/github-created-branch

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git branch -r
  origin/HEAD -> origin/main
  origin/feature-branch
  origin/github-created-branch
  origin/main

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git switch --track origin/github-created-branch
branch 'github-created-branch' set up to track 'origin/github-created-branch'.
Switched to a new branch 'github-created-branch'

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (github-created-branch)
$ git branch -a
  feature-branch
* github-created-branch
  main
  remotes/origin/HEAD -> origin/main
  remotes/origin/feature-branch
  remotes/origin/github-created-branch
  remotes/origin/main
```

Explanation

A remote branch is a branch that exists on the remote repository. git fetch updates the local repository's knowledge of remote branches, while git switch --track creates a local branch linked to the remote branch.

Tracking allows Git to determine the upstream branch for operations such as pulling and pushing.

Real-Time Use Case

A developer checks out a branch created by a teammate or through GitHub before reviewing or continuing the work.

Common Errors and Troubleshooting

Error	**Cause**	**Resolution**
Remote branch not found	Branch was not created or fetched	Check the branch on GitHub and run git fetch origin
Local branch already exists	Branch was previously checked out	Use git switch github-created-branch

Permission denied	Authentication problem	Verify GitHub SSH or HTTPS access

---

Task 8: Merge a Created Branch into Main Locally

Outcome

The feature-branch was merged into the local main branch using a fast-forward merge. The updated main branch was then pushed to GitHub.

Objective

To understand how changes from a feature branch can be incorporated into the main branch through a local Git merge.

Prerequisites

- Local main branch.
- Local feature-branch.
- Feature branch changes committed.
- Remote repository configured.

Step-by-Step Implementation

Step 1: Switch to Main

```
git switch main
```

The command switches the working directory to the main branch.

Step 2: Merge the Feature Branch

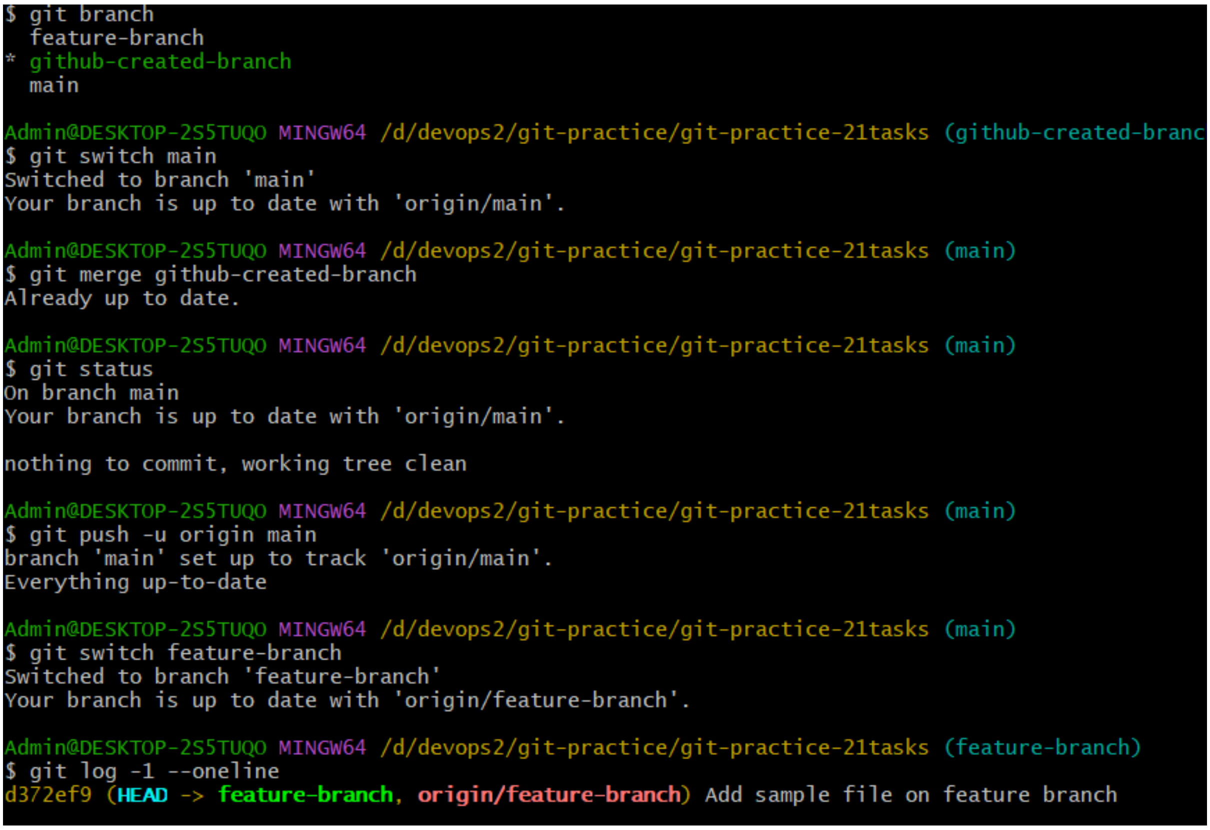
```
git merge feature-branch
```

The merge output showed:

```
Updating 2b9438d..d372ef9
Fast-forward
 branch-demo.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 branch-demo.txt
```

This confirmed that the feature branch's changes were incorporated into main.

Screenshot Evidence



```
$ git branch
  feature-branch
* github-created-branch
  main

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (github-created-branc
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git merge github-created-branch
Already up to date.

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git switch feature-branch
Switched to branch 'feature-branch'
Your branch is up to date with 'origin/feature-branch'.

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git log -1 --oneline
d372ef9 (HEAD -> feature-branch, origin/feature-branch) Add sample file on feature branch
```

Step 3: Verify and Push the Merge

git status

git push -u origin main

The push completed successfully and updated the remote main branch.

Validation Steps

Run:

git status

git log --oneline --graph --all

Verify that the working tree is clean and that branch-demo.txt is included in main.

Explanation

git merge feature-branch incorporates the changes from the specified branch into the current branch. In this task, main was the current branch.

A fast-forward merge is possible when the target branch can advance directly to the source branch's commit without creating a separate merge commit. The updated branch was then pushed to GitHub.

Real-Time Use Case

A developer merges a completed feature into the main development branch after testing and validation.

Common Errors and Troubleshooting

Error	Cause	Resolution
Already up to date	The source branch has no changes missing from the target	Verify branch history with git log --oneline --graph --all
Merge conflict	Both branches changed overlapping parts of files	Resolve conflicts, stage the files, and complete the merge
Push rejected	Remote branch contains newer changes	Fetch and inspect remote history before integrating changes
master branch not found	Repository uses main as its default branch	Use main as the target branch

---

Task 9: Merge a Branch into Main on GitHub Using a Pull Request

Outcome

A GitHub Pull Request was created from feature-branch into main and merged successfully. The resulting merge commit was 01b3f22.

Objective

To learn how to merge branch changes through GitHub's pull request workflow. Pull requests provide a place to review changes before incorporating them into the target branch.

Prerequisites

- GitHub repository with main and feature-branch.
- Feature branch pushed to GitHub.
- Changes available on the source branch that are not yet present in the target branch.
- Permission to create and merge pull requests.

Step-by-Step Implementation

Step 1: Check Branch Differences

From Git Bash, run:

```
git fetch origin
```

```
git switch main
```

```
git log --oneline main..origin/feature-branch
```

Initially, the existing branch had no commits missing from main, so a new change was added to the existing feature-branch.

Step 2: Add and Push a New Change to the Existing Branch

Switch to the existing branch:

```
git switch feature-branch
```

Create a sample file:

echo "Changes for GitHub pull request practice" > pr-demo.txt

Stage and commit:

git add pr-demo.txt

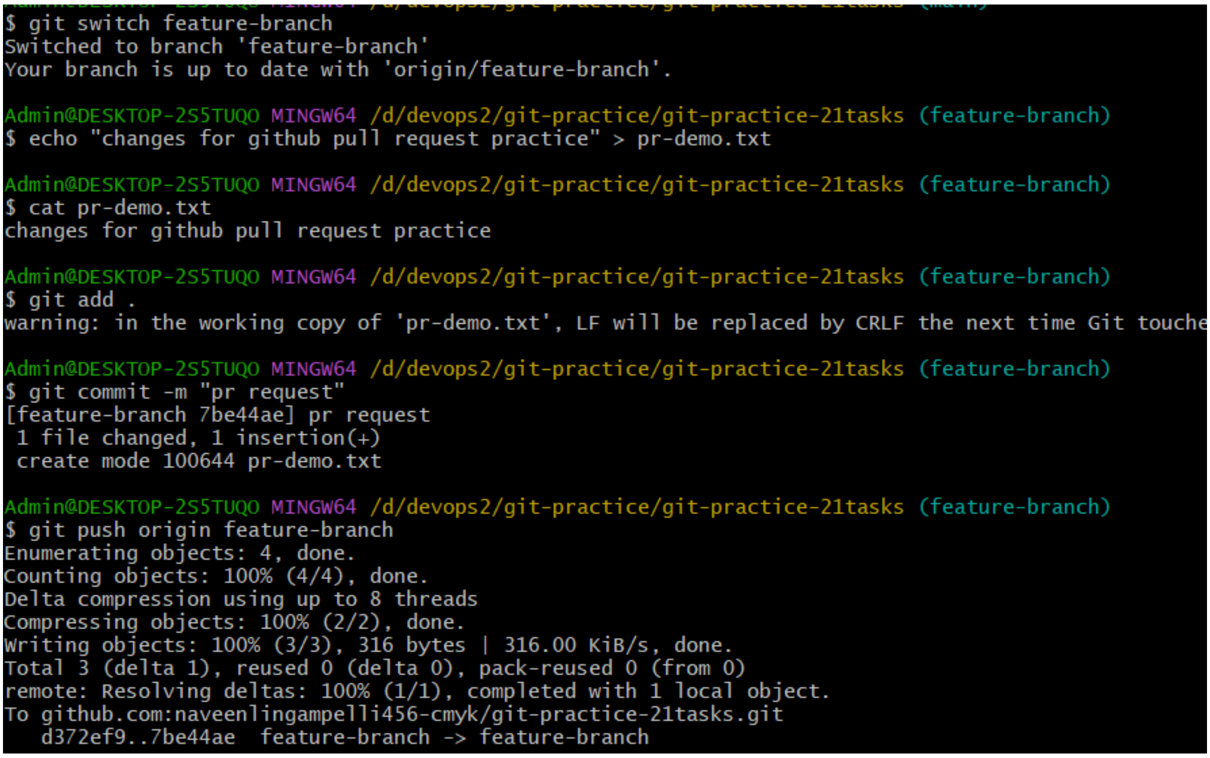
git commit -m "pr request"

Push the branch:

git push origin feature-branch

The branch was updated without creating a new branch.

Screenshot Evidence



```
$ git switch feature-branch
Switched to branch 'feature-branch'
Your branch is up to date with 'origin/feature-branch'.

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ echo "changes for github pull request practice" > pr-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ cat pr-demo.txt
changes for github pull request practice

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git add .
warning: in the working copy of 'pr-demo.txt', LF will be replaced by CRLF the next time Git touche

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git commit -m "pr request"
[feature-branch 7be44ae] pr request
 1 file changed, 1 insertion(+)
 create mode 100644 pr-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git push origin feature-branch
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 316 bytes | 316.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git
   d372ef9..7be44ae  feature-branch -> feature-branch
```

Step 3: Create the Pull Request on GitHub

1. Open the repository:
   https://github.com/naveenlingampelli456-cmyk/git-practice-21tasks
2. Select **Pull requests**.
3. Click **New pull request**.
4. Choose the following branches:

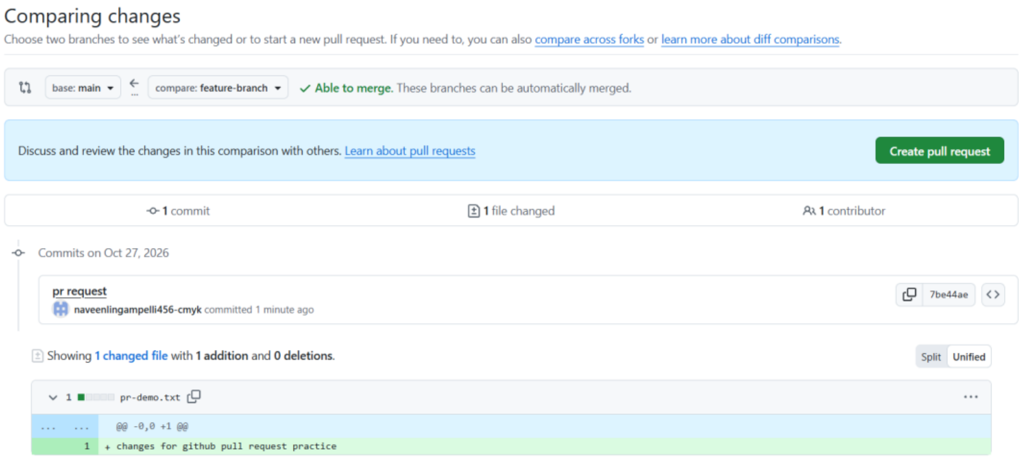
Parameter	**Value**
Base branch	Main
Compare branch	feature-branch

Pull request title pr request or an appropriate descriptive title

5. Review the proposed changes.
6. Click **Create pull request**.

Screenshot Evidence

[



Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also compare across forks or learn more about diff comparisons.

base: main ← compare: feature-branch ✓ **Able to merge.** These branches can be automatically merged.

Discuss and review the changes in this comparison with others. Learn about pull requests

1 commit · 1 file changed · 1 contributor

Commits on Oct 27, 2026

pr request
naveenlingampelli456-cmyk committed 1 minute ago · 7be44ae

Showing 1 changed file with **1 addition** and **0 deletions**.

```
1 pr-demo.txt
... ...  @@ -0,0 +1 @@
      1  + changes for github pull request practice
```

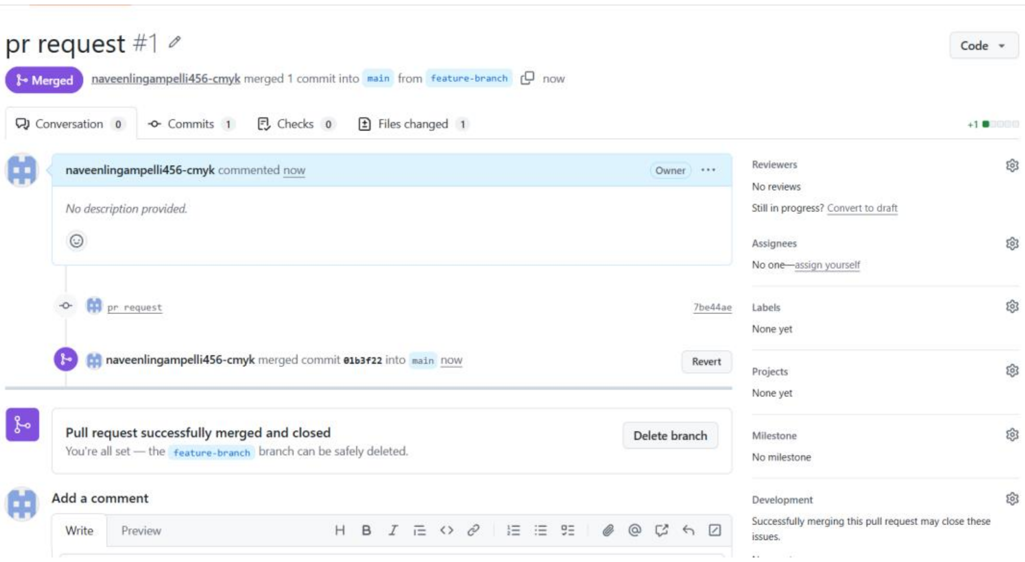
Step 4: Merge the Pull Request

1. Open the created pull request.
2. Review the changes.
3. Click **Merge pull request**.
4. Click **Confirm merge**.

GitHub created merge commit 01b3f22.

Screenshot Evidence

[



pr request #1

Merged naveenlingampelli456-cmyk merged 1 commit into `main` from `feature-branch` now

Conversation 0 · Commits 1 · Checks 0 · Files changed 1 · +1

naveenlingampelli456-cmyk commented now · Owner

No description provided.

pr request · 7be44ae

naveenlingampelli456-cmyk merged commit 01b3f22 into `main` now

Pull request successfully merged and closed

You're all set — the `feature-branch` branch can be safely deleted.

Add a comment

Reviewers
No reviews
Still in progress? Convert to draft

Assignees
No one—assign yourself

Labels
None yet

Projects
None yet

Milestone
No milestone

Development
Successfully merging this pull request may close these issues.

Step 5: Synchronize and Verify Locally

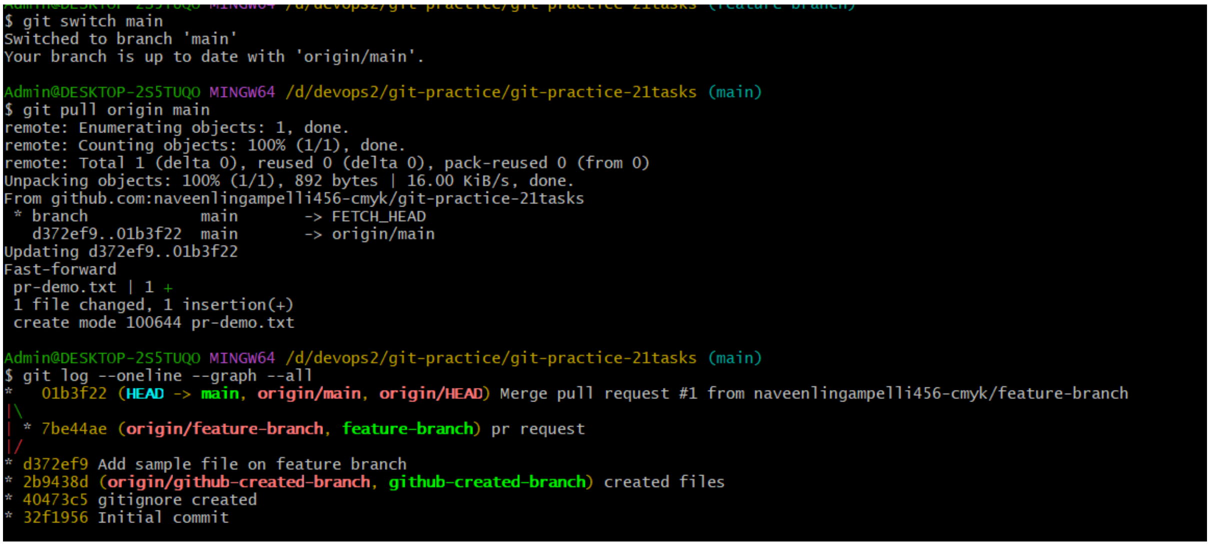
Run:

```
git switch main

git pull origin main

git log --oneline --graph --all
```

The Git log showed:

```
*   01b3f22 (HEAD -> main, origin/main, origin/HEAD) Merge pull request #1 from naveenlingampelli456-cmyk/feature-branch

|\

| * 7be44ae (origin/feature-branch, feature-branch) pr request

|/

* d372ef9 Add sample file on feature branch

* 2b9438d (origin/github-created-branch, github-created-branch) created files

* 40473c5 gitignore created

* 32f1956 Initial commit
```

Screenshot Evidence



```
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git pull origin main
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 892 bytes | 16.00 KiB/s, done.
From github.com:naveenlingampelli456-cmyk/git-practice-21tasks
 * branch            main       -> FETCH_HEAD
   d372ef9..01b3f22  main       -> origin/main
Updating d372ef9..01b3f22
Fast-forward
 pr-demo.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 pr-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git log --oneline --graph --all
*   01b3f22 (HEAD -> main, origin/main, origin/HEAD) Merge pull request #1 from naveenlingampelli456-cmyk/feature-branch
|\
| * 7be44ae (origin/feature-branch, feature-branch) pr request
|/
* d372ef9 Add sample file on feature branch
* 2b9438d (origin/github-created-branch, github-created-branch) created files
* 40473c5 gitignore created
* 32f1956 Initial commit
```

Validation Steps

Run:

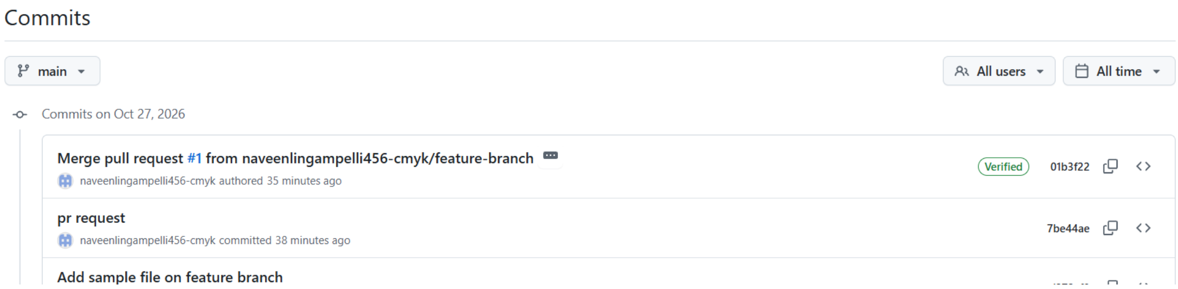
```
git status

git log --oneline --graph --all
```

Verify the following:

- The Pull Request is marked **Merged** on GitHub.

- The merge commit appears in the history.
- main and origin/main point to the same commit.
- The working tree is clean.

Validation Screenshot



Commits on Oct 27, 2026

Merge pull request #1 from naveenlingampelli456-cmyk/feature-branch
naveenlingampelli456-cmyk authored 35 minutes ago
01b3f22

pr request
naveenlingampelli456-cmyk committed 38 minutes ago
7be44ae

Add sample file on feature branch

Explanation

A pull request is a GitHub collaboration feature that proposes merging changes from one branch into another. It allows developers to review code, discuss changes, run checks, and approve the changes before merging.

In this task, feature-branch was used as the source and main as the target. GitHub created merge commit 01b3f22, joining the source branch's changes into the target branch. The local repository was then synchronized with GitHub.

Real-Time Use Case

In a software development team, developers create feature branches for individual tasks. They open pull requests so teammates can review code and automated checks can run before changes are merged into the main branch.

Common Errors and Troubleshooting

Error	Cause	Resolution
There is nothing to compare	Source branch has no changes missing from target	Commit and push a new change to the source branch
Pull request cannot be merged automatically	Conflicting changes exist	Resolve conflicts using GitHub or locally
Merge button unavailable	Insufficient permissions or required checks incomplete	Check repository permissions and branch protection rules
Local main does not show the merge	Local branch has not been updated	Run git pull origin main
Wrong target branch selected	Base branch was selected incorrectly	Select main as the base branch for this repository

---

Conclusion

Tasks 1–8 cover the fundamentals of Git and GitHub: installing Git, creating a repository, cloning it, committing files, creating and pushing branches, checking out remote branches, merging locally, and merging through a GitHub Pull Request.

These exercises demonstrate the basic version control workflow used in software development. The completed pull request is verified by merge commit 01b3f22, with main and origin/main synchronized.

Task 10: Create a File Locally and Push It to a GitHub Branch

Outcome

A file named local-branch-demo.txt is created in the local Git repository, committed to the existing feature-branch, and pushed successfully to GitHub.

Objective

To understand how to create a file locally, track it with Git, commit the changes, and push the commit to a specific remote branch on GitHub. This workflow is essential for publishing code changes from a developer's local environment to a shared repository.

Prerequisites

- Git installed on Windows.
- GitHub account and repository access.
- Local repository cloned to the computer.
- Git configured with a user name and email.
- SSH authentication or HTTPS authentication configured.
- Existing local and remote feature-branch.

Repository details:

Parameter	**Value**
Repository name	git-practice-21tasks
Local repository path	/d/devops2/git-practice/git-practice-21tasks
GitHub repository	naveenlingampelli456-cmyk/git-practice-21tasks
Target branch	feature-branch
File name	local-branch-demo.txt

Step-by-Step Implementation

Step 1: Open the Local Repository

Open Git Bash and navigate to the local repository.

```
cd /d/devops2/git-practice/git-practice-21tasks
```

Verify the current directory:

pwd

Step 2: Switch to the Target Branch

Switch to the existing feature-branch.

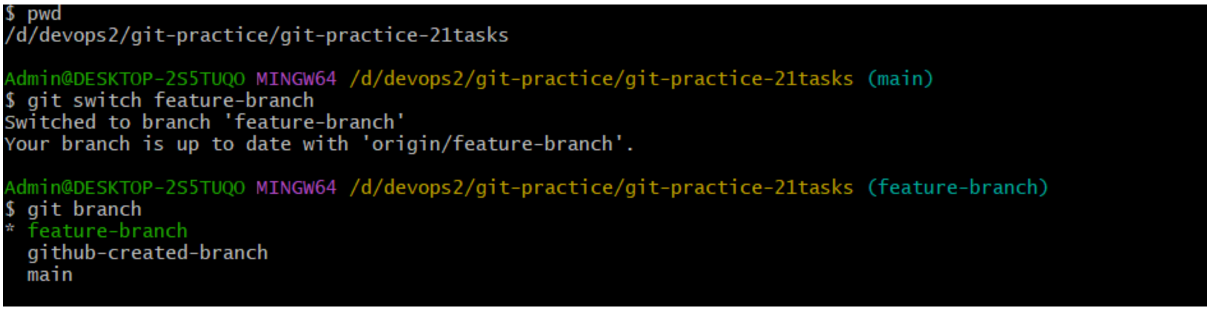
git switch feature-branch

Verify the current branch:

git branch

The asterisk (*) should appear beside feature-branch.

Screenshot Evidence



```
$ pwd
/d/devops2/git-practice/git-practice-21tasks

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git switch feature-branch
Switched to branch 'feature-branch'
Your branch is up to date with 'origin/feature-branch'.

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git branch
* feature-branch
  github-created-branch
  main
```

Step 3: Create a File Locally

Create a sample text file and add content.

echo "File created locally and pushed to GitHub branch" > local-branch-demo.txt

Verify the file contents:

cat local-branch-demo.txt

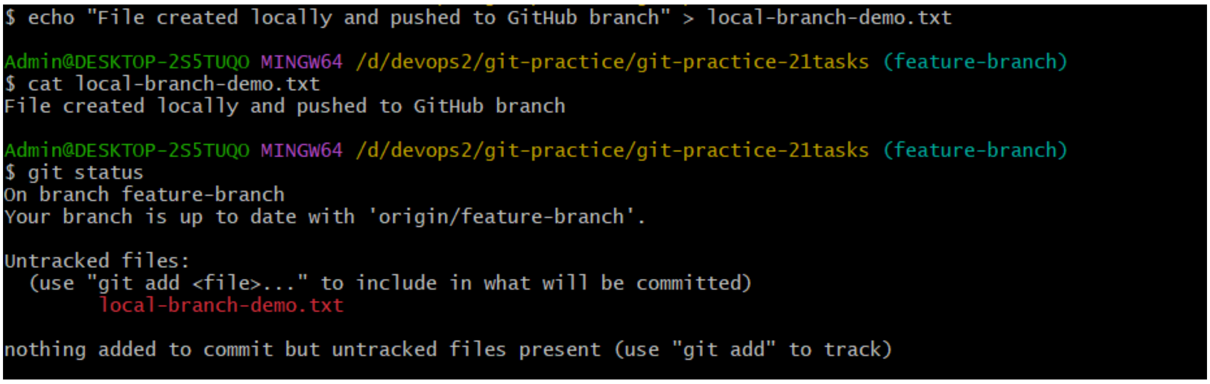
Check the working-tree status:

git status

The new file should appear as an untracked file.

Screenshot Evidence

[



```
$ echo "File created locally and pushed to GitHub branch" > local-branch-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ cat local-branch-demo.txt
File created locally and pushed to GitHub branch

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git status
On branch feature-branch
Your branch is up to date with 'origin/feature-branch'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        local-branch-demo.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Step 4: Stage the File

Stage the newly created file.

```
git add local-branch-demo.txt
```

Verify the staged change:

```
git status
```

The file should appear under **Changes to be committed**.

Step 5: Commit the File

Create a commit with a descriptive message.

```
git commit -m "Add locally created file to feature branch"
```

The commit records the file in the local Git history.

Step 6: Push the Commit to GitHub

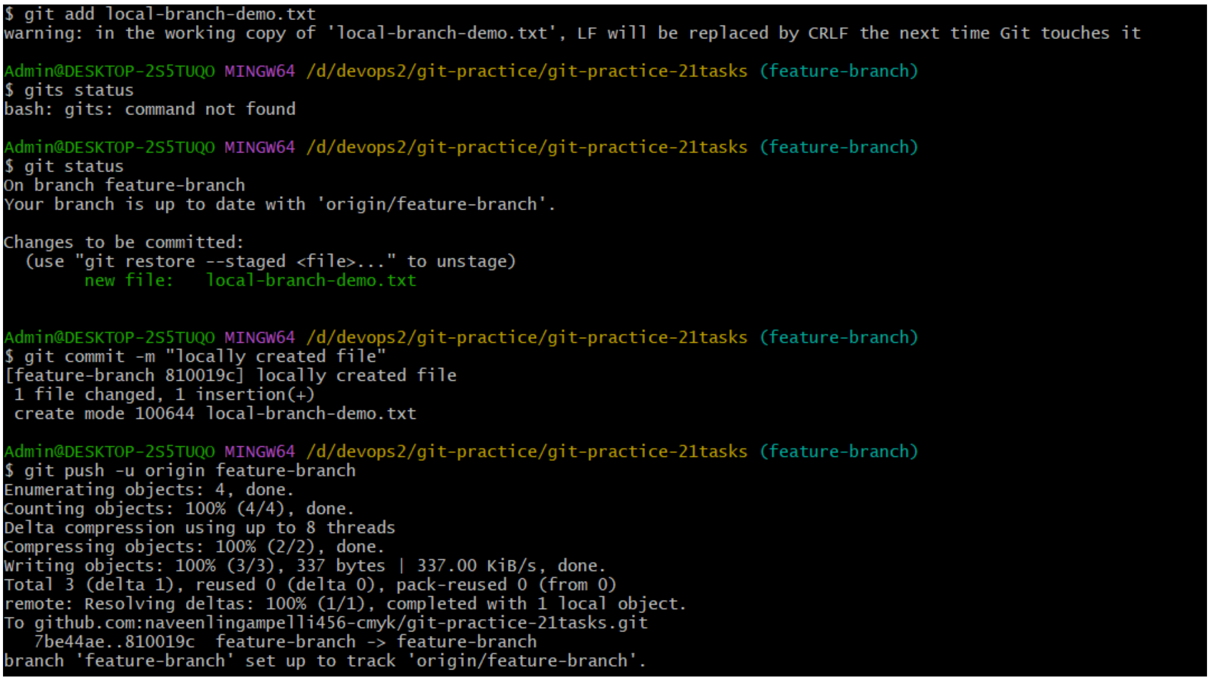
Push the committed change to the existing remote branch.

```
git push origin feature-branch
```

If the branch has not been configured to track its remote branch, use:

```
git push -u origin feature-branch
```

Screenshot Evidence



```
$ git add local-branch-demo.txt
warning: in the working copy of 'local-branch-demo.txt', LF will be replaced by CRLF the next time Git touches it

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ gits status
bash: gits: command not found

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git status
On branch feature-branch
Your branch is up to date with 'origin/feature-branch'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   local-branch-demo.txt


Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git commit -m "locally created file"
[feature-branch 810019c] locally created file
 1 file changed, 1 insertion(+)
 create mode 100644 local-branch-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git push -u origin feature-branch
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 337 bytes | 337.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git
   7be44ae..810019c  feature-branch -> feature-branch
branch 'feature-branch' set up to track 'origin/feature-branch'.
```

Validation Steps

Step 1: Verify the Local Repository

Run:

```
git status

git log -1 --oneline
```

Confirm that the working tree is clean and the latest commit message is displayed.

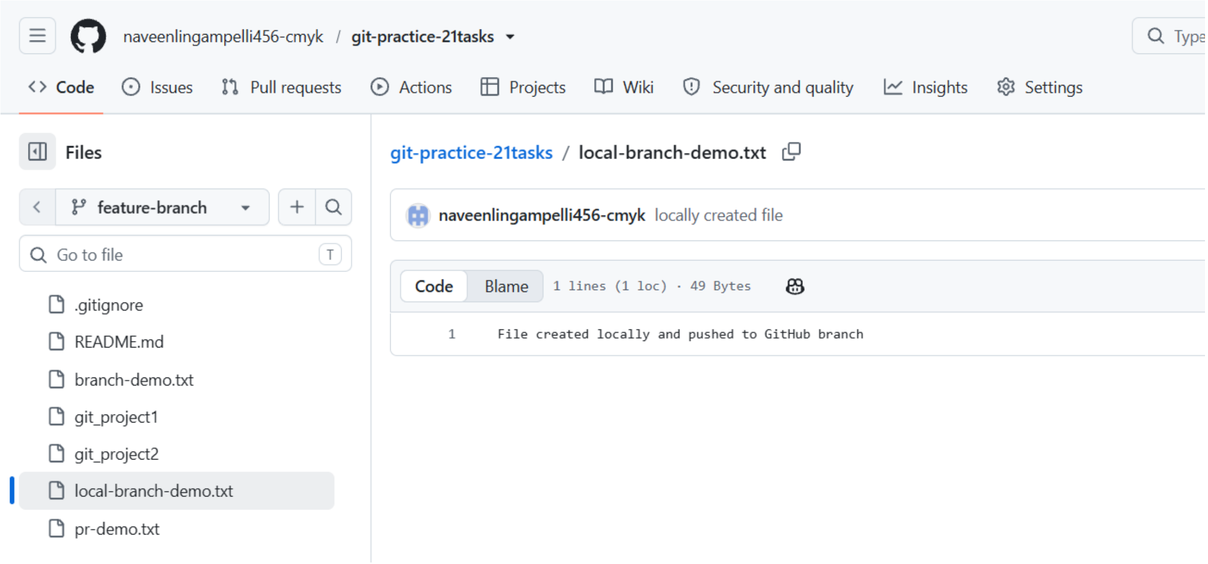
Step 2: Verify the File on GitHub

1. Open the GitHub repository:

https://github.com/naveenlingampelli456-cmyk/git-practice-21tasks

2. Open the branch selector.
3. Select feature-branch.
4. Locate local-branch-demo.txt.
5. Open the file and verify its contents.

Validation Screenshot



naveenlingampelli456-cmyk / git-practice-21tasks
Type
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
Files
feature-branch
Go to file
T
.gitignore
README.md
branch-demo.txt
git_project1
git_project2
local-branch-demo.txt
pr-demo.txt
git-practice-21tasks / local-branch-demo.txt
Code
Blame
1 lines (1 loc) · 49 Bytes

naveenlingampelli456-cmyk locally created file

```
1    File created locally and pushed to GitHub branch
```

Explanation

In this task, a file is created in the local repository and added to Git's tracking process. The git add command stages the file, and git commit records the change in the local repository's history. The git push origin feature-branch command transfers the new commit to the remote feature-branch on GitHub.

It is important to switch to the correct branch before creating and committing the file. Pushing to feature-branch publishes the change on that branch without directly modifying main.

Real-Time Use Case

In a DevOps or software development team, a developer may update an application configuration file, deployment script, or project documentation. The developer commits the changes to a feature branch and pushes them to GitHub for review before merging them into the main branch.

Common Errors and Troubleshooting

Error	Cause	Resolution
fatal: not a git repository	Git Bash is not inside the repository directory	Navigate to the correct repository path
File appears as untracked	File has not been staged	Run git add local-branch-demo.txt
nothing to commit	No new changes are staged	Check the file name and run git status
Permission denied (publickey)	SSH authentication is not configured correctly	Verify the SSH key and GitHub account configuration
src refspec feature-branch does not match any	Local branch name is incorrect or the branch has no commit	Check git branch and verify the commit exists
Push rejected	Remote branch contains changes not present locally	Fetch and inspect the remote changes before pushing
File is missing on GitHub	The file was pushed to a different branch	Select feature-branch in GitHub and verify the latest push

Conclusion

This task demonstrates the complete process of creating a file locally, staging it, committing it, and pushing it to an existing GitHub branch. Successful validation on GitHub confirms that the local change has been published to the correct remote branch.

The task is complete when local-branch-demo.txt is visible on feature-branch on GitHub and the local working tree is clean.

Task 11: Clone Only One Branch from GitHub to Local

Outcome

The feature-branch of the GitHub repository git-practice-21tasks is cloned into a separate local directory. Only the selected branch is downloaded as a local branch, rather than cloning all remote branches.

Objective

To learn how to clone a specific branch from a remote GitHub repository. Cloning only the required branch can reduce the amount of repository history and branch information downloaded, making it useful when a developer needs to work on a particular feature or maintenance branch.

Prerequisites

- Git installed on Windows.
- GitHub account and repository access.
- Internet connectivity.
- SSH authentication or HTTPS authentication configured.

- Existing GitHub repository with the feature-branch branch.

Repository Configuration

Parameter	Value
Repository name	git-practice-21tasks
GitHub owner	naveenlingampelli456-cmyk
Repository URL	git@github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git
Branch to clone	feature-branch
Clone option	--single-branch
Destination directory	git-practice-feature-only

Step-by-Step Implementation

Step 1: Open Git Bash

Open Git Bash on Windows and navigate to the directory where you want to store the cloned repository.

```
cd /d/devops2/git-practice
```

Verify the current directory:

```
pwd
```

Step 2: Clone Only the Required Branch

Execute the following command:

```
git clone --branch feature-branch --single-branch git@github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git git-practice-feature-only
```

Explanation of the command:

- git clone downloads the repository.
- --branch feature-branch selects the branch to check out.
- --single-branch limits the clone to the selected branch instead of fetching other branches.
- The SSH URL identifies the remote GitHub repository.
- git-practice-feature-only is the destination directory.

If SSH authentication is unavailable, use the HTTPS URL instead:

```
git clone --branch feature-branch --single-branch https://github.com/naveenlingampelli456-cmyk/git-practice-21tasks.git git-practice-feature-only
```

Step 3: Navigate to the Cloned Repository

cd git-practice-feature-only

Verify the current directory:

Step 4: Verify the Current Branch

Execute:

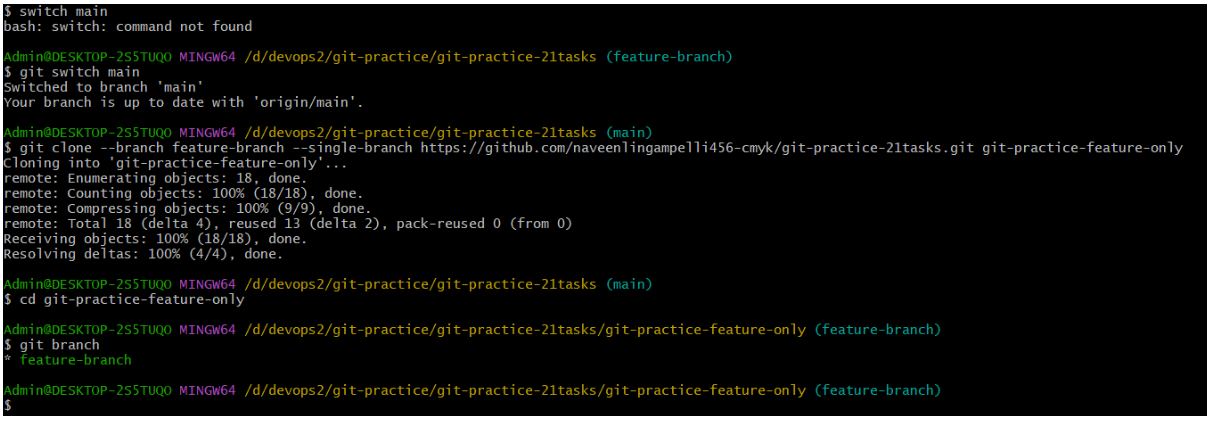
git branch

Expected output:

* feature-branch

The asterisk indicates the currently checked-out branch.

Screenshot Evidence



```
$ switch main
bash: switch: command not found

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (feature-branch)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git clone --branch feature-branch --single-branch https://github.com/naveenlingampelli456-cmyk/git-practice-21tasks.git git-practice-feature-only
Cloning into 'git-practice-feature-only'...
remote: Enumerating objects: 18, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 18 (delta 4), reused 13 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (18/18), done.
Resolving deltas: 100% (4/4), done.

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ cd git-practice-feature-only

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks/git-practice-feature-only (feature-branch)
$ git branch
* feature-branch

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks/git-practice-feature-only (feature-branch)
$
```

Step 5: Verify the Remote Branch Configuration

Execute:

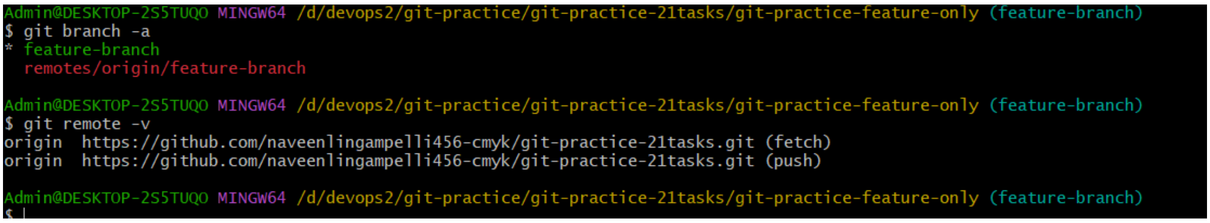
git branch -a

With a single-branch clone, the output should normally show the local feature-branch, its remote-tracking branch origin/feature-branch, and the remote HEAD reference. Other remote branches should not be fetched as normal remote-tracking branches.

To verify the configured remote:

git remote -v

Screenshot Evidence



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks/git-practice-feature-only (feature-branch)
$ git branch -a
* feature-branch
  remotes/origin/feature-branch

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks/git-practice-feature-only (feature-branch)
$ git remote -v
origin  https://github.com/naveenlingampelli456-cmyk/git-practice-21tasks.git (fetch)
origin  https://github.com/naveenlingampelli456-cmyk/git-practice-21tasks.git (push)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks/git-practice-feature-only (feature-branch)
$ |
```

Validation Steps

Step 1: Verify the Branch

Run:

git status

git branch -a

Confirm that feature-branch is checked out and that the clone is configured to track the selected remote branch.

Step 2: Verify Repository Files

Run:

ls -la

You should see the repository files committed on feature-branch, such as branch-demo.txt and any other files included in that branch.

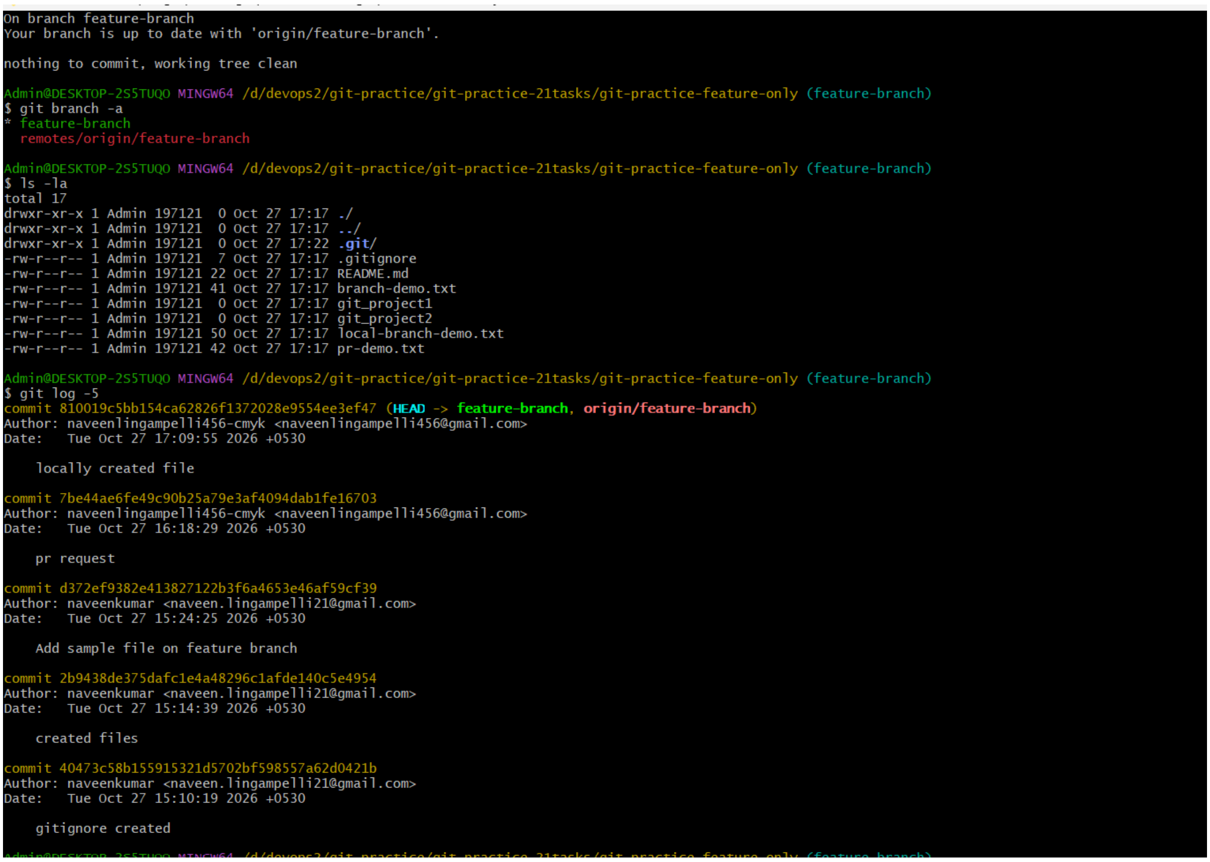
Step 3: Verify the Commit History

Run:

git log --oneline -5

Confirm that Git displays the recent commits available in the cloned branch.

Validation Screenshot



```
On branch feature-branch
Your branch is up to date with 'origin/feature-branch'.

nothing to commit, working tree clean

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks/git-practice-feature-only (feature-branch)
$ git branch -a
* feature-branch
  remotes/origin/feature-branch

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks/git-practice-feature-only (feature-branch)
$ ls -la
total 17
drwxr-xr-x 1 Admin 197121  0 Oct 27 17:17 ./
drwxr-xr-x 1 Admin 197121  0 Oct 27 17:17 ../
drwxr-xr-x 1 Admin 197121  0 Oct 27 17:22 .git/
-rw-r--r-- 1 Admin 197121  7 Oct 27 17:17 .gitignore
-rw-r--r-- 1 Admin 197121 22 Oct 27 17:17 README.md
-rw-r--r-- 1 Admin 197121 41 Oct 27 17:17 branch-demo.txt
-rw-r--r-- 1 Admin 197121  0 Oct 27 17:17 git_project1
-rw-r--r-- 1 Admin 197121  0 Oct 27 17:17 git_project2
-rw-r--r-- 1 Admin 197121 50 Oct 27 17:17 local-branch-demo.txt
-rw-r--r-- 1 Admin 197121 42 Oct 27 17:17 pr-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks/git-practice-feature-only (feature-branch)
$ git log -5
commit 810019c5bb154ca62826f1372028e9554ee3ef47 (HEAD -> feature-branch, origin/feature-branch)
Author: naveenlingampelli456-cmyk <naveenlingampelli456@gmail.com>
Date:   Tue Oct 27 17:09:55 2026 +0530

    locally created file

commit 7be44ae6fe49c90b25a79e3af4094dab1fe16703
Author: naveenlingampelli456-cmyk <naveenlingampelli456@gmail.com>
Date:   Tue Oct 27 16:18:29 2026 +0530

    pr request

commit d372ef9382e413827122b3f6a4653e46af59cf39
Author: naveenkumar <naveen.lingampelli21@gmail.com>
Date:   Tue Oct 27 15:24:25 2026 +0530

    Add sample file on feature branch

commit 2b9438de375dafc1e4a48296c1afde140c5e4954
Author: naveenkumar <naveen.lingampelli21@gmail.com>
Date:   Tue Oct 27 15:14:39 2026 +0530

    created files

commit 40473c58b155915321d5702bf598557a62d0421b
Author: naveenkumar <naveen.lingampelli21@gmail.com>
Date:   Tue Oct 27 15:10:19 2026 +0530

    gitignore created
```

Explanation

The git clone --branch option selects the branch to check out when cloning a repository. Adding --single-branch restricts the clone's fetched branch history to that selected branch instead of fetching the histories of other branches. This is useful when working with a particular feature branch or when limiting unnecessary branch data.

The command creates a new local repository directory, downloads the selected branch's history, and configures the remote connection. This task uses a separate destination directory so the existing working repository remains unchanged.

Real-Time Use Case

A developer may need to work only on a particular feature branch in a large project. By cloning that branch with --single-branch, the developer can focus on the required work without fetching the histories of every other branch.

Common Errors and Troubleshooting

Error	Cause	Resolution
Repository not found	Incorrect URL or insufficient repository access	Verify the GitHub repository URL and permissions
Permission denied (publickey)	SSH key is missing or not accepted	Configure the SSH key in GitHub or use HTTPS
Remote branch feature-branch not found	The specified branch does not exist remotely	Verify the branch name on GitHub
destination path already exists	The destination directory already exists and is not empty	Choose a new destination directory or inspect the existing directory
Other branches appear in the branch list	The clone was not restricted to a single branch, or additional references were fetched later	Verify the clone command and remote configuration
Expected file is missing	The file may not exist on the selected branch	Check the branch's files on GitHub

---

Conclusion

This task demonstrates how to clone only one branch from GitHub using git clone --branch with the --single-branch option. The selected feature-branch is checked out in a separate local directory, and the branch configuration, repository files, remote URL, and commit history are verified.

The task is complete when the new directory contains the cloned repository, feature-branch is checked out, and the clone is configured to fetch only the selected branch by default.

Task 12: Create a Password File and Make It Untrackable with Git

Outcome

A file named passwords.txt is created locally to simulate storing sensitive information. The file is added to .gitignore so Git ignores it and does not include it in future commits when it is untracked.

Objective

To understand how Git's .gitignore file prevents specified files from being included in version control. This exercise demonstrates a basic way to avoid accidentally committing sensitive files, such as local password lists or environment-specific configuration files.

Security note: Use only dummy credentials in this exercise. .gitignore does not encrypt a file or protect it from being accessed locally. It also does not automatically stop tracking a file that has already been committed.

Prerequisites

- Git installed on Windows.
- Local Git repository available.
- GitHub repository configured as a remote.
- Access to Git Bash.
- .gitignore file present in the repository.
- Only dummy password values for testing.

Repository Configuration

Parameter	Value
Repository	git-practice-21tasks
Local directory	/d/devops2/git-practice/git-practice-21tasks
Sensitive test file	passwords.txt
Ignore configuration	.gitignore
Git ignore rule	passwords.txt

Step-by-Step Implementation

Step 1: Open the Local Repository

Open Git Bash and navigate to the repository.

```
cd /d/devops2/git-practice/git-practice-21tasks
```

Verify the current directory:

```
pwd
```

Step 2: Create a Dummy Password File

Create a file containing sample, non-real credentials.

printf '%s\n' \

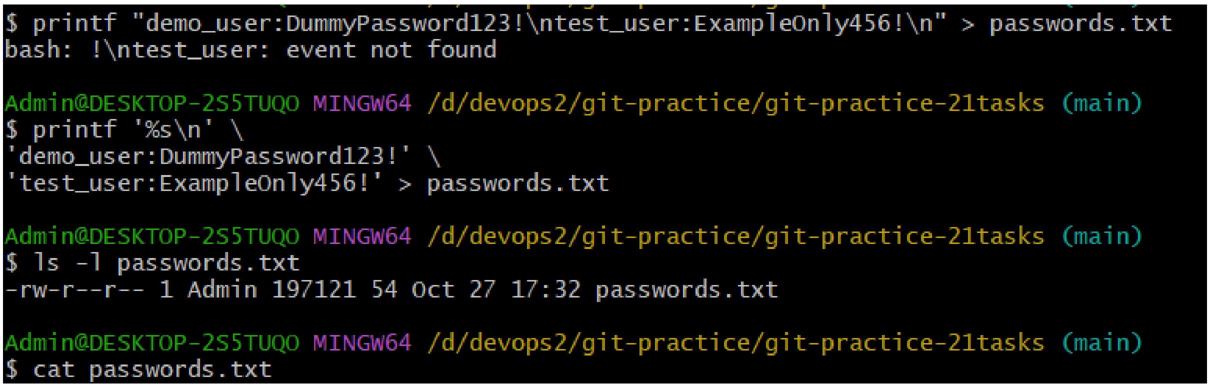
'demo_user:DummyPassword123!' \

'test_user:ExampleOnly456!' > passwords.txt

Verify that the file exists:

ls -l passwords.txt

For this exercise, do not use real usernames or passwords.



```
$ printf "demo_user:DummyPassword123!\ntest_user:ExampleOnly456!\n" > passwords.txt
bash: !\ntest_user: event not found

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ printf '%s\n' \
'demo_user:DummyPassword123!' \
'test_user:ExampleOnly456!' > passwords.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ ls -l passwords.txt
-rw-r--r-- 1 Admin 197121 54 Oct 27 17:32 passwords.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ cat passwords.txt
```

Step 3: Add the Password File to .gitignore

Open the .gitignore file:

vim .gitignore

Add this line:

passwords.txt

Save and exit Vim:

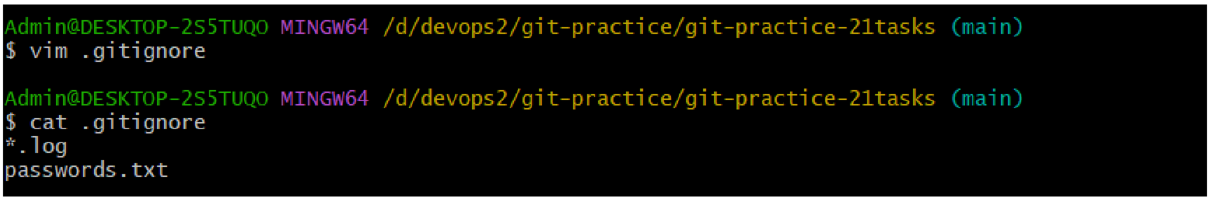
1. Press Esc.
2. Type :wq.
3. Press Enter.

Alternatively, append the rule from the terminal:

echo "passwords.txt" >> .gitignore

Use one method only, and ensure the rule is added just once.

Screenshot Evidence



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ vim .gitignore

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ cat .gitignore
*.log
passwords.txt
```

Step 4: Verify the Ignore Rule

Run:

```
git check-ignore -v passwords.txt
```

Expected output will identify .gitignore and the rule matching passwords.txt, for example:

```
.gitignore:...:passwords.txt    passwords.txt
```

The exact line number may differ.

Step 5: Verify Git Status

Run:

```
git status
```

The untracked passwords.txt file should not appear in the list of untracked files. The .gitignore modification may appear as a changed file if it has not yet been committed.

Step 6: Commit the .gitignore Change

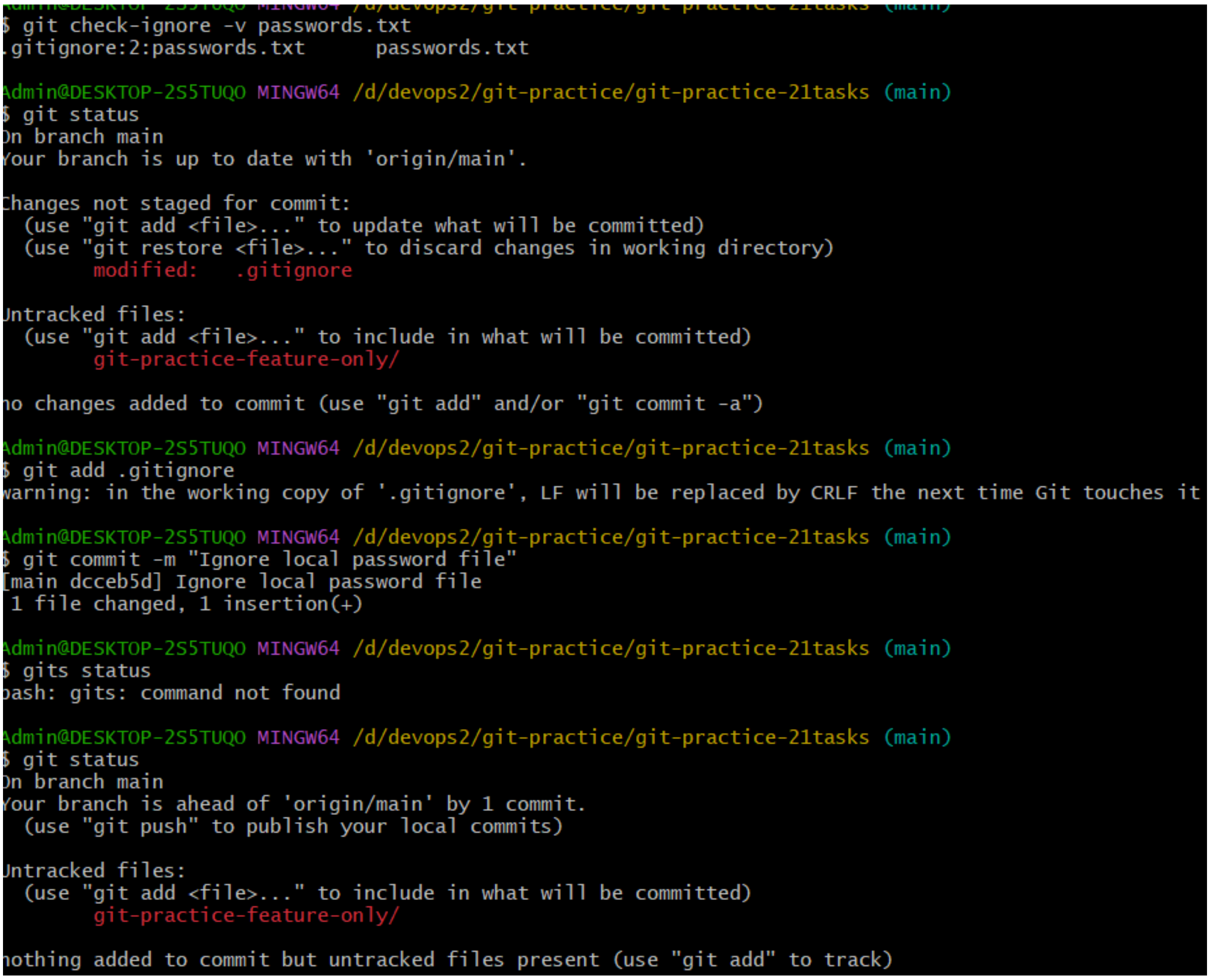
Stage and commit only .gitignore:

```
git add .gitignore
```

```
git commit -m "Ignore local password file"
```

Do not stage passwords.txt.

Screenshot Evidence



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git check-ignore -v passwords.txt
.gitignore:2:passwords.txt      passwords.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        git-practice-feature-only/

no changes added to commit (use "git add" and/or "git commit -a")

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git add .gitignore
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git commit -m "Ignore local password file"
[main dcceb5d] Ignore local password file
 1 file changed, 1 insertion(+)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ gits status
bash: gits: command not found

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        git-practice-feature-only/

nothing added to commit but untracked files present (use "git add" to track)
```

Validation Steps

Step 1: Confirm Git Ignores the File

Run:

git check-ignore -v passwords.txt

Confirm that the output identifies the matching ignore rule.

Step 2: Confirm the File Is Not Staged

Run:

git status --short

The file passwords.txt should not appear. If .gitignore has already been committed, there may be no output when the working tree is clean.

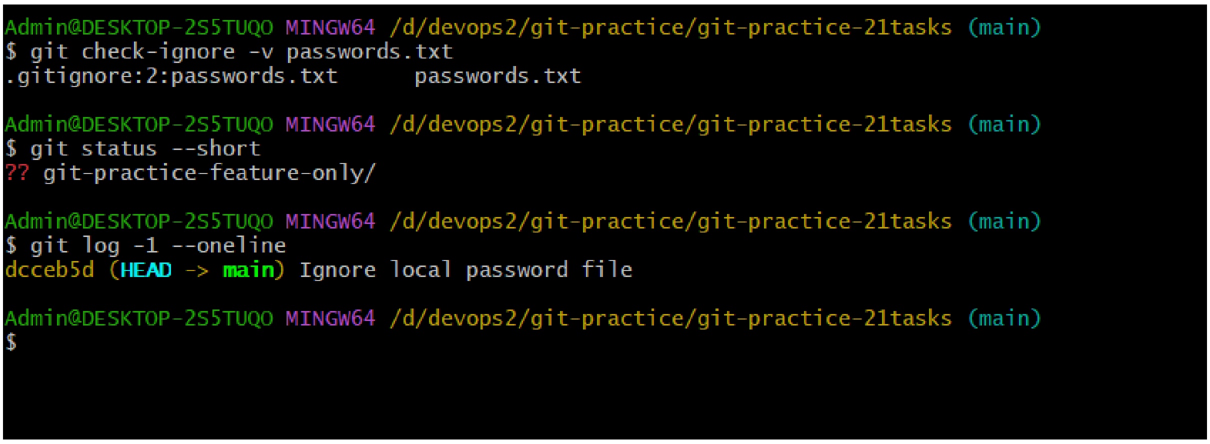
Step 3: Verify the Ignore Rule Is Committed

Run:

git log -1 --oneline

Confirm that the latest commit is the commit that records the .gitignore change, assuming no later commits have been created.

Validation Screenshot



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git check-ignore -v passwords.txt
.gitignore:2:passwords.txt      passwords.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git status --short
?? git-practice-feature-only/

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git log -1 --oneline
dcceb5d (HEAD -> main) Ignore local password file

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$
```

Explanation

The .gitignore file tells Git which untracked files or file patterns to exclude from normal status reporting and staging workflows. Adding passwords.txt to .gitignore prevents that file from being included accidentally when staging ordinary project changes.

However, .gitignore is not a security mechanism. It does not encrypt the file, prevent local access, or remove a file from Git history if it was already committed. Sensitive values should be stored in a secure secrets manager or another approved credential-storage system.

Real-Time Use Case

Developers often have local configuration files containing development credentials or environment-specific settings. A .gitignore rule helps prevent these files from being accidentally committed and pushed to shared repositories.

In production environments, credentials should be managed using dedicated secrets-management solutions, such as a cloud secrets manager or an approved enterprise vault.

Common Errors and Troubleshooting

Error	Cause	Resolution
passwords.txt still appears in git status	Ignore rule is missing, incorrect, or in the wrong directory	Check .gitignore and run git check-ignore -v passwords.txt
Password file is still tracked	The file was already added to Git's index or committed	Remove it from tracking with git rm --cached passwords.txt, then commit the change; keep the local file if required
.gitignore is not visible on GitHub	The file was not committed or pushed	Commit .gitignore and push the relevant branch

File is ignored but still visible locally	Git ignore rules do not delete local files	This is expected; .gitignore only affects Git tracking behavior
A real password was accidentally committed	Sensitive data entered Git history	Rotate or revoke the credential immediately; removing the file from the latest commit alone may not remove it from history

Conclusion

This task demonstrates how to create a local dummy password file and configure .gitignore to prevent the untracked file from being included in future Git commits. The git check-ignore -v command validates that the rule is active.

The task is complete when Git ignores passwords.txt, the ignore rule is committed, and the password file remains local rather than being pushed to GitHub.

Task 13: Make a Commit and Reset It Without Saving Changes

Outcome

A test file is created and committed to the local Git repository. The latest commit is then reset using git reset --hard HEAD~1, removing that commit from the current branch and discarding its associated changes.

Objective

To understand how Git reset works and how to remove a commit from the current branch history. This task demonstrates a hard reset, which resets the branch pointer and working directory to the previous commit.

Warning: git reset --hard permanently discards uncommitted changes in tracked files and can make commits harder to recover. Never run it on important work or a shared branch without understanding the consequences. Perform this exercise on a local test branch, not directly on main.

Prerequisites

- Git installed on Windows.
- Local Git repository available.
- Git Bash access.
- A clean working tree before starting.
- A local test branch for this exercise.
- No important uncommitted changes.

Configuration

Parameter	Value
Repository	git-practice-21tasks

Test branch	reset-demo
Test file	reset-demo.txt
Commit message	Add file for reset demonstration
Reset command	git reset --hard HEAD~1

Step-by-Step Implementation

Step 1: Open the Local Repository

Open Git Bash and navigate to the main repository.

```
cd /d/devops2/git-practice/git-practice-21tasks
```

Check the working tree:

```
git status
```

Ensure there are no important uncommitted changes before proceeding. If the output shows modified or untracked files, inspect them before continuing.

Step 2: Create a Test Branch

Create a separate local branch for the reset demonstration.

```
git switch -c reset-demo
```

If the branch already exists, switch to it instead:

```
git switch reset-demo
```

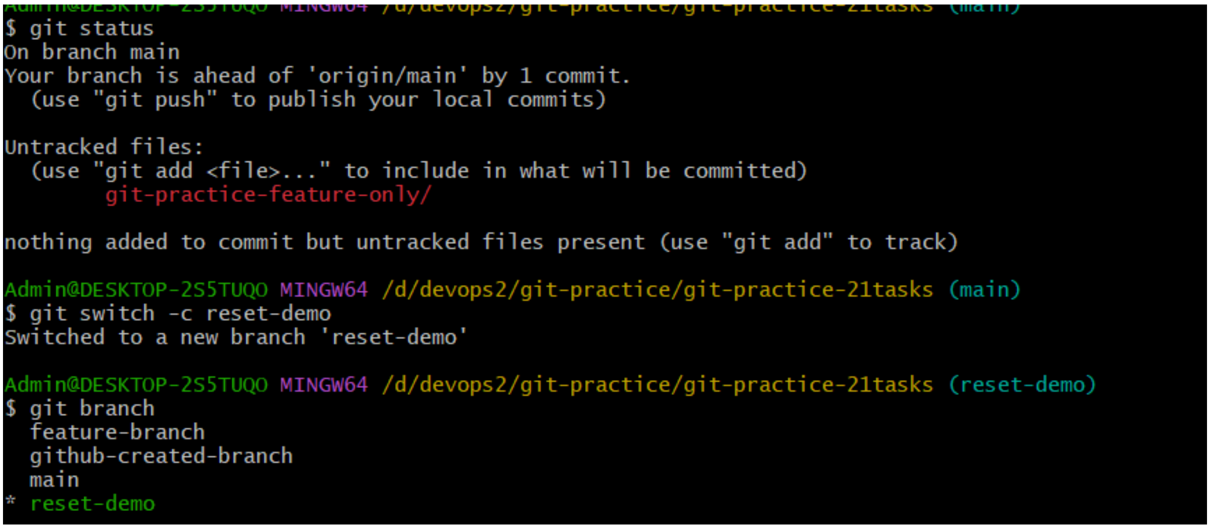
Verify the current branch:

```
git branch
```

The asterisk should appear beside reset-demo.

Screenshot Evidence

[



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        git-practice-feature-only/

nothing added to commit but untracked files present (use "git add" to track)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git switch -c reset-demo
Switched to a new branch 'reset-demo'

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (reset-demo)
$ git branch
  feature-branch
  github-created-branch
  main
* reset-demo
```

Step 3: Create a Test File

Create a temporary file:

echo "This file is for Git reset practice" > reset-demo.txt

Verify the contents:

cat reset-demo.txt

git status

The file should appear as untracked.

Step 4: Stage and Commit the File

Stage the file:

git add reset-demo.txt

Commit the change:

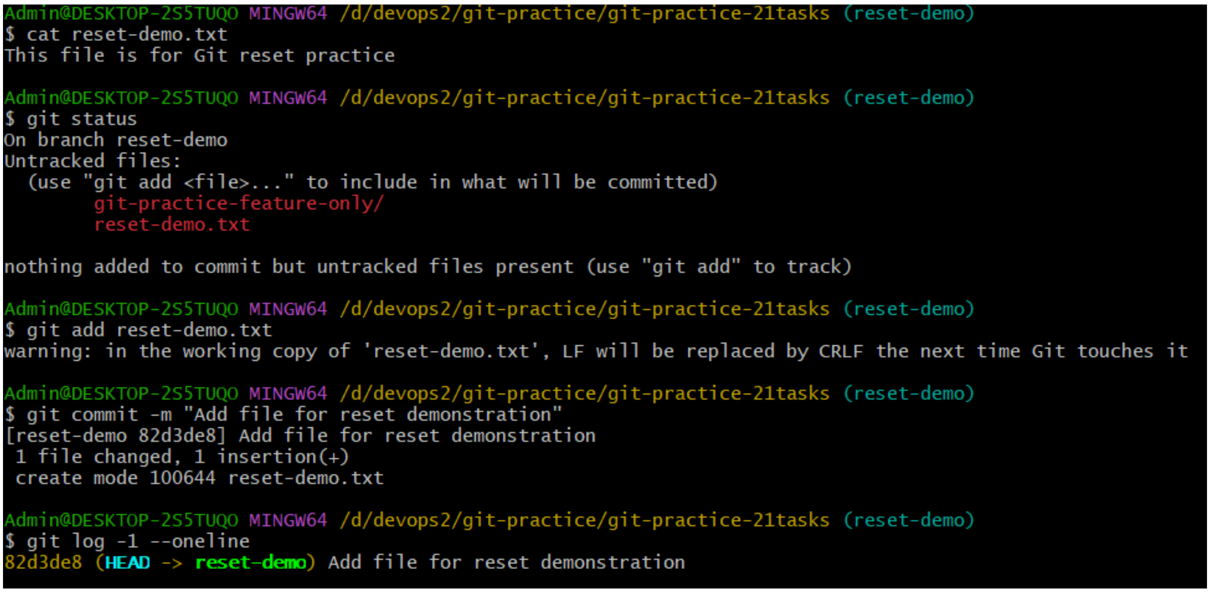
git commit -m "Add file for reset demonstration"

Verify the latest commit:

git log -1 --oneline

Record the commit hash displayed in the output.

Screenshot Evidence



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (reset-demo)
$ cat reset-demo.txt
This file is for Git reset practice

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (reset-demo)
$ git status
On branch reset-demo
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        git-practice-feature-only/
        reset-demo.txt

nothing added to commit but untracked files present (use "git add" to track)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (reset-demo)
$ git add reset-demo.txt
warning: in the working copy of 'reset-demo.txt', LF will be replaced by CRLF the next time Git touches it

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (reset-demo)
$ git commit -m "Add file for reset demonstration"
[reset-demo 82d3de8] Add file for reset demonstration
 1 file changed, 1 insertion(+)
 create mode 100644 reset-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (reset-demo)
$ git log -1 --oneline
82d3de8 (HEAD -> reset-demo) Add file for reset demonstration
```

Step 5: Reset the Latest Commit Without Saving Its Changes

Execute:

git reset --hard HEAD~1

This resets the current branch to the previous commit and discards the changes introduced by the latest commit from the working directory and staging area.

Important: Run this command only after confirming that reset-demo is the current branch and the test commit is the latest commit.

Step 6: Verify the Reset

Check the latest commit:

git log -1 --oneline

Check the working tree:

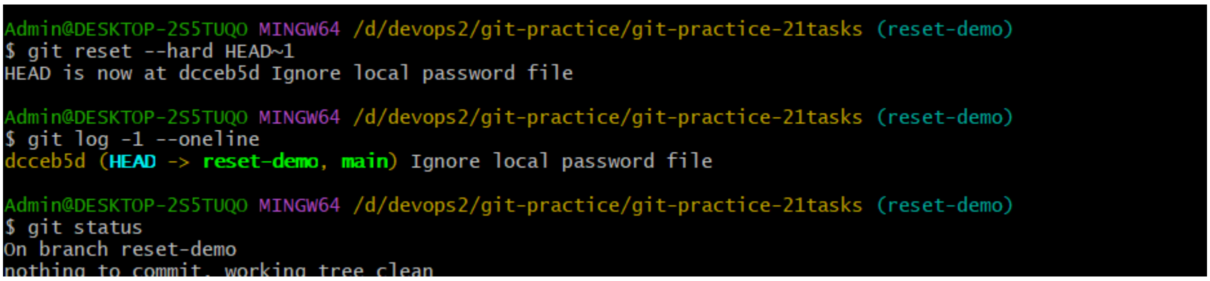
git status

Check whether the test file still exists:

ls reset-demo.txt

If the reset was successful, the latest commit should now be the commit that existed before the test commit. The test file should no longer exist because it was created and committed after that previous commit.

Screenshot Evidence



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (reset-demo)
$ git reset --hard HEAD~1
HEAD is now at dcceb5d Ignore local password file

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (reset-demo)
$ git log -1 --oneline
dcceb5d (HEAD -> reset-demo, main) Ignore local password file

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (reset-demo)
$ git status
On branch reset-demo
nothing to commit, working tree clean
```

Validation Steps

Verify the following:

1. The current branch is reset-demo.
2. The test commit no longer appears as the branch's latest commit.
3. The test file is absent from the working directory.
4. The working tree is clean.

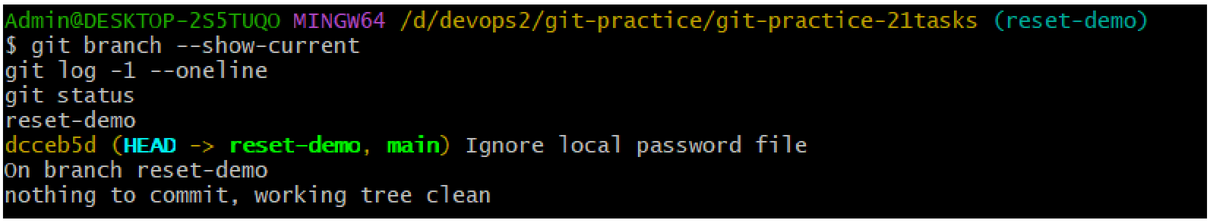
Use:

git branch --show-current

git log -1 --oneline

git status

Validation Screenshot



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (reset-demo)
$ git branch --show-current
git log -1 --oneline
git status
reset-demo
dcceb5d (HEAD -> reset-demo, main) Ignore local password file
On branch reset-demo
nothing to commit, working tree clean
```

Explanation

git reset --hard HEAD~1 moves the current branch pointer back by one commit. It also resets the staging area and working directory to match that earlier commit. As a result, the latest commit is removed from the current branch's visible history, and the changes introduced by that commit are discarded from the working directory.

This operation is different from git revert, which creates a new commit to undo the changes while preserving the existing history. A hard reset is best suited to controlled local work that has not been shared with others.

Real-Time Use Case

A developer may create a test commit locally and then discover that the commit should not be part of the branch. If the commit has not been shared and its changes are intentionally being discarded, a hard reset can return the branch to the previous state.

In collaborative projects, avoid resetting commits that have already been pushed and shared unless the team has agreed on the approach.

Common Errors and Troubleshooting

Error	Cause	Resolution
HEAD~1 does not exist	The repository has only one commit or no parent commit is available	Verify history with git log --oneline
Wrong commit was reset	The latest commit was not the intended test commit	Check the branch and commit history before resetting
Important changes disappeared	Hard reset discarded local changes	Check git reflog for recoverable commit references and recover carefully
Test file still exists	The file was not part of the commit being reset, or it existed earlier	Inspect the commit and file history
Push rejected after a reset	Local history differs from remote history	Avoid force-pushing shared branches; inspect the remote history and coordinate before proceeding

---

Conclusion

This task demonstrates how to create a commit and remove it from the current local branch using git reset --hard HEAD~1. The reset returns the branch, staging area, and working directory to the state of the previous commit.

The task is complete when the test commit is no longer the branch's latest commit, the test file has been removed, and the working tree is clean.

Task 14: Revert a Committed Change to an Older Version

Outcome

A sample file is created and committed to a local Git branch. The commit is then reverted using git revert, creating a new commit that reverses the original change and restores the previous state of the file.

Objective

To understand how to undo a previously committed change without deleting the original commit from Git history. The git revert command is useful when a change needs to be undone while preserving a clear record of what happened.

Prerequisites

- Git installed on Windows.

- Local Git repository available.
- Git Bash access.
- A clean working tree.
- A local test branch for the exercise.
- No important uncommitted changes.

Configuration

Parameter	Value
Repository	git-practice-21tasks
Test branch	revert-demo
Test file	revert-demo.txt
Original commit message	Add initial version for revert practice
Update commit message	Update file for revert practice
Revert command	git revert HEAD

Step-by-Step Implementation

Step 1: Open the Local Repository

Open Git Bash and navigate to the repository.

```
cd /d/devops2/git-practice/git-practice-21tasks
```

Check the current state:

```
git status
```

```
git log -1 --oneline
```

Before continuing, make sure there are no important uncommitted changes. If an unrelated untracked directory appears, inspect it and keep it outside the repository rather than staging it.

Step 2: Create a Local Test Branch

Create and switch to a dedicated branch:

```
git switch -c revert-demo
```

If the branch already exists, use:

```
git switch revert-demo
```

Verify the current branch:

```
git branch --show-current
```

Expected output:

revert-demo

Step 3: Create and Commit the Initial File

Create a sample file:

echo "Original version of the file" > revert-demo.txt

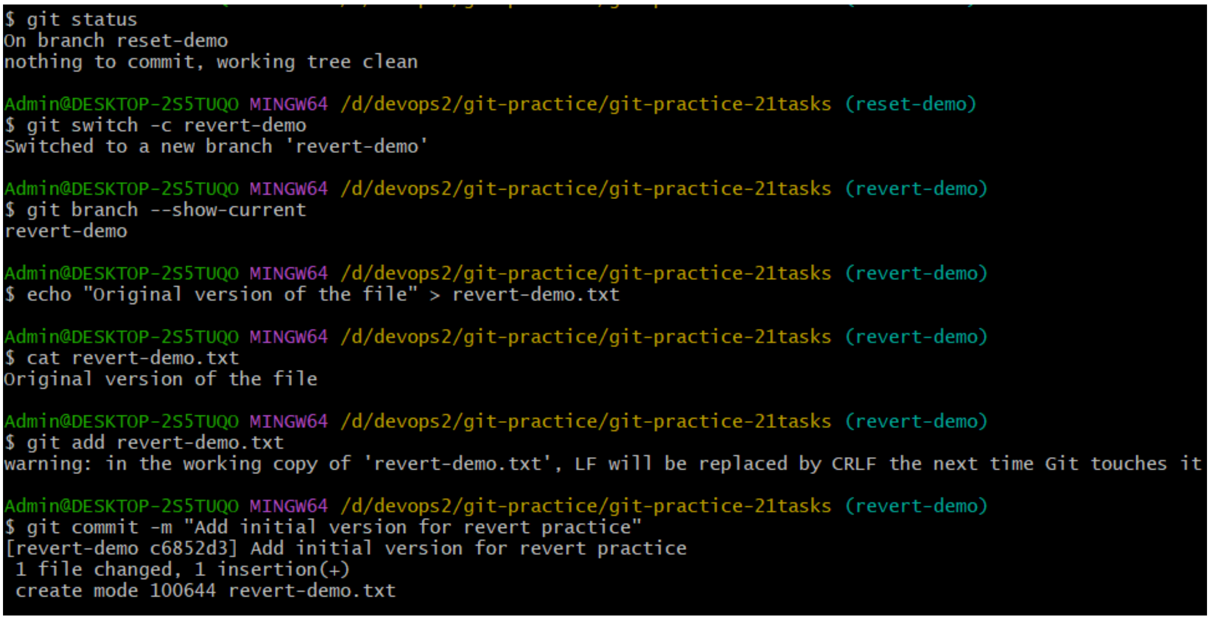
Verify its contents:

cat revert-demo.txt

Stage and commit the file:

git add revert-demo.txt

git commit -m "Add initial version for revert practice"



```
$ git status
On branch reset-demo
nothing to commit, working tree clean

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (reset-demo)
$ git switch -c revert-demo
Switched to a new branch 'revert-demo'

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ git branch --show-current
revert-demo

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ echo "Original version of the file" > revert-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ cat revert-demo.txt
Original version of the file

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ git add revert-demo.txt
warning: in the working copy of 'revert-demo.txt', LF will be replaced by CRLF the next time Git touches it

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ git commit -m "Add initial version for revert practice"
[revert-demo c6852d3] Add initial version for revert practice
 1 file changed, 1 insertion(+)
 create mode 100644 revert-demo.txt
```

Step 4: Modify the File and Commit the Update

Update the file:

echo "Updated version with changes" > revert-demo.txt

Verify the new contents:

cat revert-demo.txt

Commit the update:

git add revert-demo.txt

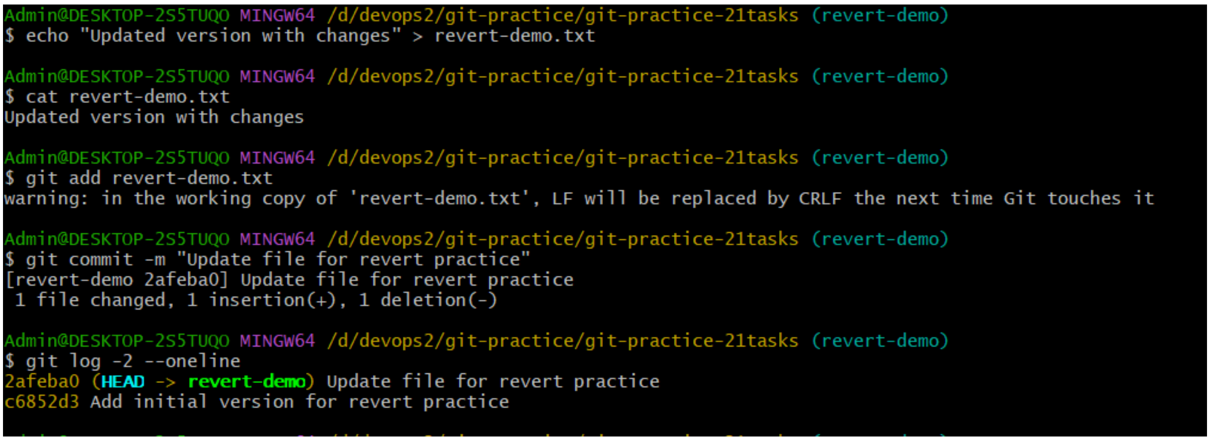
git commit -m "Update file for revert practice"

Verify the latest commit:

git log -2 --oneline

The latest commit should be Update file for revert practice.

Screenshot Evidence



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ echo "Updated version with changes" > revert-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ cat revert-demo.txt
Updated version with changes

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ git add revert-demo.txt
warning: in the working copy of 'revert-demo.txt', LF will be replaced by CRLF the next time Git touches it

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ git commit -m "Update file for revert practice"
[revert-demo 2afeba0] Update file for revert practice
 1 file changed, 1 insertion(+), 1 deletion(-)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ git log -2 --oneline
2afeba0 (HEAD -> revert-demo) Update file for revert practice
c6852d3 Add initial version for revert practice
```

Step 5: Revert the Latest Commit

Run:

git revert HEAD

Git may open a text editor for the revert commit message. Save and close the editor to complete the revert.

If Vim opens:

1. Press Esc.
2. Type :wq.
3. Press Enter.

The revert creates a new commit that reverses the changes introduced by the latest commit.

Step 6: Verify the Older File Version

Display the file:

cat revert-demo.txt

Expected output:

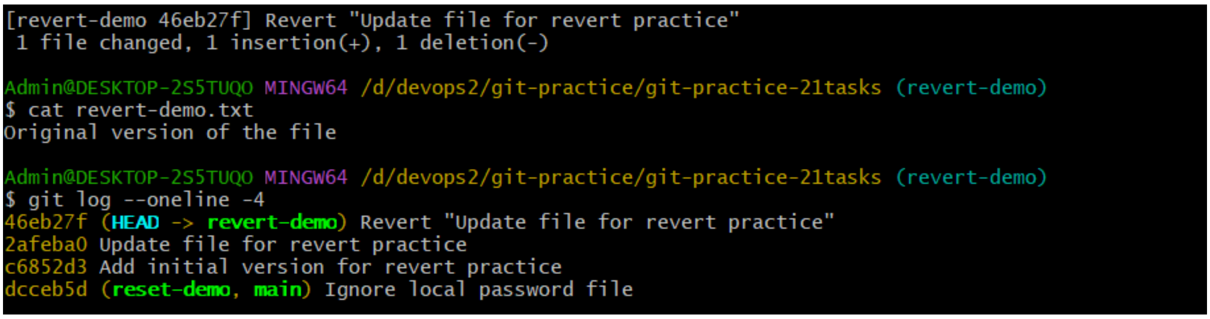
Original version of the file

Check the commit history:

git log --oneline -4

The history should show a new revert commit above the original update commit.

Screenshot Evidence



```
[revert-demo 46eb27f] Revert "Update file for revert practice"
 1 file changed, 1 insertion(+), 1 deletion(-)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ cat revert-demo.txt
Original version of the file

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ git log --oneline -4
46eb27f (HEAD -> revert-demo) Revert "Update file for revert practice"
2afeba0 Update file for revert practice
c6852d3 Add initial version for revert practice
dcceb5d (reset-demo, main) Ignore local password file
```

Validation Steps

Step 1: Verify the File Contents

cat revert-demo.txt

Confirm that the file contains:

Original version of the file

Step 2: Verify the Commit History

git log --oneline -4

Confirm that:

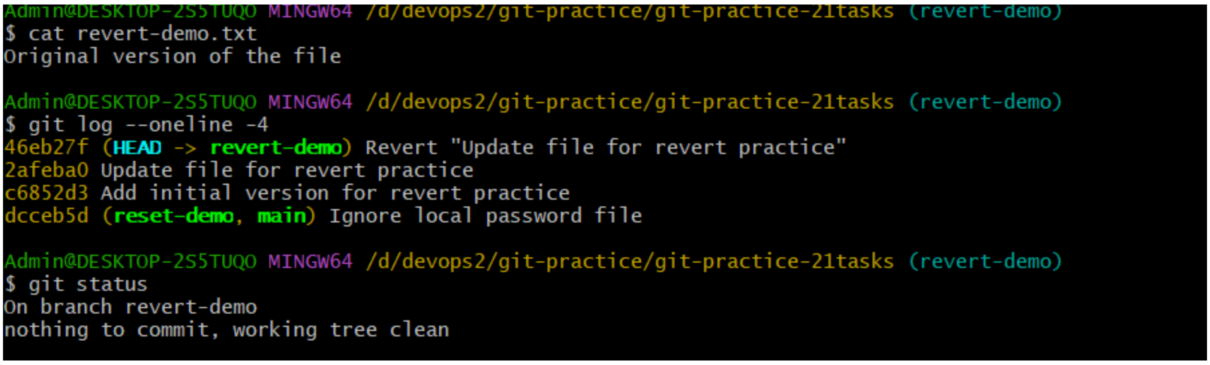
- The newest commit is the revert commit.
- The original update commit is still present in history.
- The initial version commit is also present.

Step 3: Verify the Working Tree

git status

The working tree should be clean after the revert completes.

Validation Screenshot



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ cat revert-demo.txt
Original version of the file

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ git log --oneline -4
46eb27f (HEAD -> revert-demo) Revert "Update file for revert practice"
2afeba0 Update file for revert practice
c6852d3 Add initial version for revert practice
dcceb5d (reset-demo, main) Ignore local password file

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ git status
On branch revert-demo
nothing to commit, working tree clean
```

Explanation

The git revert command undoes the changes introduced by a particular commit by creating a new commit. In this task, the latest update to revert-demo.txt was reverted, restoring the contents from the earlier commit.

Unlike git reset --hard, git revert does not remove the original commit from the branch history. Instead, it records the reversal as a separate commit. This makes it suitable for undoing changes on branches that have already been pushed or shared with other developers.

Real-Time Use Case

A developer may deploy a change that introduces a bug. Instead of rewriting shared Git history, the developer can revert the faulty commit and push the new revert commit. The team can then investigate the issue while preserving the original change and its history.

Common Errors and Troubleshooting

Error	Cause	Resolution
nothing to commit	No changes were made before attempting to revert	Confirm that the intended commit contains changes
your local changes would be overwritten	The working tree contains uncommitted changes	Commit or safely set aside the changes before reverting
Merge conflict during revert	Later commits changed the same lines	Resolve conflicts, stage the resolved files, and continue with git revert --continue
Wrong commit selected	The latest commit was not the intended change	Inspect git log --oneline and revert the correct commit hash
Editor does not close	Git opened an editor for the revert message	In Vim, press Esc, type :wq, and press Enter

---

Conclusion

This task demonstrates how to revert a committed change using git revert. A new commit reverses the most recent update and restores the previous file contents without removing the original update commit from history.

The task is complete when the file contains its earlier content, the Git history shows the original update and the new revert commit, and the working tree is clean.

Task 15: Stash Changes Without Committing and Work on Another File

Outcome

Changes made to one file are temporarily saved in Git's stash without creating a commit. The working directory is then used to create and commit another file, demonstrating how to pause one task and work on another.

Objective

To understand how git stash temporarily stores changes in the working directory so that a developer can switch to another task without committing incomplete work. This is useful when urgent work arrives before the current changes are ready to be committed.

Prerequisites

- Git installed on Windows.
- Local Git repository available.
- Git Bash access.
- A clean working tree before starting.
- A local branch for the exercise.
- No important uncommitted changes.

Configuration

Parameter	Value
Repository	git-practice-21tasks
Test branch	stash-demo
File with temporary changes	task-in-progress.txt
File for alternate work	urgent-task.txt
Stash command	git stash push -m "Save work in progress"

Step-by-Step Implementation

Step 1: Open the Local Repository

Open Git Bash and navigate to the repository.

```
cd /d/devops2/git-practice/git-practice-21tasks
```

Check the current working-tree status:

```
git status
```

Ensure there are no important uncommitted changes before starting.

Step 2: Create or Switch to a Test Branch

Create a dedicated local branch:

```
git switch -c stash-demo
```

If the branch already exists, switch to it instead:

```
git switch stash-demo
```

Verify the current branch:

git branch --show-current

Expected output:

stash-demo

Step 3: Create and Commit the Initial File

Create a file that will be modified later:

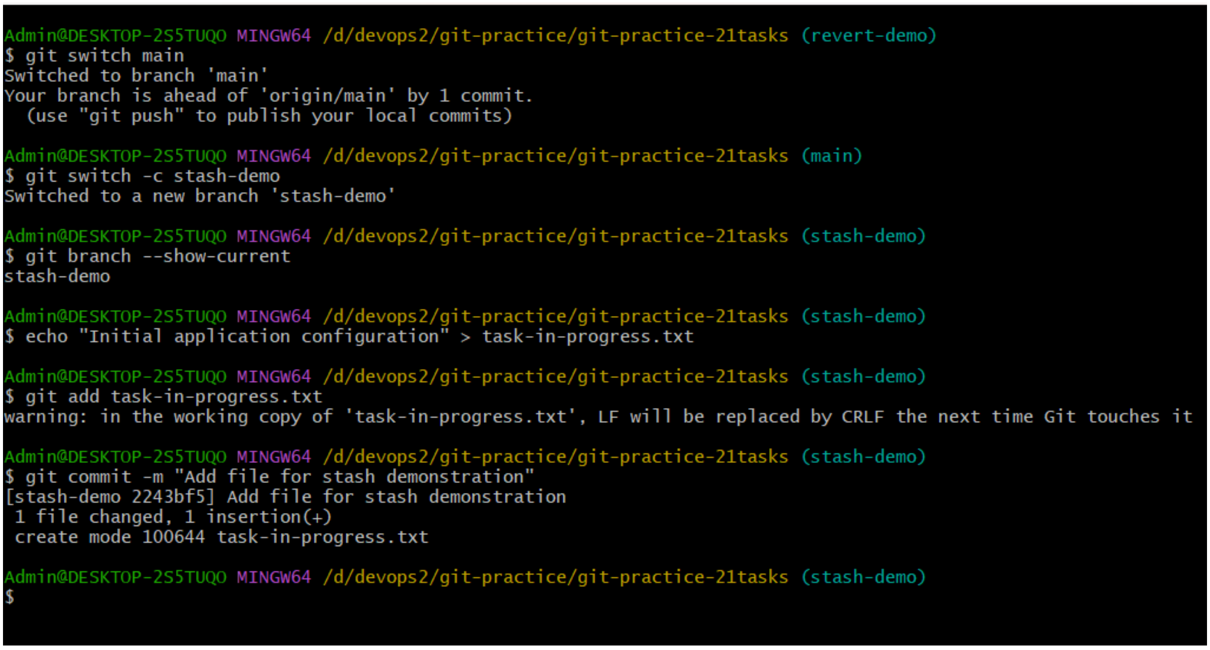
echo "Initial application configuration" > task-in-progress.txt

Stage and commit it:

git add task-in-progress.txt

git commit -m "Add file for stash demonstration"

Screenshot Evidence



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (revert-demo)
$ git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git switch -c stash-demo
Switched to a new branch 'stash-demo'

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git branch --show-current
stash-demo

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ echo "Initial application configuration" > task-in-progress.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git add task-in-progress.txt
warning: in the working copy of 'task-in-progress.txt', LF will be replaced by CRLF the next time Git touches it

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git commit -m "Add file for stash demonstration"
[stash-demo 2243bf5] Add file for stash demonstration
 1 file changed, 1 insertion(+)
 create mode 100644 task-in-progress.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$
```

Step 4: Modify the File Without Committing

Append a change to the file:

echo "Temporary configuration change" >> task-in-progress.txt

Check its contents:

cat task-in-progress.txt

Check the working-tree status:

git status

The file should appear as modified.

Step 5: Stash the Changes

Save the modification temporarily:

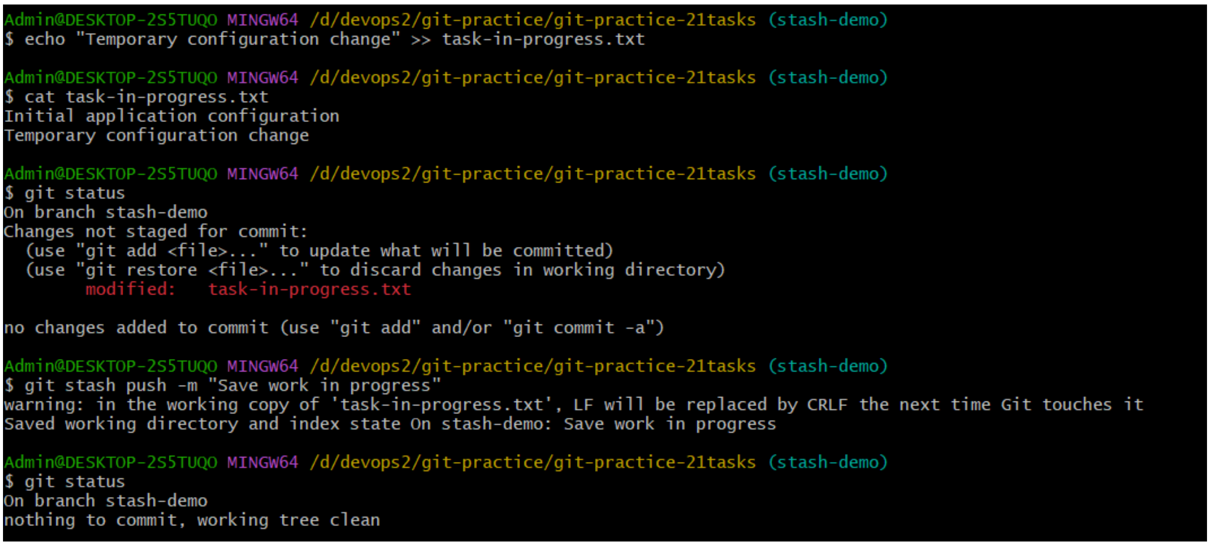
git stash push -m "Save work in progress"

Verify the working-tree status:

git status

The modification should no longer appear in the working tree because it has been saved in the stash.

Screenshot Evidence



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ echo "Temporary configuration change" >> task-in-progress.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ cat task-in-progress.txt
Initial application configuration
Temporary configuration change

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git status
On branch stash-demo
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   task-in-progress.txt

no changes added to commit (use "git add" and/or "git commit -a")

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git stash push -m "Save work in progress"
warning: in the working copy of 'task-in-progress.txt', LF will be replaced by CRLF the next time Git touches it
Saved working directory and index state On stash-demo: Save work in progress

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git status
On branch stash-demo
nothing to commit, working tree clean
```

Step 6: Verify the Stash

Run:

git stash list

Expected output will resemble:

stash@{0}: On stash-demo: Save work in progress

The stash reference may differ if other stashes already exist.

Step 7: Work on Another File

Create a separate file to represent the alternate task:

echo "Urgent task completed" > urgent-task.txt

Stage and commit the alternate work:

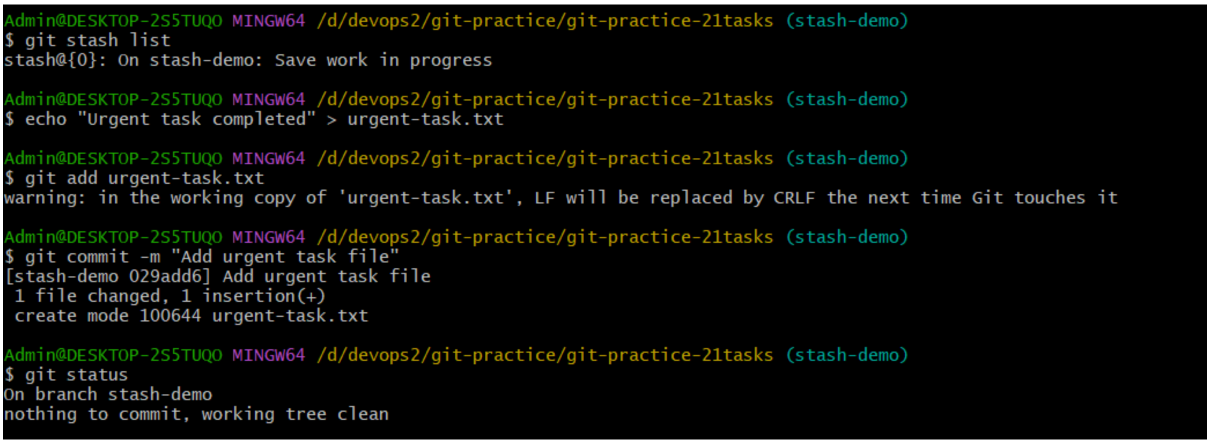
git add urgent-task.txt

git commit -m "Add urgent task file"

Check the repository status:

git status

Screenshot Evidence



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git stash list
stash@{0}: On stash-demo: Save work in progress

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ echo "Urgent task completed" > urgent-task.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git add urgent-task.txt
warning: in the working copy of 'urgent-task.txt', LF will be replaced by CRLF the next time Git touches it

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git commit -m "Add urgent task file"
[stash-demo 029add6] Add urgent task file
 1 file changed, 1 insertion(+)
 create mode 100644 urgent-task.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git status
On branch stash-demo
nothing to commit, working tree clean
```

Validation Steps

Step 1: Confirm the Stash Still Exists

git stash list

Verify that the stash entry named Save work in progress remains available.

Step 2: Confirm the Alternate File Was Committed

git log -1 --oneline

The latest commit should show:

Add urgent task file

Step 3: Confirm the Original Modification Is Stashed

Check the current contents:

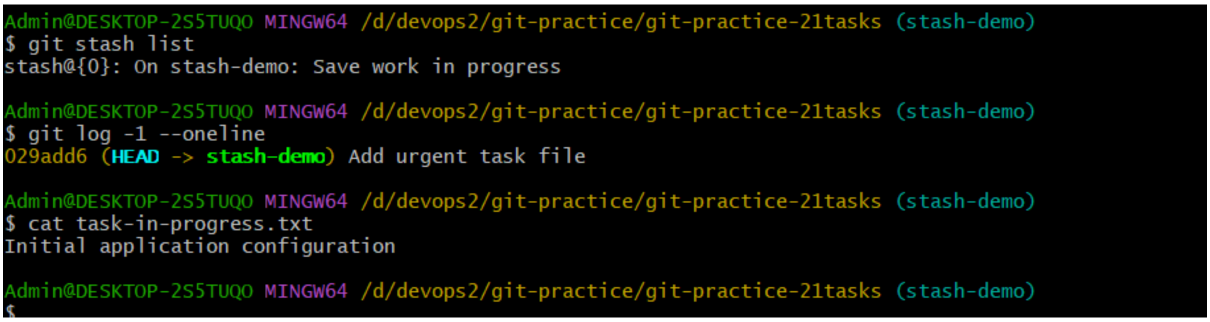
cat task-in-progress.txt

It should show only the original committed line:

Initial application configuration

The temporary modification should be available in the stash rather than in the working file.

Validation Screenshot



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git stash list
stash@{0}: On stash-demo: Save work in progress

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git log -1 --oneline
029add6 (HEAD -> stash-demo) Add urgent task file

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ cat task-in-progress.txt
Initial application configuration

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$
```

Explanation

The git stash command temporarily saves changes from the working directory and staging area without creating a normal commit. After stashing, the working directory returns to the state of the current commit, allowing the developer to work on another task.

In this exercise, changes to task-in-progress.txt were stashed, while urgent-task.txt was created and committed separately. The original work remains available in the stash for later restoration.

By default, git stash handles modifications to tracked files. Newly created untracked files are not included unless options such as git stash -u are used. This exercise stashes a modification to a file that was already committed.

Real-Time Use Case

A developer is working on a feature when an urgent production issue needs attention. Instead of committing incomplete feature work, the developer stashes the current changes, handles the urgent task, and later restores the original changes.

Common Errors and Troubleshooting

Error	Cause	Resolution
No local changes to save	No eligible changes exist in tracked files	Modify a tracked file and retry
Changes remain after stashing	The changes may be in an untracked file	Use git stash -u if untracked files must also be stashed
Stash entry not visible	Stash command may not have succeeded	Run git stash list and inspect the output
Conflicts when restoring a stash	The same lines changed in the meantime	Resolve the conflicts before completing restoration
Wrong stash selected	Multiple stash entries exist	Inspect git stash list and use the correct stash reference

---

Conclusion

This task demonstrates how to temporarily save changes using git stash and continue working on another file without committing the incomplete work. The stash preserves the original changes for later restoration, while the alternate task can be committed independently.

The task is complete when the original modification appears in git stash list, the alternate file has been committed, and the working tree is clean.

Task 16: Restore Stashed Changes and Resume Working on the File

Outcome

The previously stashed changes to task-in-progress.txt are successfully restored to the working directory using git stash pop. The developer can continue working on the file without losing the changes saved during Task 15.

Objective

To understand how to retrieve previously stashed changes using Git and resume incomplete work. This task demonstrates how git stash pop restores saved modifications and removes the stash entry when the operation completes successfully.

Prerequisites

- Git installed on Windows 11.
- Git Bash available.
- Local Git repository cloned.
- Task 15 completed successfully.
- Existing stash-demo branch.
- Changes previously saved using git stash.
- Clean working tree before restoring the stash.

Configuration

Parameter	Value
Repository	git-practice-21tasks
Local path	/d/devops2/git-practice/git-practice-21tasks
Branch	stash-demo
File to restore	task-in-progress.txt
Stash description	Save work in progress
Restore command	git stash pop
Expected result	Previous modifications restored

Step-by-Step Implementation

Step 1: Open the Local Repository

Open Git Bash and navigate to the repository.

```
cd /d/devops2/git-practice/git-practice-21tasks
```

Verify the directory:

```
pwd
```

Step 2: Switch to the Stash Demo Branch

Switch to the branch used in Task 15.

```
git switch stash-demo
```

Verify the current branch:

git branch --show-current

Expected output:

stash-demo

Check the working tree:

git status

Ensure there are no unrelated uncommitted changes that could interfere with restoring the stash.

Step 3: Check the Available Stash Entries

List the saved stash entries:

git stash list

Expected output:

stash@{0}: On stash-demo: Save work in progress

The stash reference may be different if multiple stash entries exist.

Step 4: Verify the File Before Restoring

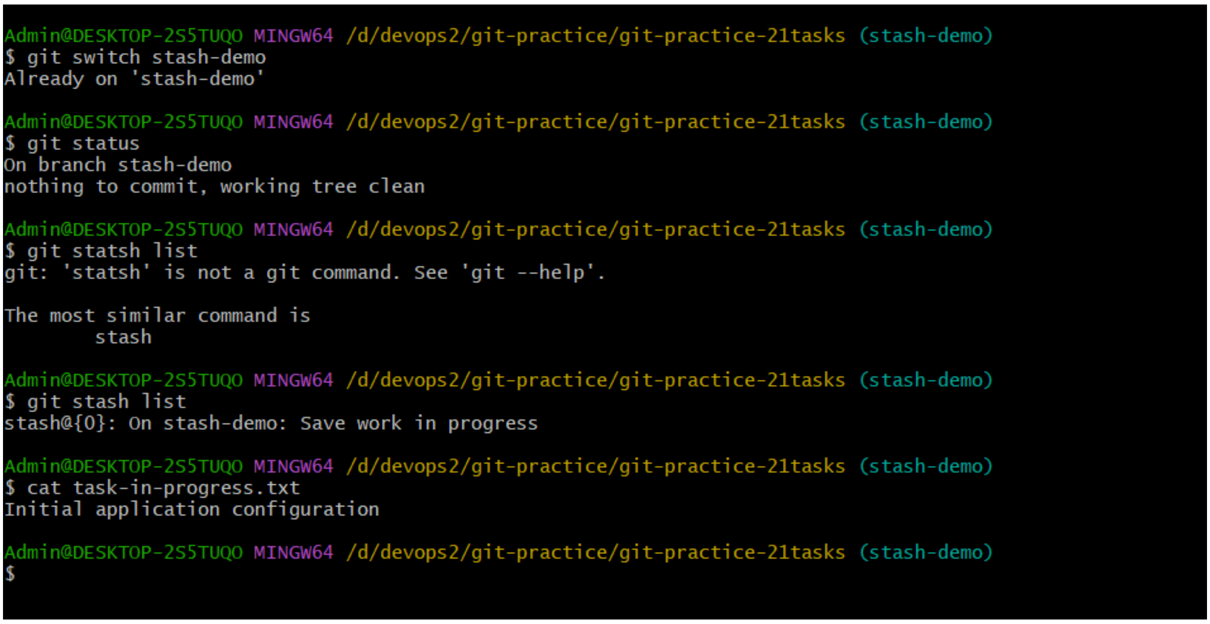
Display the current file contents:

cat task-in-progress.txt

Expected output:

Initial application configuration

The temporary modification from Task 15 is not currently visible because it was saved in the stash.



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git switch stash-demo
Already on 'stash-demo'

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git status
On branch stash-demo
nothing to commit, working tree clean

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git statsh list
git: 'statsh' is not a git command. See 'git --help'.

The most similar command is
        stash

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git stash list
stash@{0}: On stash-demo: Save work in progress

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ cat task-in-progress.txt
Initial application configuration

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$
```

Step 5: Restore the Stashed Changes

Restore the latest stash entry:

```
git stash pop
```

If the stash from Task 15 is not the latest entry, identify its reference using git stash list and restore the appropriate entry, for example:

```
git stash pop 'stash@{1}'
```

The command reapplies the saved modifications to the working directory. If restoration completes successfully, Git removes that stash entry.

Step 6: Verify the Restored File

Display the file contents:

```
cat task-in-progress.txt
```

Expected output:

```
Initial application configuration

Temporary configuration change
```

The temporary change saved in Task 15 is now restored.

Check Git status:

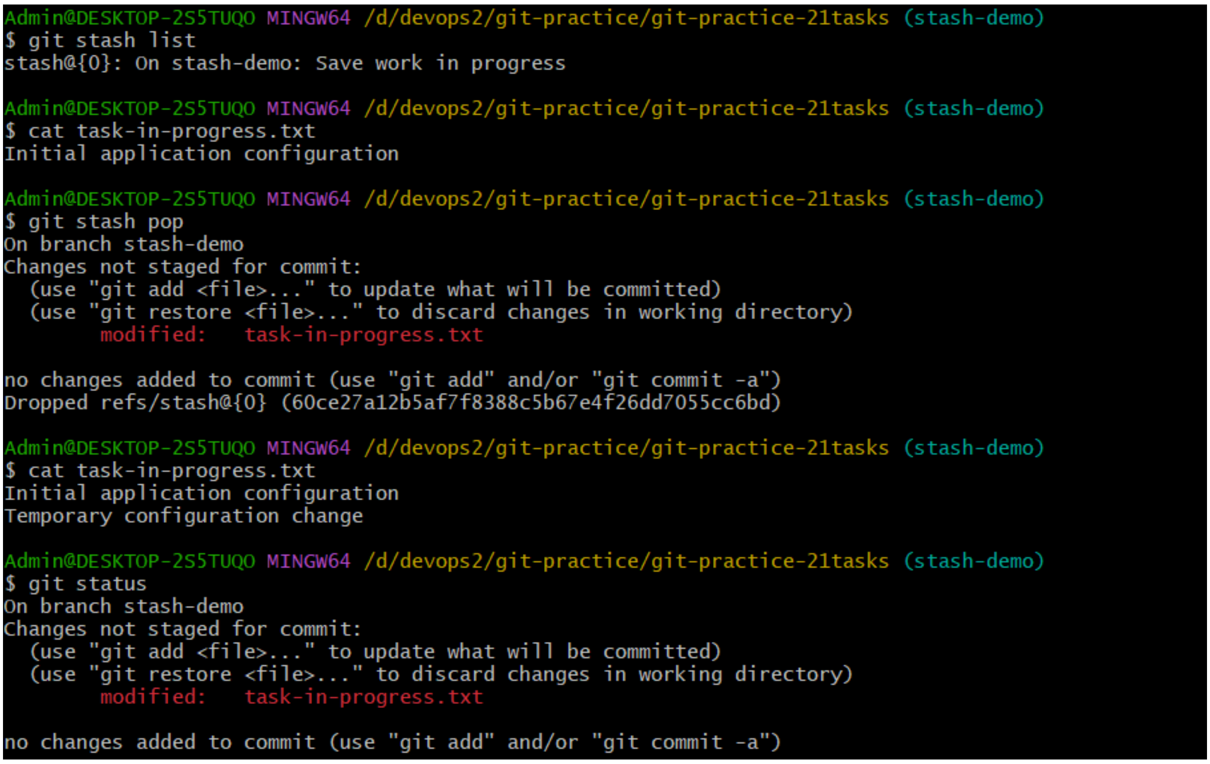
```
git status
```

Expected status includes:

```
modified: task-in-progress.txt
```

This indicates that the restored change is available in the working directory but has not yet been committed.

Screenshot Evidence



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git stash list
stash@{0}: On stash-demo: Save work in progress

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ cat task-in-progress.txt
Initial application configuration

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git stash pop
On branch stash-demo
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   task-in-progress.txt

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (60ce27a12b5af7f8388c5b67e4f26dd7055cc6bd)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ cat task-in-progress.txt
Initial application configuration
Temporary configuration change

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git status
On branch stash-demo
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   task-in-progress.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Step 7: Verify the Stash Entry Was Removed

Run:

git stash list

If the Task 15 stash was the only stash entry and restoration completed successfully, no entries should appear.

If other stash entries existed, they will remain in the list.

Step 8: Resume Working on the File

Append another line to demonstrate continued development:

echo "Development resumed after stash restoration" >> task-in-progress.txt

Verify the updated contents:

cat task-in-progress.txt

Expected output:

Initial application configuration

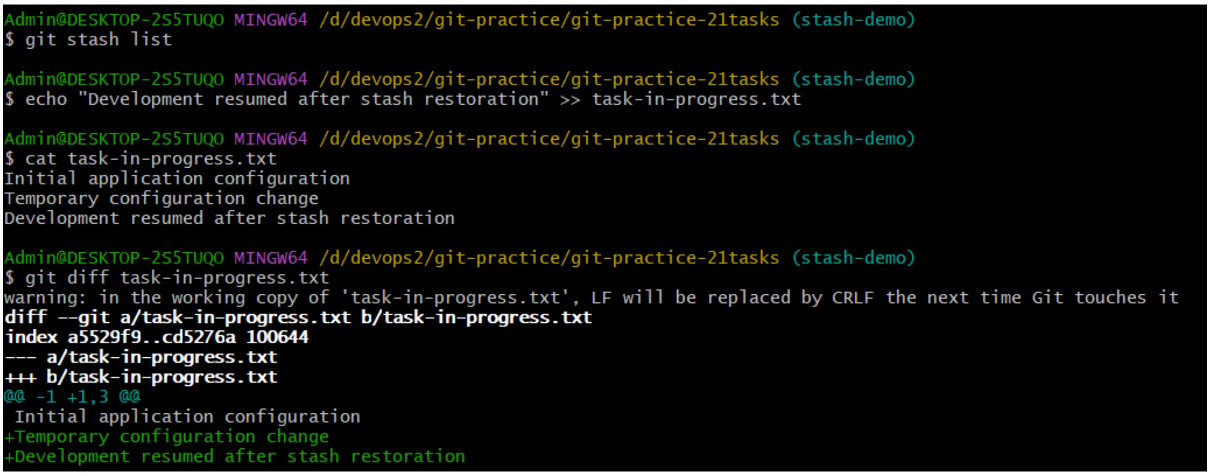
Temporary configuration change

Development resumed after stash restoration

Check the differences:

git diff task-in-progress.txt

This displays the restored modification and the newly added line compared with the last committed version.



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git stash list

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ echo "Development resumed after stash restoration" >> task-in-progress.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ cat task-in-progress.txt
Initial application configuration
Temporary configuration change
Development resumed after stash restoration

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git diff task-in-progress.txt
warning: in the working copy of 'task-in-progress.txt', LF will be replaced by CRLF the next time Git touches it
diff --git a/task-in-progress.txt b/task-in-progress.txt
index a5529f9..cd5276a 100644
--- a/task-in-progress.txt
+++ b/task-in-progress.txt
@@ -1 +1,3 @@
 Initial application configuration
+Temporary configuration change
+Development resumed after stash restoration
```

Validation Steps

Step 1: Verify the Restored Changes

Run:

cat task-in-progress.txt

Confirm that the file contains both the previously stashed changes and the new changes made after restoration.

Step 2: Verify Git Status

Run:

git status

The file should appear as modified.

Step 3: Verify the Stash Entry

Run:

git stash list

Confirm that the restored stash entry has been removed following a successful git stash pop.

Step 4: Verify the File Differences

Run:

git diff task-in-progress.txt

Confirm that Git displays the uncommitted modifications.

Explanation

The git stash command temporarily saves uncommitted changes so that developers can work on another task without committing incomplete work. In Task 15, the changes made to task-in-progress.txt were saved in a stash while another file was created and committed.

In Task 16, git stash pop restores those saved changes to the working directory. Once the stash is successfully applied, the corresponding stash entry is removed. The restored modifications remain uncommitted, allowing the developer to continue editing the file before deciding when to commit it.

An important distinction is that git stash apply restores changes while keeping the stash entry, whereas git stash pop restores changes and removes the entry after successful application.

Real-Time Use Case

A DevOps engineer may be modifying a Jenkins pipeline or deployment configuration when an urgent production issue requires immediate attention.

The engineer can stash the unfinished configuration changes, resolve the urgent issue, and then use git stash pop to restore the earlier work. This avoids creating unnecessary commits for incomplete changes and allows development to continue from where it was paused.

Common Errors and Troubleshooting

Error	Cause	Resolution
No stash entries found	No saved stash exists	Check git stash list and verify Task 15 was completed
Your local changes would be overwritten	Existing working-tree changes conflict with the stash	Commit or safely set aside existing changes before restoring
Merge conflict during stash pop	Saved changes conflict with current file contents	Resolve conflicts manually and stage the resolved files when appropriate
Wrong changes restored	An incorrect stash entry was selected	Review git stash list and identify the correct stash
Stash entry still exists	Restoration encountered conflicts or did not complete successfully	Resolve the issue and verify the stash before deleting anything
File does not show expected changes	Incorrect stash or branch was used	Inspect stash contents using git stash show -p

---

Conclusion

This task demonstrates how to restore previously stashed changes using git stash pop and resume working on an incomplete file.

The saved changes from Task 15 are restored to task-in-progress.txt, the successful pop removes the corresponding stash entry, and further modifications can be made without losing the earlier work.

The task is complete when the file contains its restored changes, Git recognizes the file as modified, and the developer can continue working on it.

Task 17: Generate a ssh-keygen and configure into github.

The existing ED25519 SSH key was loaded into the SSH agent:

```
eval "$(ssh-agent -s)"
```

```
ssh-add ~/.ssh/id_ed25519
```

The screenshot showed the identity being added successfully.

Verify the loaded key:

```
ssh-add -l -E sha256
```

The fingerprint shown was:

```
SHA256:dUN2chC0v/+F8O4dJtv3T92Z34ZB29R3TMgdovZSVs0
```

The public key was displayed and copied with:

```
cat ~/.ssh/id_ed25519.pub
```

```
clip < ~/.ssh/id_ed25519.pub
```

The key was registered with the GitHub account. SSH authentication was then tested:

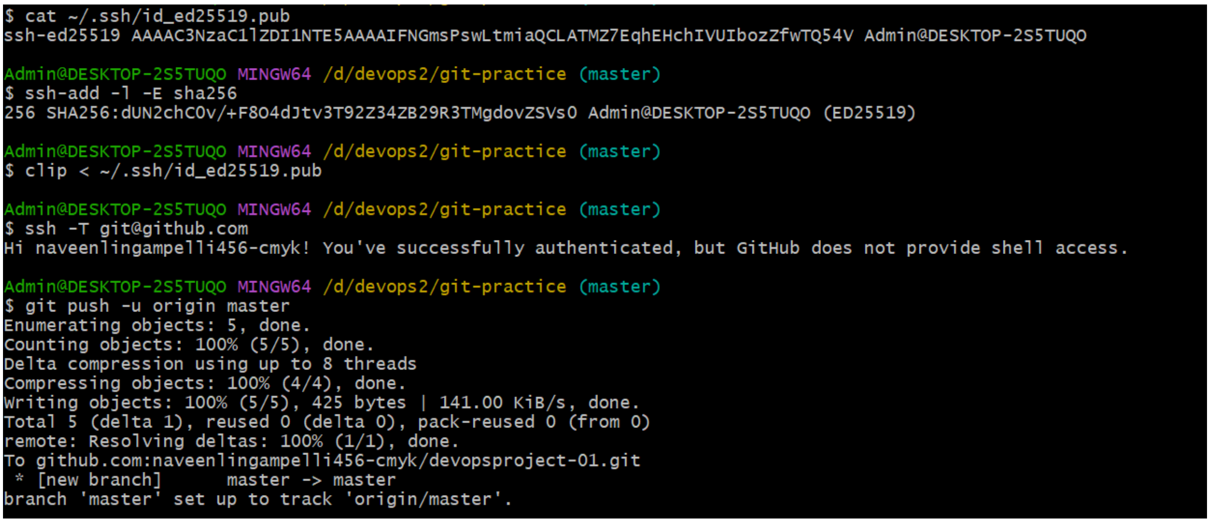
```
ssh -T git@github.com
```

The successful output was similar to:

```
Hi naveenlingampelli456-cmyk! You've successfully authenticated, but GitHub does not provide shell access.
```

Screenshot Evidence

Screenshot 10



```
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFNGmsPswLtmiaQCLATMZ7EqhEHchIVUIbozZfwTQ54V Admin@DESKTOP-2S5TUQO

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ ssh-add -l -E sha256
256 SHA256:dUN2chC0v/+F8O4dJtv3T92Z34ZB29R3TMgdovZSVs0 Admin@DESKTOP-2S5TUQO (ED25519)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ clip < ~/.ssh/id_ed25519.pub

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ ssh -T git@github.com
Hi naveenlingampelli456-cmyk! You've successfully authenticated, but GitHub does not provide shell access.

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git push -u origin master
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 425 bytes | 141.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To github.com:naveenlingampelli456-cmyk/devopsproject-01.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

Task 18: Configure Webhooks in GitHub

1. Outcome

Configure a repository webhook in GitHub to send notifications when selected repository events occur.

The webhook will be configured for the git-practice-21tasks repository. A push event will be selected to demonstrate how GitHub can notify an external service whenever new commits are pushed to the repository.

Repository: git-practice-21tasks

GitHub URL: https://github.com/naveenlingampelli456-cmyk/git-practice-21tasks

Platform: GitHub

Operating System: Windows 11

Note: A working webhook requires a valid, publicly reachable payload URL. This task documents the configuration process; successful delivery must be verified in GitHub's webhook delivery history.

2. Objective

The objective of this task is to understand and configure GitHub Webhooks.

A webhook is a mechanism that sends an HTTP request from GitHub to another application or server when a selected event occurs. Instead of repeatedly checking GitHub for changes, an external application can receive event notifications automatically.

Examples of webhook events include:

- Push events when code is pushed to a repository.
- Pull request events when a pull request is opened, updated, or merged.
- Issue events when issues are created or updated.
- Release events when a new release is published.

3. Prerequisites

The following prerequisites are required:

- A GitHub account.
- An existing GitHub repository.
- Repository owner or administrator access to configure webhooks.
- A valid payload URL for an external service or webhook receiver.
- An internet connection.
- A browser to access GitHub repository settings.

For a real integration, the receiving application must be accessible from GitHub over the internet. A local URL such as http://localhost:8080 is not directly reachable by GitHub.

4. Step-by-Step Implementation

Step 1: Open the GitHub Repository

1. Open a web browser.
2. Sign in to GitHub.
3. Navigate to the repository:

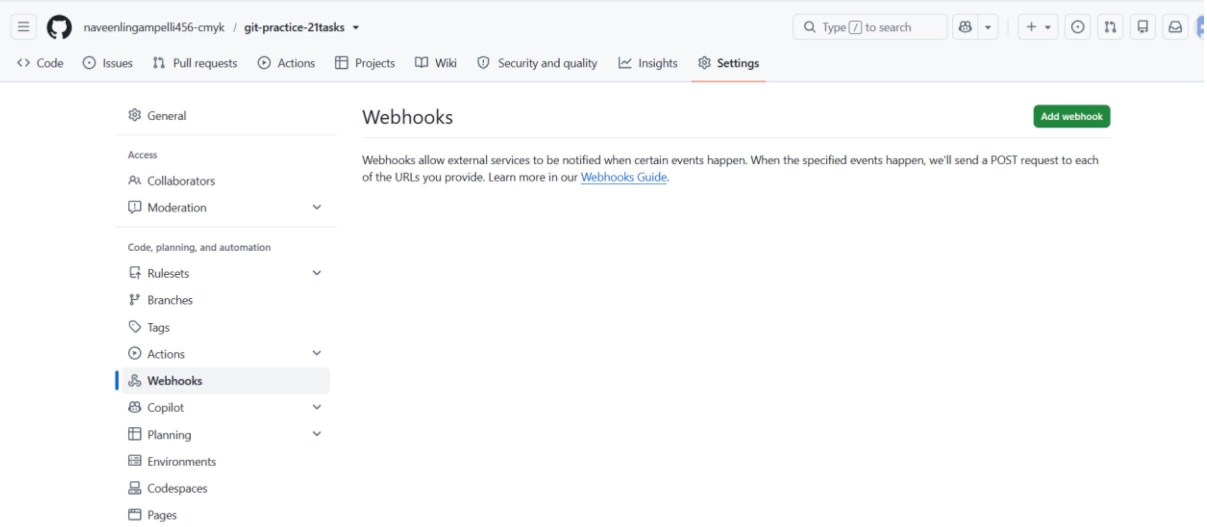
https://github.com/naveenlingampelli456-cmyk/git-practice-21tasks

4. Confirm that the repository opens successfully.

Step 2: Open Repository Settings

1. On the repository page, click Settings.
2. In the left-hand sidebar, locate and click Webhooks.
3. Click Add webhook.

You must have the required repository permissions to add a webhook.


naveenlingampelli456-cmyk / git-practice-21tasks
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
General
Access
Collaborators
Moderation
Code, planning, and automation
Rulesets
Branches
Tags
Actions
Webhooks
Copilot
Planning
Environments
Codespaces
Pages
Add webhook

Webhooks

Webhooks allow external services to be notified when certain events happen. When the specified events happen, we'll send a POST request to each of the URLs you provide. Learn more in our Webhooks Guide.

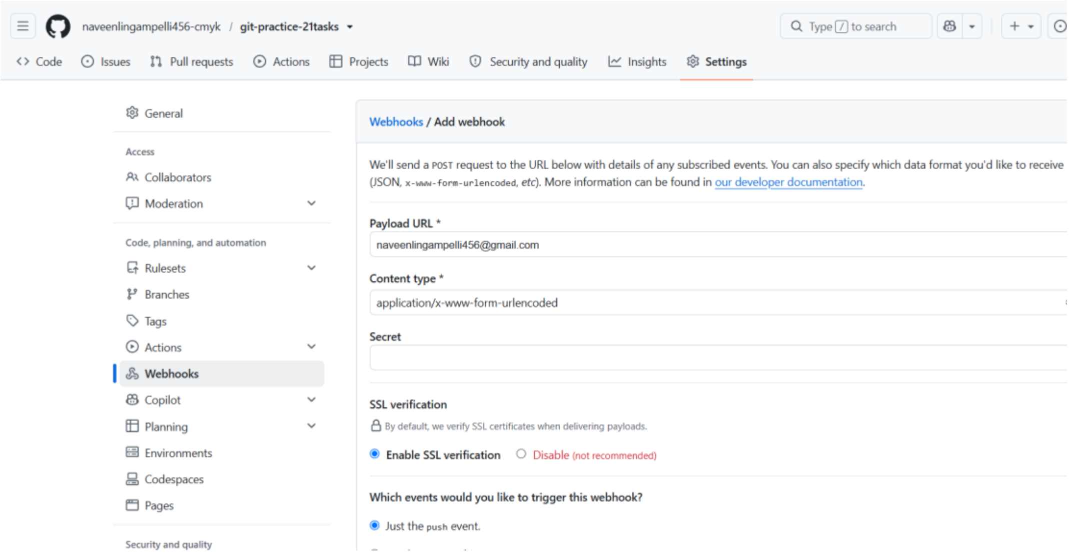
.

Step 3: Understand the Webhook Configuration Fields

GitHub displays a form containing the webhook configuration options.

Field	Description
Payload URL	The destination URL that receives webhook requests.
Content type	Defines the format used to send the payload.

Secret	An optional shared secret used by the receiving service to verify webhook signatures.
SSL verification	Controls certificate verification for HTTPS delivery. It should remain enabled.
Which events would you like to trigger this webhook?	Specifies the repository events that cause webhook deliveries.
Active	Enables or disables the webhook.


naveenlingampelli456-cmyk / git-practice-21tasks
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
General
Access
Collaborators
Moderation
Code, planning, and automation
Rulesets
Branches
Tags
Actions
Webhooks
Copilot
Planning
Environments
Codespaces
Pages
Security and quality

Webhooks / Add webhook

We'll send a `POST` request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, `x-www-form-urlencoded`, *etc*). More information can be found in our developer documentation.

Payload URL *
naveenlingampelli456@gmail.com

Content type *
application/x-www-form-urlencoded

Secret

SSL verification
By default, we verify SSL certificates when delivering payloads.
● Enable SSL verification ○ Disable (not recommended)

Which events would you like to trigger this webhook?
● Just the `push` event.

Step 4: Enter the Payload URL

In the Payload URL field, enter the HTTPS endpoint of the receiving application.

For example, if integrating with a properly configured Jenkins server, the endpoint commonly used for GitHub webhooks is:

https://YOUR_PUBLIC_JENKINS_HOST/github-webhook/

Replace YOUR_PUBLIC_JENKINS_HOST with the actual publicly reachable hostname of your Jenkins server. This example is applicable only when Jenkins is configured to receive GitHub webhooks.

For a generic webhook receiver, use the endpoint URL supplied by that service.

Important: Do not enter the example URL literally. Do not use a made-up URL or a local-only address as the payload URL. If no receiver is available, arrange a suitable test endpoint before attempting to verify delivery.

Step 5: Select the Content Type

From the Content type dropdown, select:

application/json

This setting sends the event payload as JSON in the body of the HTTP request. JSON is commonly used when integrating GitHub with automation tools and web services.

Step 6: Configure a Webhook Secret

In the Secret field, enter a strong, randomly generated secret if the receiving service supports signature verification.

The receiving application must use the same secret to validate the signature of incoming webhook deliveries.

For a secure production setup:

- Generate a unique, high-entropy secret.
- Store it securely.

- Configure the receiving service with the same secret.
- Never include the secret in screenshots, source code, or public documentation.

If this is a basic learning exercise and the receiver does not support secrets, follow the receiver's documentation. Do not assume that entering a secret alone secures the integration; the receiver must validate it.

Screenshot 6: Capture the secret field with the secret concealed or masked.

Step 7: Select the Webhook Events

Under Which events would you like to trigger this webhook?, select the events to receive.

For this exercise, select:

Just the push event

This means GitHub will send webhook notifications when changes are pushed to the repository.

Selecting only the necessary events keeps the integration focused and avoids unnecessary notifications.

Screenshot 7: Capture the selected push event.

Step 8: Enable SSL Verification and Activate the Webhook

1. Keep Enable SSL verification selected.
2. Ensure the Active checkbox is selected.
3. Review the Payload URL, content type, secret configuration, and selected events.
4. Click Add webhook.

GitHub should save the webhook configuration and send a ping event to the endpoint when the webhook is created. The ping event helps indicate that the webhook was created, but successful configuration should also be checked in the delivery history.

Screenshot 8: Capture the saved webhook details showing the configured URL, event selection, and active status. Do not reveal the secret.

Step 9: Verify Webhook Deliveries

1. Return to the repository's Settings → Webhooks page.
2. Select the configured webhook.
3. Locate the Recent Deliveries section.
4. Open the latest delivery.
5. Review the delivery status, request information, response status, and any error details.

For a successful delivery, look for a successful HTTP response from the receiving service. The exact response depends on the receiver; a 2xx response generally indicates that the HTTP request was accepted.

If the delivery fails, inspect the response and error details to identify whether the endpoint is unreachable, the service is unavailable, or the receiver rejected the request.

Screenshot 9: Capture a successful delivery in the Recent Deliveries section, if available.

Step 10: Test the Push Event

After the receiver is configured and the webhook is saved:

1. Open Git Bash.
2. Navigate to the local repository:

```
cd /d/devops2/git-practice/git-practice-21tasks
```

3. Create a small test file:

```
echo "Testing GitHub webhook" > webhook-test.txt
```

4. Stage and commit the file:

```
git add webhook-test.txt
git commit -m "Test GitHub webhook"
```

5. Push the commit to the repository's main branch:

```
git push origin main
```

Run this test only if pushing directly to main is allowed by the repository's rules. Otherwise, use an appropriate test branch and push that branch.

6. Return to Settings → Webhooks → Recent Deliveries.
7. Check whether GitHub generated a push delivery for the new commit.
8. Open the delivery and inspect the result.

Screenshot 10: Capture the successful Git push and the corresponding webhook delivery.

If the assignment does not require a test commit, the test file can be omitted. Avoid committing unrelated files or secrets.

5. Validation Steps

Use the following checklist to validate the configuration.

- The correct GitHub repository was opened.
- The Webhooks section is accessible.
- A valid payload URL was configured.
- The content type is set to application/json.
- A webhook secret was configured where supported and appropriate.
- Only the required events, such as push, were selected.
- SSL verification remains enabled.

- The webhook is active.
- The webhook appears in the repository's Webhooks settings.
- A ping or other test delivery appears in Recent Deliveries.
- The receiving service returned a successful response.
- A test push generated a corresponding webhook delivery, if the push test was performed.

A webhook appearing in the settings confirms that its configuration was saved. It does not, by itself, prove that the receiving service is reachable or processing events successfully.

6. Explanation of Important Concepts

What is a GitHub Webhook?

A GitHub Webhook automatically sends an HTTP request to a configured endpoint when a selected repository event occurs.

What is a Payload?

A payload is the data included in the webhook request. For a push event, it can include details about the repository, branch reference, commits, and pusher.

What is a Payload URL?

The Payload URL identifies the destination application that receives the webhook request. It must be accessible to GitHub for a real delivery to succeed.

What is a Webhook Secret?

A webhook secret helps the receiving application verify that a request was signed using the expected secret. The receiver must validate the signature; merely configuring a secret in GitHub is not sufficient.

What is a Push Event?

A push event is generated when commits are pushed to a repository. Subscribing to this event allows an external service to react to code changes.

What is a Ping Event?

GitHub sends a ping event when a webhook is created to help confirm that the configuration has been registered. It is not a substitute for testing the actual event your integration needs to process.

7. Real-Time Use Case

In a DevOps environment, a GitHub Webhook can be integrated with Jenkins to automate the CI/CD workflow.

For example:

1. A developer commits application changes.
2. The developer pushes the changes to GitHub.
3. GitHub sends a push-event webhook to Jenkins.

4. Jenkins receives the event and triggers the configured job or pipeline.
5. The pipeline checks out the source code and performs build, test, and deployment stages according to its configuration.

This approach reduces the need for Jenkins to repeatedly poll GitHub for changes and can help make software delivery more responsive.

Note: The webhook does not automatically create a complete CI/CD pipeline. Jenkins must also be configured with a suitable job or pipeline and the required GitHub integration.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
Webhook delivery fails	Payload URL is incorrect or unavailable.	Verify the URL and ensure the receiving application is running and reachable from the internet.
Connection refused	The destination service is not listening on the expected port or a firewall blocks access.	Check the service, port, firewall, and network configuration.
Connection timeout	Network routing, firewall rules, or an unavailable endpoint prevents the connection.	Check endpoint reachability and allow the required inbound traffic securely.
HTTP 404 Not Found	The configured path does not exist on the receiver.	Verify the complete endpoint path, including any required trailing slash.
HTTP 401 or 403	The receiver requires authentication or denies the request.	Review the receiver's authentication and access-control configuration.
HTTP 5xx response	The receiving application encountered an internal error.	Inspect the receiver's logs and correct the application or service issue.
Webhook saved but no push notification appears	The wrong events were selected, or no new push event occurred.	Confirm that the push event is enabled and push a test commit to the appropriate branch.
SSL certificate error	The HTTPS endpoint has an invalid, expired, or untrusted certificate.	Install or renew a valid certificate and keep SSL verification enabled.
Webhook secret validation fails	GitHub and the receiver use different secrets, or signature validation is implemented incorrectly.	Confirm that both sides use the same secret and follow the receiver's signature-validation instructions.
Jenkins does not start a build	The webhook may reach Jenkins, but job integration or trigger settings are missing.	Check the Jenkins job configuration, GitHub integration, plugin setup, and logs.

9. Security Best Practices

- Use HTTPS for webhook endpoints.
- Keep SSL verification enabled.
- Configure a strong webhook secret where supported.
- Validate webhook signatures in the receiving application.
- Subscribe only to the events required by the integration.
- Keep webhook secrets out of source control and screenshots.
- Restrict access to the receiving service and monitor delivery logs.
- Avoid exposing development servers directly to the public internet without appropriate security controls.

10. Conclusion

In this task, a GitHub repository webhook is configured to notify an external service when selected events occur. The push event is used as the example so that a Git push can trigger a webhook delivery.

The configuration is considered fully validated only when the webhook is saved, the endpoint is reachable, and a test delivery succeeds. A real integration with Jenkins or another application also requires the receiving service to be configured to process the event.

Documentation note: Replace the screenshot placeholders with actual screenshots from your implementation. If no payload receiver was available, document the configuration steps completed and clearly identify delivery testing as pending rather than claiming a successful integration.

Task 19: Basic Understanding of the .git Directory

1. Outcome

Understand the purpose and internal structure of the .git directory and learn how Git stores repository configuration, commit history, branches, and other version-control information.

By completing this task, we will inspect the .git directory in the local Git repository and explore important files and folders used by Git.

Environment: Windows 11 with Git Bash

Local Repository: git-practice-21tasks

Repository Path: /d/devops2/git-practice/git-practice-21tasks

2. Objective

The objective of this task is to understand how Git manages a repository internally.

When we initialize a Git repository using git init or clone an existing repository using git clone, Git creates a hidden directory named .git.

This directory contains the metadata and internal information Git needs to track changes, maintain commit history, manage branches, and communicate with remote repositories.

Understanding .git helps developers troubleshoot repository issues and understand how Git operations work internally.

3. Prerequisites

- Git must be installed on the system.
- Git Bash must be available.
- A local Git repository must exist.
- Basic knowledge of Git commands such as git status, git log, and git branch.

4. Step-by-Step Implementation

Step 1: Open Git Bash

Open Git Bash and navigate to the local repository.

```
cd /d/devops2/git-practice/git-practice-21tasks
```

Verify the current directory:

```
pwd
```

Expected output:

```
/d/devops2/git-practice/git-practice-21tasks
```

Step 2: Verify That the Repository Is a Git Repository

Run:

```
git status
```

Expected output will show the current branch and working-tree status. The exact output depends on whether there are uncommitted changes.

Now check the Git repository's root directory:

```
git rev-parse --show-toplevel
```

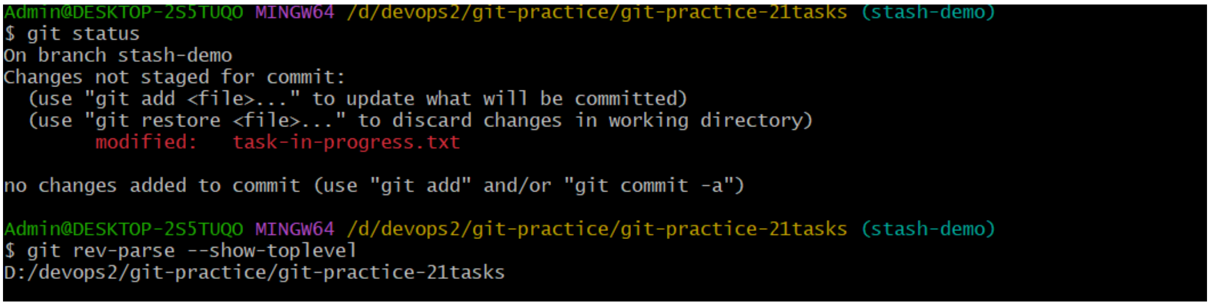
Expected output:

```
D:/devops2/git-practice/git-practice-21tasks
```

The path format may vary depending on your Git Bash configuration.

Explanation: The git rev-parse --show-toplevel command identifies the root directory of the current working tree.

Screenshot 2:



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git status
On branch stash-demo
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   task-in-progress.txt

no changes added to commit (use "git add" and/or "git commit -a")

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git rev-parse --show-toplevel
D:/devops2/git-practice/git-practice-21tasks
```

Step 3: Locate the .git Directory

Run:

ls -la

The -a option displays hidden entries, and -l displays detailed file information.

Look for the .git directory in the output.

You can also inspect it directly:

ls -la .git

Example output may contain:

HEAD

config

description

hooks/

index

info/

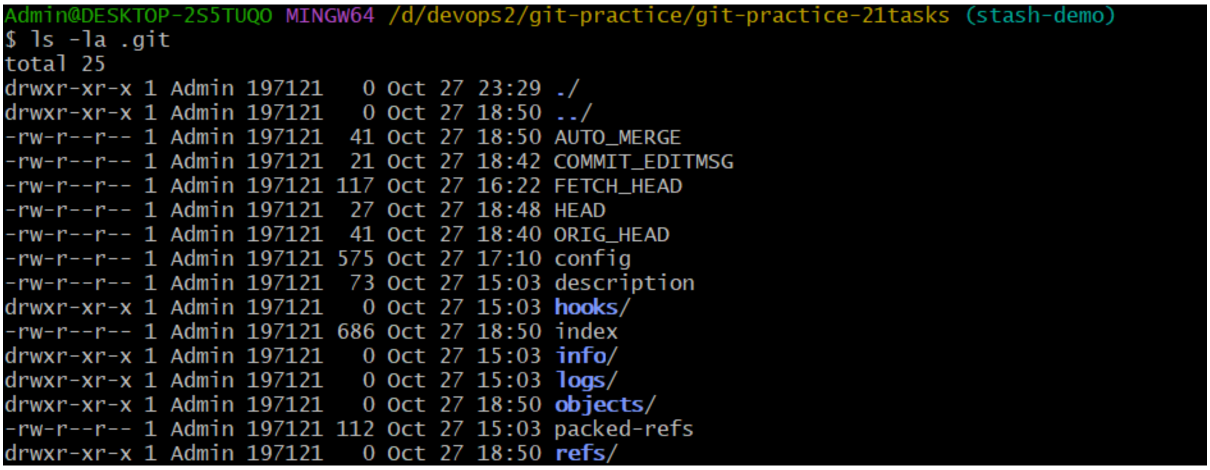
logs/

objects/

refs/

The exact contents may vary depending on the Git version, repository history, and configuration.

Screenshot 3:



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ ls -la .git
total 25
drwxr-xr-x 1 Admin 197121   0 Oct 27 23:29 ./
drwxr-xr-x 1 Admin 197121   0 Oct 27 18:50 ../
-rw-r--r-- 1 Admin 197121  41 Oct 27 18:50 AUTO_MERGE
-rw-r--r-- 1 Admin 197121  21 Oct 27 18:42 COMMIT_EDITMSG
-rw-r--r-- 1 Admin 197121 117 Oct 27 16:22 FETCH_HEAD
-rw-r--r-- 1 Admin 197121  27 Oct 27 18:48 HEAD
-rw-r--r-- 1 Admin 197121  41 Oct 27 18:40 ORIG_HEAD
-rw-r--r-- 1 Admin 197121 575 Oct 27 17:10 config
-rw-r--r-- 1 Admin 197121  73 Oct 27 15:03 description
drwxr-xr-x 1 Admin 197121   0 Oct 27 15:03 hooks/
-rw-r--r-- 1 Admin 197121 686 Oct 27 18:50 index
drwxr-xr-x 1 Admin 197121   0 Oct 27 15:03 info/
drwxr-xr-x 1 Admin 197121   0 Oct 27 15:03 logs/
drwxr-xr-x 1 Admin 197121   0 Oct 27 18:50 objects/
-rw-r--r-- 1 Admin 197121 112 Oct 27 15:03 packed-refs
drwxr-xr-x 1 Admin 197121   0 Oct 27 18:50 refs/
```

Step 4: Understand the Important Files and Folders

The following table describes common items found inside .git.

File or Directory	**Purpose**
HEAD	Identifies the current branch reference or, in a detached-HEAD state, a specific commit.
Config	Stores repository-specific Git configuration, including remote URLs.
Description	Contains a repository description, mainly used by some Git hosting tools.
Index	Stores the staging area's information about files prepared for the next commit.
objects/	Stores Git objects, including file contents, trees, and commits.
refs/	Contains references such as local branches and tags.
logs/	Stores reflog records when reflogs are enabled, documenting changes to references.
hooks/	Contains sample or configured scripts that can run during certain Git operations.
info/	Contains repository-specific information, including exclude, which can specify local ignore rules.

Not every item will be visible or populated in every repository. Some references may be stored in a packed-refs file instead of individual files under refs/.

Step 5: Inspect the HEAD File

Run:

cat .git/HEAD

Example output:

ref: refs/heads/main

This means HEAD points to the local main branch. The actual output depends on the currently checked-out branch.

To see the current branch, run:

```
git branch --show-current
```

Explanation: When a branch is checked out, HEAD normally points to that branch's reference. In a detached-HEAD state, HEAD can point directly to a commit instead.

Step 6: Inspect the Git Configuration File

Run:

```
cat .git/config
```

Example configuration:

```
[core]
    repositoryformatversion = 0
    filemode = false
    bare = false
    logallrefupdates = true

[remote "origin"]
    url = git@github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git
    fetch = +refs/heads/*:refs/remotes/origin/*
```

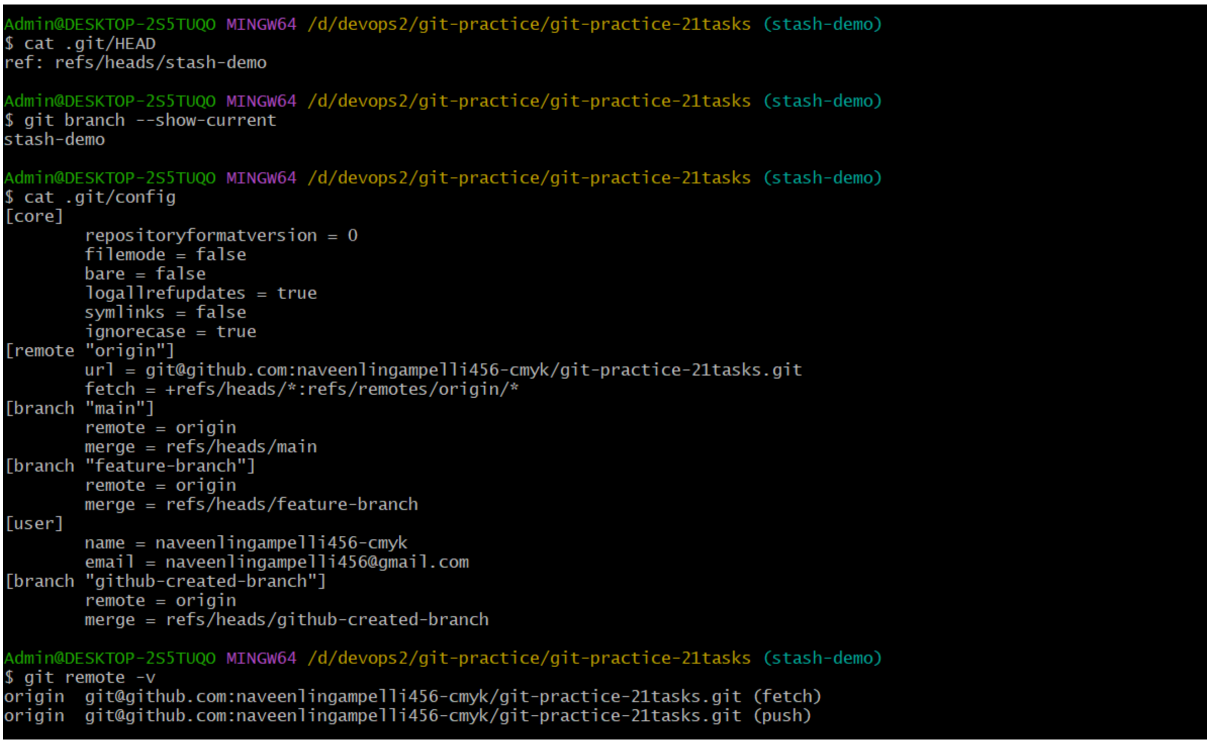
Your actual configuration may contain additional sections or different values.

Explanation: The .git/config file stores settings specific to this repository. In this example, the origin remote points to the GitHub repository using SSH.

To display the remote URL using a Git command, run:

```
git remote -v
```

Screenshot 6:



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ cat .git/HEAD
ref: refs/heads/stash-demo

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git branch --show-current
stash-demo

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ cat .git/config
[core]
        repositoryformatversion = 0
        filemode = false
        bare = false
        logallrefupdates = true
        symlinks = false
        ignorecase = true
[remote "origin"]
        url = git@github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git
        fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
        remote = origin
        merge = refs/heads/main
[branch "feature-branch"]
        remote = origin
        merge = refs/heads/feature-branch
[user]
        name = naveenlingampelli456-cmyk
        email = naveenlingampelli456@gmail.com
[branch "github-created-branch"]
        remote = origin
        merge = refs/heads/github-created-branch

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git remote -v
origin  git@github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git (fetch)
origin  git@github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git (push)
```

Step 7: Inspect Local Branch References

Run:

ls -la .git/refs

Then inspect the local branch reference directory:

ls -la .git/refs/heads

If the repository uses loose references, files such as main or feature-branch may be visible.

To display local branches using Git, run:

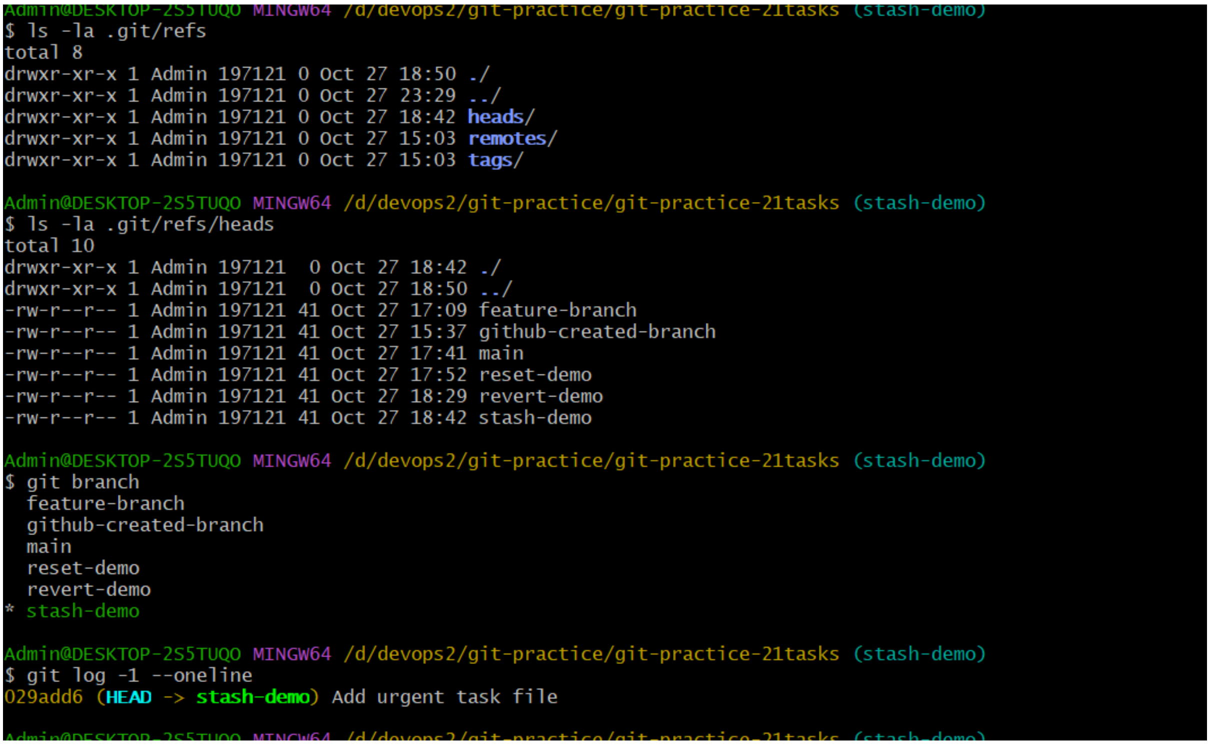
git branch

To inspect the current branch's commit, run:

git log -1 --oneline

Explanation: Branch references identify commits. Git may store references in packed form, so not every branch necessarily appears as an individual file under .git/refs/heads.

Screenshot 7:



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ ls -la .git/refs
total 8
drwxr-xr-x 1 Admin 197121 0 Oct 27 18:50 ./
drwxr-xr-x 1 Admin 197121 0 Oct 27 23:29 ../
drwxr-xr-x 1 Admin 197121 0 Oct 27 18:42 heads/
drwxr-xr-x 1 Admin 197121 0 Oct 27 15:03 remotes/
drwxr-xr-x 1 Admin 197121 0 Oct 27 15:03 tags/

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ ls -la .git/refs/heads
total 10
drwxr-xr-x 1 Admin 197121  0 Oct 27 18:42 ./
drwxr-xr-x 1 Admin 197121  0 Oct 27 18:50 ../
-rw-r--r-- 1 Admin 197121 41 Oct 27 17:09 feature-branch
-rw-r--r-- 1 Admin 197121 41 Oct 27 15:37 github-created-branch
-rw-r--r-- 1 Admin 197121 41 Oct 27 17:41 main
-rw-r--r-- 1 Admin 197121 41 Oct 27 17:52 reset-demo
-rw-r--r-- 1 Admin 197121 41 Oct 27 18:29 revert-demo
-rw-r--r-- 1 Admin 197121 41 Oct 27 18:42 stash-demo

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git branch
  feature-branch
  github-created-branch
  main
  reset-demo
  revert-demo
* stash-demo

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git log -1 --oneline
029add6 (HEAD -> stash-demo) Add urgent task file

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
```

.

Step 8: Understand Git Objects

Git stores repository data in the object database under .git/objects/, although objects can also be stored in packfiles.

The main Git object types are:

- **Blob:** Stores file contents.
- **Tree:** Stores directory contents and references to blobs or other trees.
- **Commit:** Stores information about a snapshot, including its tree, parent commits, author, committer, and message.
- **Tag:** Stores tag objects when annotated tags are used.

To inspect the latest commit, run:

git log -1 --oneline

To inspect the object type associated with a commit, copy its commit hash and run:

git cat-file -t COMMIT_HASH

Replace COMMIT_HASH with the actual hash from your repository.

For a commit object, the command returns:

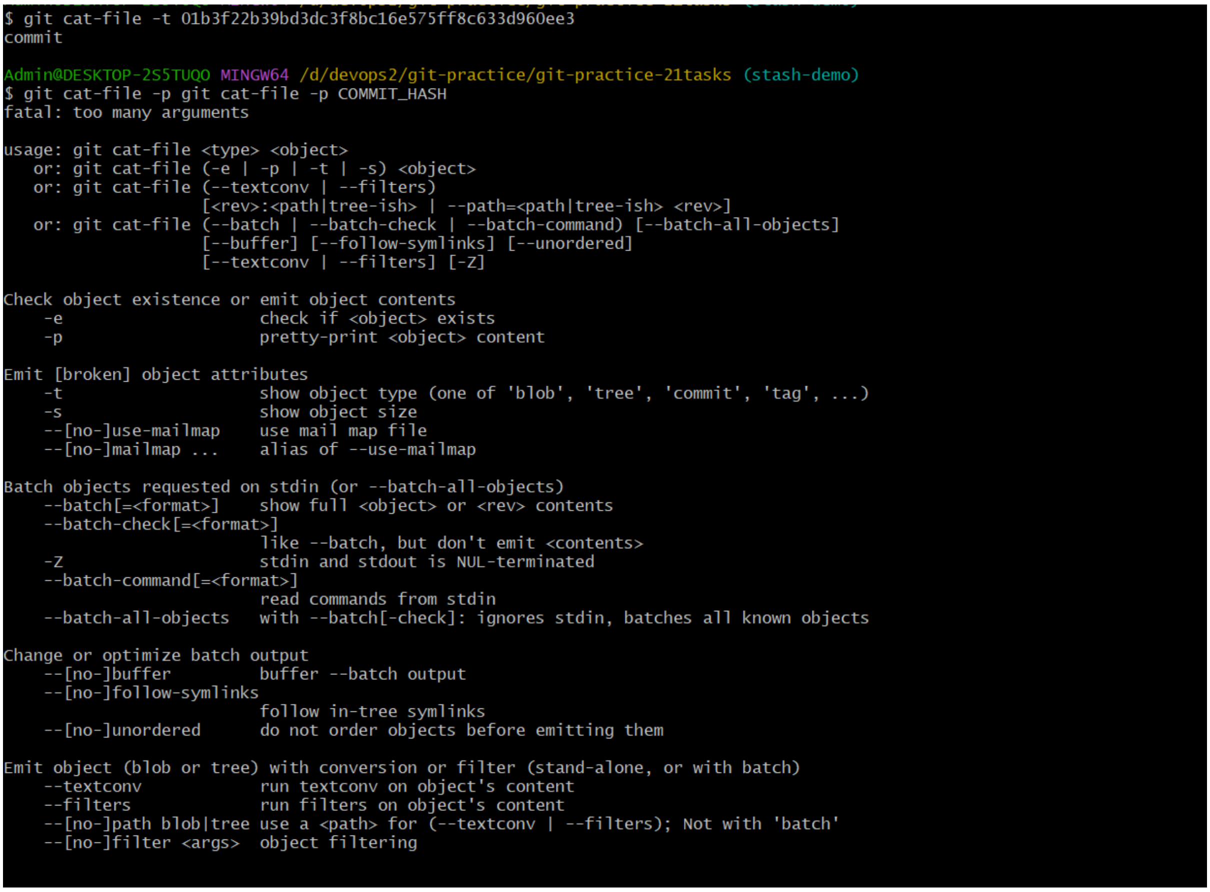
commit

To inspect the commit's metadata, run:

git cat-file -p COMMIT_HASH

Explanation: Git uses its object database to maintain snapshots and history. The git cat-file command lets us examine objects without manually interpreting the files under .git/objects/.

Screenshot 8:



```
$ git cat-file -t 01b3f22b39bd3dc3f8bc16e575ff8c633d960ee3
commit

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git cat-file -p git cat-file -p COMMIT_HASH
fatal: too many arguments

usage: git cat-file <type> <object>
   or: git cat-file (-e | -p | -t | -s) <object>
   or: git cat-file (--textconv | --filters)
                    [<rev>:<path|tree-ish> | --path=<path|tree-ish> <rev>]
   or: git cat-file (--batch | --batch-check | --batch-command) [--batch-all-objects]
                    [--buffer] [--follow-symlinks] [--unordered]
                    [--textconv | --filters] [-Z]

Check object existence or emit object contents
    -e                    check if <object> exists
    -p                    pretty-print <object> content

Emit [broken] object attributes
    -t                    show object type (one of 'blob', 'tree', 'commit', 'tag', ...)
    -s                    show object size
    --[no-]use-mailmap    use mail map file
    --[no-]mailmap ...    alias of --use-mailmap

Batch objects requested on stdin (or --batch-all-objects)
    --batch[=<format>]    show full <object> or <rev> contents
    --batch-check[=<format>]
                          like --batch, but don't emit <contents>
    -Z                    stdin and stdout is NUL-terminated
    --batch-command[=<format>]
                          read commands from stdin
    --batch-all-objects   with --batch[-check]: ignores stdin, batches all known objects

Change or optimize batch output
    --[no-]buffer         buffer --batch output
    --[no-]follow-symlinks
                          follow in-tree symlinks
    --[no-]unordered      do not order objects before emitting them

Emit object (blob or tree) with conversion or filter (stand-alone, or with batch)
    --textconv            run textconv on object's content
    --filters             run filters on object's content
    --[no-]path blob|tree use a <path> for (--textconv | --filters); Not with 'batch'
    --[no-]filter <args>  object filtering
```

Step 9: Inspect the Staging Area

Run:

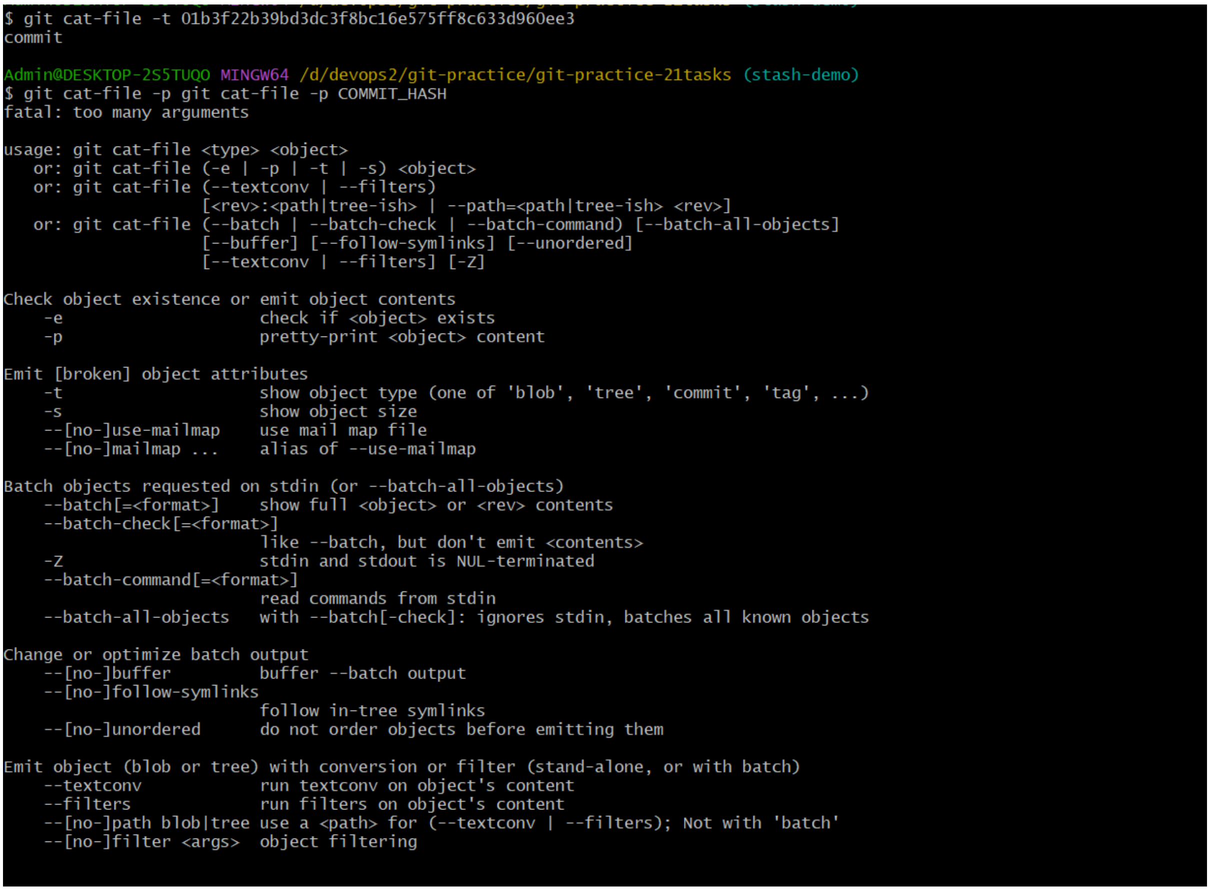
git status

Then inspect the index file's existence:

ls -l .git/index

Explanation: The .git/index file represents the staging area. When we run git add, Git updates the index with the file state prepared for the next commit.

Do not edit or delete .git/index manually.



```
$ git cat-file -t 01b3f22b39bd3dc3f8bc16e575ff8c633d960ee3
commit

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git cat-file -p git cat-file -p COMMIT_HASH
fatal: too many arguments

usage: git cat-file <type> <object>
   or: git cat-file (-e | -p | -t | -s) <object>
   or: git cat-file (--textconv | --filters)
                    [<rev>:<path|tree-ish> | --path=<path|tree-ish> <rev>]
   or: git cat-file (--batch | --batch-check | --batch-command) [--batch-all-objects]
                    [--buffer] [--follow-symlinks] [--unordered]
                    [--textconv | --filters] [-Z]

Check object existence or emit object contents
    -e                    check if <object> exists
    -p                    pretty-print <object> content

Emit [broken] object attributes
    -t                    show object type (one of 'blob', 'tree', 'commit', 'tag', ...)
    -s                    show object size
    --[no-]use-mailmap    use mail map file
    --[no-]mailmap ...    alias of --use-mailmap

Batch objects requested on stdin (or --batch-all-objects)
    --batch[=<format>]    show full <object> or <rev> contents
    --batch-check[=<format>]
                          like --batch, but don't emit <contents>
    -Z                    stdin and stdout is NUL-terminated
    --batch-command[=<format>]
                          read commands from stdin
    --batch-all-objects   with --batch[-check]: ignores stdin, batches all known objects

Change or optimize batch output
    --[no-]buffer         buffer --batch output
    --[no-]follow-symlinks
                          follow in-tree symlinks
    --[no-]unordered      do not order objects before emitting them

Emit object (blob or tree) with conversion or filter (stand-alone, or with batch)
    --textconv            run textconv on object's content
    --filters             run filters on object's content
    --[no-]path blob|tree use a <path> for (--textconv | --filters); Not with 'batch'
    --[no-]filter <args>  object filtering
```

Step 10: Understand the Relationship Between the Working Directory and .git

Git generally tracks changes through three important areas:

1. **Working directory:** Files you edit.
2. **Staging area:** Changes selected for the next commit.
3. **Repository history:** Committed snapshots and their related objects stored in Git's database.

Example workflow:

echo "Understanding Git internals" > git-internals-demo.txt

git status

git add git-internals-demo.txt

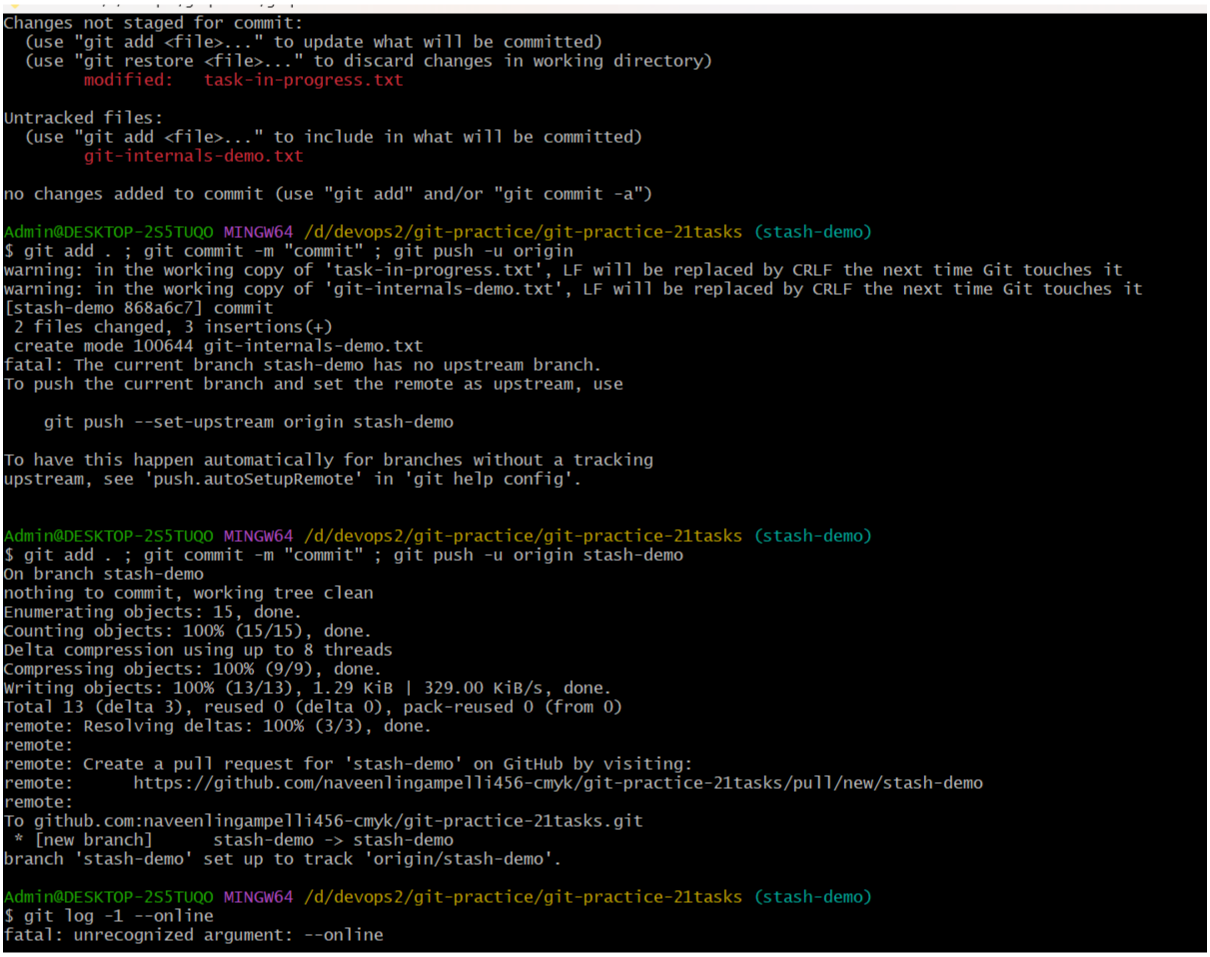
git status

git commit -m "Learn Git internal structure"

git log -1 --oneline

Run this demonstration only if you want to create an additional commit in your repository. It is optional and is not required just to inspect .git.

Screenshot 10:



```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   task-in-progress.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        git-internals-demo.txt

no changes added to commit (use "git add" and/or "git commit -a")

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git add . ; git commit -m "commit" ; git push -u origin
warning: in the working copy of 'task-in-progress.txt', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'git-internals-demo.txt', LF will be replaced by CRLF the next time Git touches it
[stash-demo 868a6c7] commit
 2 files changed, 3 insertions(+)
 create mode 100644 git-internals-demo.txt
fatal: The current branch stash-demo has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin stash-demo

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.


Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git add . ; git commit -m "commit" ; git push -u origin stash-demo
On branch stash-demo
nothing to commit, working tree clean
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 8 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (13/13), 1.29 KiB | 329.00 KiB/s, done.
Total 13 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), done.
remote:
remote: Create a pull request for 'stash-demo' on GitHub by visiting:
remote:      https://github.com/naveenlingampelli456-cmyk/git-practice-21tasks/pull/new/stash-demo
remote:
To github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git
 * [new branch]      stash-demo -> stash-demo
branch 'stash-demo' set up to track 'origin/stash-demo'.

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git log -1 --online
fatal: unrecognized argument: --online
```

5. Validation Steps

Use the following checklist to validate your understanding.

- Confirm that the current directory is a Git repository.
- Locate the .git directory using ls -la.
- Inspect the contents of .git.
- Read .git/HEAD and identify the current branch reference.
- Inspect .git/config and identify the configured remote.
- View local branches using git branch.
- Inspect the latest commit using git log.
- Understand the purpose of .git/index.
- Understand the roles of blobs, trees, commits, and tags.
- Confirm that repository commands work as expected.

6. Explanation of Important Commands

Command	Purpose

git status	Displays the current branch and working-tree changes.
git rev-parse --show-toplevel	Displays the root of the working tree.
ls -la	Lists files, including hidden entries.
ls -la .git	Lists the contents of the .git directory.
cat .git/HEAD	Displays the current HEAD reference.
cat .git/config	Displays repository-specific configuration.
git remote -v	Displays configured remote URLs.
git branch	Lists local branches.
git log -1 –oneline	Displays the latest commit in concise form.
git cat-file -t COMMIT_HASH	Displays the type of a Git object.
git cat-file -p COMMIT_HASH	Displays the contents of a Git object.

7. Real-Time Use Case

In a DevOps environment, understanding .git is useful when troubleshooting source-control issues.

For example:

- If the wrong branch is checked out, inspect HEAD and run git branch.
- If pushes go to the wrong GitHub repository, inspect .git/config and run git remote -v.
- If a developer wants to understand a commit, use git log and git cat-file.
- If a file has been staged accidentally, use git status to identify its state.
- If a commit or branch reference is lost, Git's references and reflogs may help recover it.

This knowledge helps developers understand Git behavior and troubleshoot issues more confidently.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
.git is not visible	Hidden entries are not being displayed.	Run ls -la from the repository root.
fatal: not a git repository	The terminal is in the wrong directory.	Navigate to the repository root and run git status.
cat: .git/HEAD: No such file or directory	The current directory is not a normal repository root, or Git metadata is stored elsewhere.	Run git rev-parse --git-dir and inspect the reported Git directory.

Branch files are missing under .git/refs/heads	References may be packed or the repository may use a different reference layout.	Use git branch and git show-ref to inspect references.
.git/config contains unexpected remote URLs	The remote may have been configured incorrectly.	Verify with git remote -v and correct it only if necessary.
Git status shows modified or untracked files	Files have changed or have not been committed.	Review the files before staging or committing them.
.git/index is missing or corrupted	The index may have been removed or damaged.	Avoid manual changes; inspect Git's error messages and use appropriate Git recovery procedures.
The .git directory is not a directory	The repository may use a .git file that points to a separate Git directory, as can happen with worktrees or submodules.	Inspect the file and use git rev-parse --git-dir to locate the actual metadata directory.

9. Important Precautions

- Never delete the .git directory unless you deliberately intend to remove the repository's local Git metadata.
- Do not manually edit internal Git files unless you understand the consequences.
- Avoid copying .git metadata into public locations because it may contain repository history and configuration.
- Use Git commands such as git status, git log, git branch, and git remote instead of modifying internal files directly.
- Back up important repositories before performing recovery operations.

10. Conclusion

The .git directory is the core metadata directory of a normal Git repository. It stores information needed to manage commits, branches, repository configuration, staging, and history.

By inspecting .git/HEAD, .git/config, .git/index, refs/, and objects/, we can develop a basic understanding of how Git manages repository data internally.

This knowledge is useful for everyday Git operations, troubleshooting, and DevOps workflows involving GitHub and CI/CD tools.

Documentation note: Replace the screenshot placeholders with actual screenshots captured during implementation. The optional demonstration commit is not required to complete this task.

Task 20: Check All the Logs of Git

1. Outcome

Understand how to view and analyze Git commit history using different Git log commands.

By completing this task, we will learn how to check commit IDs, commit messages, authors, timestamps, branch history, file-specific history, and graphical commit relationships.

Environment: Windows 11 with Git Bash

Local Repository: git-practice-21tasks

Repository Path: /d/devops2/git-practice/git-practice-21tasks

GitHub Repository: naveenlingampelli456-cmyk/git-practice-21tasks

2. Objective

The objective of this task is to inspect the commit history of a Git repository and understand the changes made over time.

Git logs help developers identify who made a change, when it was committed, what the commit message says, and how branches and merges are connected.

This information is useful for debugging application issues, tracking changes, reviewing team contributions, and investigating problems in DevOps workflows.

3. Prerequisites

- Git must be installed on the system.
- Git Bash must be available.
- A local Git repository must exist.
- The repository should contain at least one commit.
- Basic knowledge of Git commands such as git status, git add, and git commit.

4. Step-by-Step Implementation

Step 1: Open Git Bash and Navigate to the Repository

Open Git Bash and navigate to the local repository:

```
cd /d/devops2/git-practice/git-practice-21tasks
```

Verify the current directory:

```
pwd
```

Check the repository status:

```
git status
```

Expected result: Git displays the current branch and working-tree status.

Step 2: View the Complete Commit History

Run:

```
git log
```

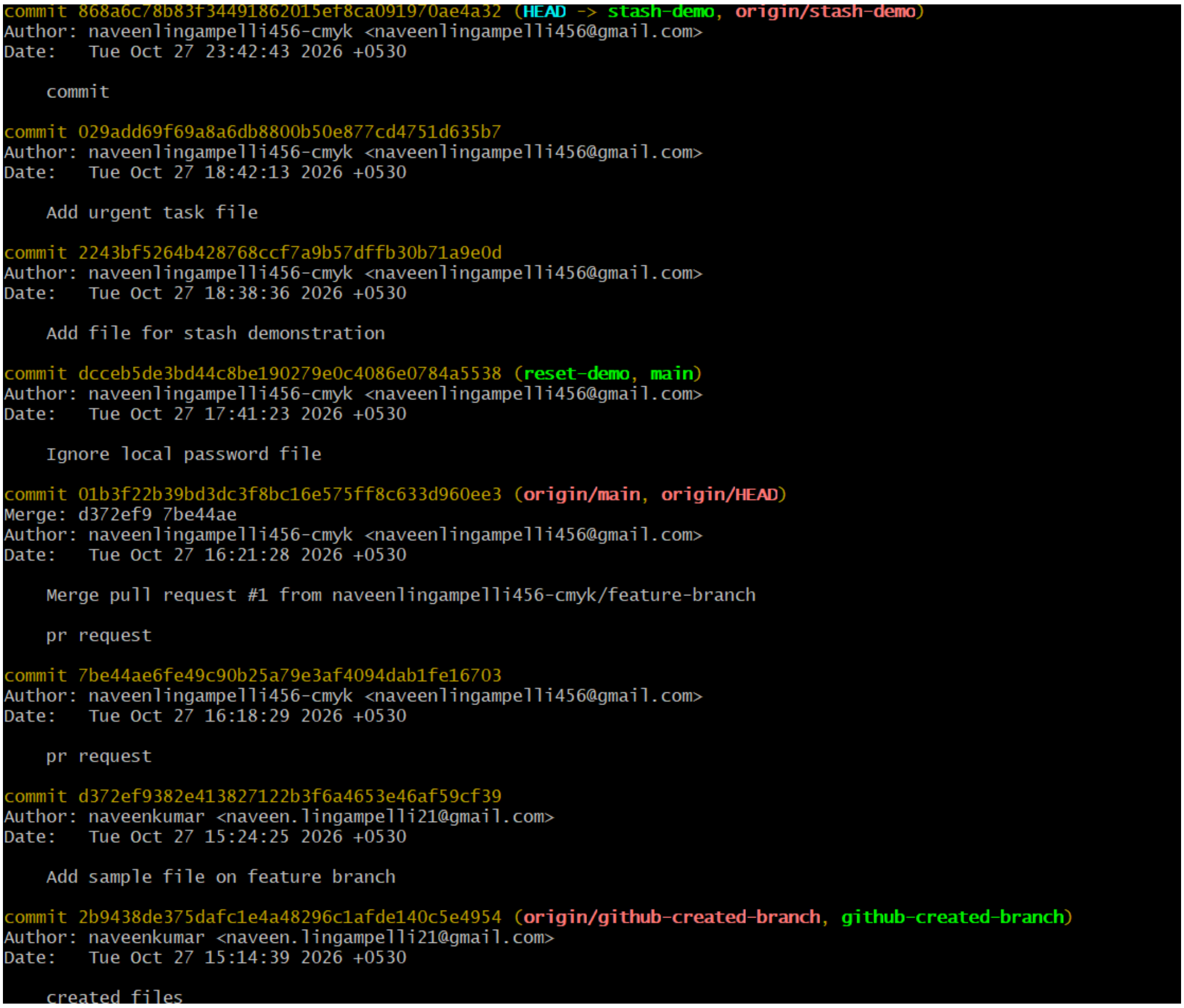
This command displays the commit history reachable from the current HEAD.

The output typically includes:

- Commit hash
- Author name and email
- Author date
- Commit message

Press q to exit the log viewer and return to the terminal.

Screenshot 2:



```
commit 868a6c78b83f34491862015ef8ca091970ae4a32 (HEAD -> stash-demo, origin/stash-demo)
Author: naveenlingampelli456-cmyk <naveenlingampelli456@gmail.com>
Date:   Tue Oct 27 23:42:43 2026 +0530

    commit

commit 029add69f69a8a6db8800b50e877cd4751d635b7
Author: naveenlingampelli456-cmyk <naveenlingampelli456@gmail.com>
Date:   Tue Oct 27 18:42:13 2026 +0530

    Add urgent task file

commit 2243bf5264b428768ccf7a9b57dffb30b71a9e0d
Author: naveenlingampelli456-cmyk <naveenlingampelli456@gmail.com>
Date:   Tue Oct 27 18:38:36 2026 +0530

    Add file for stash demonstration

commit dcceb5de3bd44c8be190279e0c4086e0784a5538 (reset-demo, main)
Author: naveenlingampelli456-cmyk <naveenlingampelli456@gmail.com>
Date:   Tue Oct 27 17:41:23 2026 +0530

    Ignore local password file

commit 01b3f22b39bd3dc3f8bc16e575ff8c633d960ee3 (origin/main, origin/HEAD)
Merge: d372ef9 7be44ae
Author: naveenlingampelli456-cmyk <naveenlingampelli456@gmail.com>
Date:   Tue Oct 27 16:21:28 2026 +0530

    Merge pull request #1 from naveenlingampelli456-cmyk/feature-branch

    pr request

commit 7be44ae6fe49c90b25a79e3af4094dab1fe16703
Author: naveenlingampelli456-cmyk <naveenlingampelli456@gmail.com>
Date:   Tue Oct 27 16:18:29 2026 +0530

    pr request

commit d372ef9382e413827122b3f6a4653e46af59cf39
Author: naveenkumar <naveen.lingampelli21@gmail.com>
Date:   Tue Oct 27 15:24:25 2026 +0530

    Add sample file on feature branch

commit 2b9438de375dafc1e4a48296c1afde140c5e4954 (origin/github-created-branch, github-created-branch)
Author: naveenkumar <naveen.lingampelli21@gmail.com>
Date:   Tue Oct 27 15:14:39 2026 +0530

    created files
```

Step 3: View a Short Commit History

Run:

git log --oneline

This displays each commit on a single line, showing an abbreviated commit hash and its commit message.

Example:

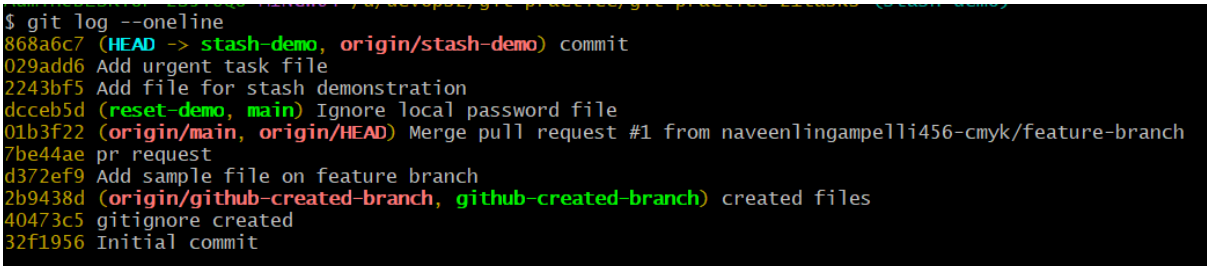
01b3f22 Merge pull request #1 from ...

```
7be44ae pr request

d372ef9 Add sample file on feature branch

2b9438d created files

40473c5 gitignore created

32f1956 Initial commit
```

The actual output may include additional commits from your completed tasks.

Explanation: The --oneline option makes the history easier to scan when a repository contains many commits.

Screenshot 3:



```
$ git log --oneline
868a6c7 (HEAD -> stash-demo, origin/stash-demo) commit
029add6 Add urgent task file
2243bf5 Add file for stash demonstration
dcceb5d (reset-demo, main) Ignore local password file
01b3f22 (origin/main, origin/HEAD) Merge pull request #1 from naveenlingampelli456-cmyk/feature-branch
7be44ae pr request
d372ef9 Add sample file on feature branch
2b9438d (origin/github-created-branch, github-created-branch) created files
40473c5 gitignore created
32f1956 Initial commit
```

Step 4: Display Commit History with Branch Information

Run:

```
git log --oneline --decorate
```

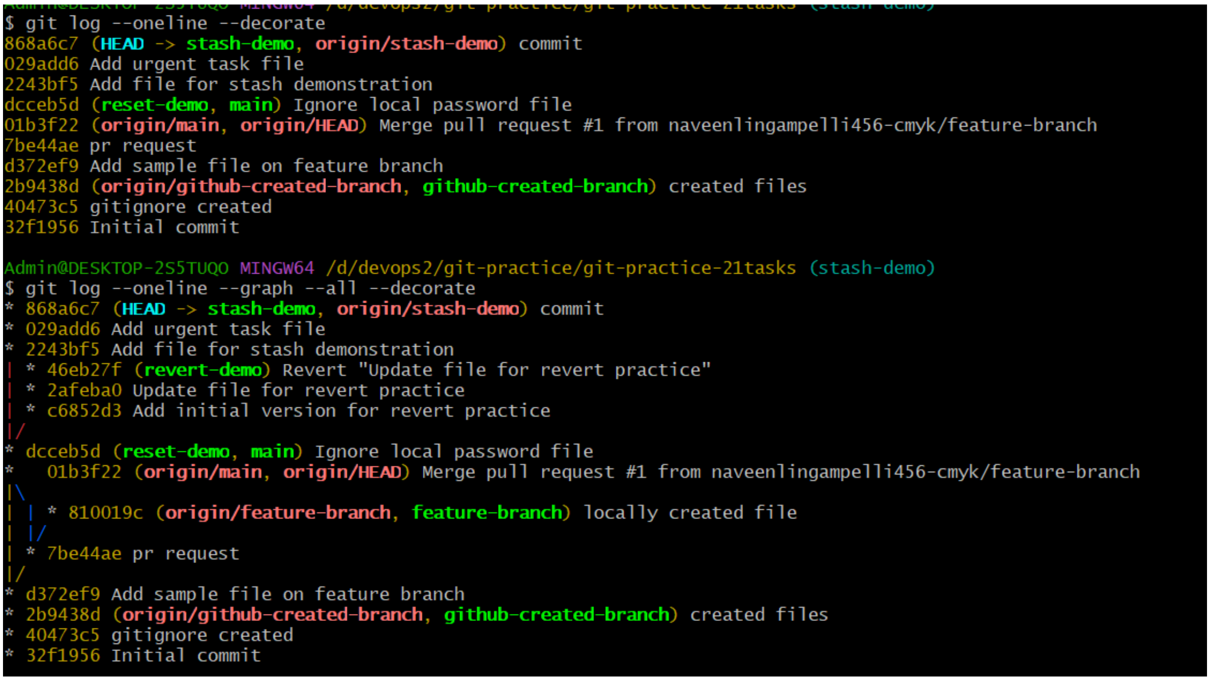
This command displays commit history with available reference labels, such as branch names and tags.

For a graphical view of branch and merge history, run:

```
git log --oneline --graph --all --decorate
```

This shows the commit graph for commits reachable from the available local and remote-tracking references. It helps visualize branching and merges.

Screenshot 4:



```
$ git log --oneline --decorate
868a6c7 (HEAD -> stash-demo, origin/stash-demo) commit
029add6 Add urgent task file
2243bf5 Add file for stash demonstration
dcceb5d (reset-demo, main) Ignore local password file
01b3f22 (origin/main, origin/HEAD) Merge pull request #1 from naveenlingampelli456-cmyk/feature-branch
7be44ae pr request
d372ef9 Add sample file on feature branch
2b9438d (origin/github-created-branch, github-created-branch) created files
40473c5 gitignore created
32f1956 Initial commit

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git log --oneline --graph --all --decorate
* 868a6c7 (HEAD -> stash-demo, origin/stash-demo) commit
* 029add6 Add urgent task file
* 2243bf5 Add file for stash demonstration
| * 46eb27f (revert-demo) Revert "Update file for revert practice"
| * 2afeba0 Update file for revert practice
| * c6852d3 Add initial version for revert practice
|/
* dcceb5d (reset-demo, main) Ignore local password file
*   01b3f22 (origin/main, origin/HEAD) Merge pull request #1 from naveenlingampelli456-cmyk/feature-branch
|\
| | * 810019c (origin/feature-branch, feature-branch) locally created file
| |/
| * 7be44ae pr request
|/
* d372ef9 Add sample file on feature branch
* 2b9438d (origin/github-created-branch, github-created-branch) created files
* 40473c5 gitignore created
* 32f1956 Initial commit
```

Step 5: View the Number of Recent Commits

To display only the five most recent commits, run:

git log -5 --oneline

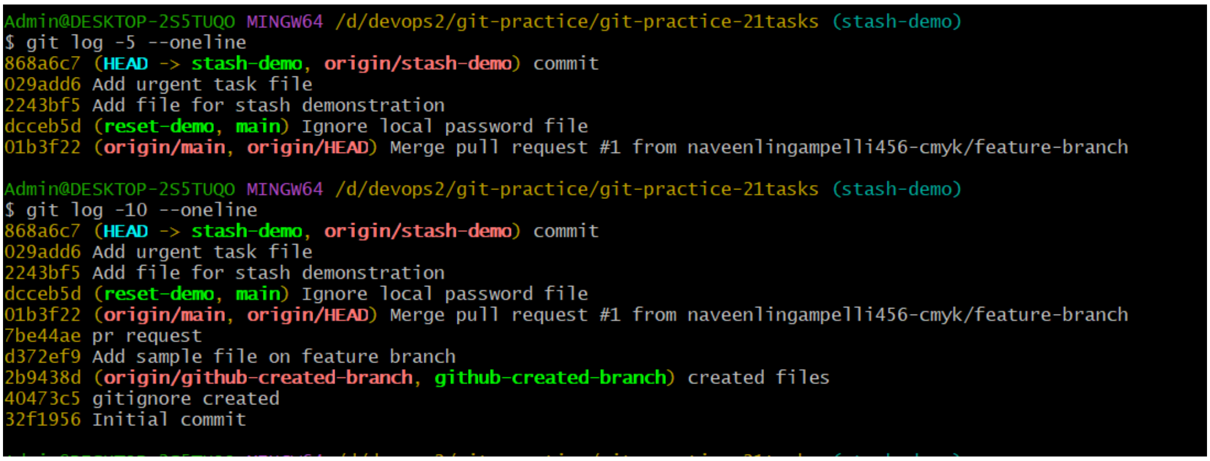
To display the ten most recent commits, run:

git log -10 --oneline

The number specifies how many commits to display, if that many commits are available.

Explanation: This is useful when you need a quick summary of recent repository activity.

Screenshot :



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git log -5 --oneline
868a6c7 (HEAD -> stash-demo, origin/stash-demo) commit
029add6 Add urgent task file
2243bf5 Add file for stash demonstration
dcceb5d (reset-demo, main) Ignore local password file
01b3f22 (origin/main, origin/HEAD) Merge pull request #1 from naveenlingampelli456-cmyk/feature-branch

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git log -10 --oneline
868a6c7 (HEAD -> stash-demo, origin/stash-demo) commit
029add6 Add urgent task file
2243bf5 Add file for stash demonstration
dcceb5d (reset-demo, main) Ignore local password file
01b3f22 (origin/main, origin/HEAD) Merge pull request #1 from naveenlingampelli456-cmyk/feature-branch
7be44ae pr request
d372ef9 Add sample file on feature branch
2b9438d (origin/github-created-branch, github-created-branch) created files
40473c5 gitignore created
32f1956 Initial commit
```

Step 6: View Commit History with Author Information

Run:

git log --pretty=format:"%h | %an | %ad | %s" --date=short

The format placeholders mean:

Placeholder	Meaning
%h	Abbreviated commit hash
%an	Author name
%ad	Author date
%s	Commit subject or message

This command presents the commit history in a compact, readable format.

Step 7: View the History of a Specific File

To check the history of .gitignore, run:

```
git log --oneline -- .gitignore
```

To view the history of another tracked file, replace .gitignore with its path.

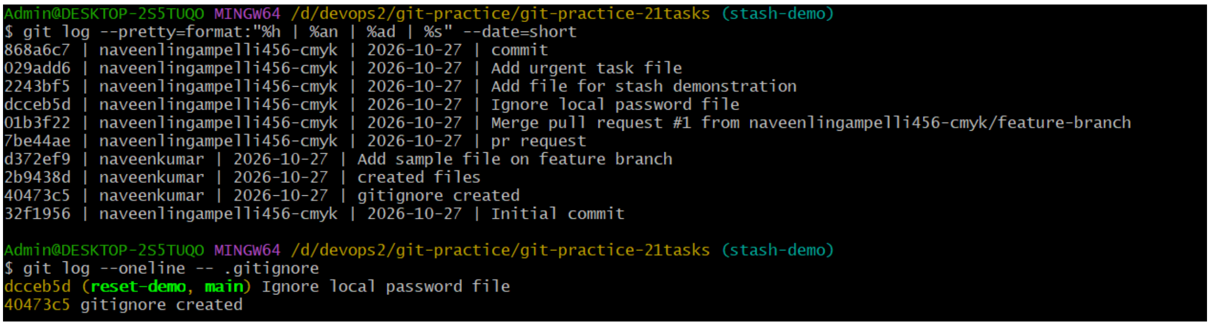
For example:

```
git log --oneline -- README.md
```

The output shows commits that affected the specified file, as long as those commits are reachable from the current history.

Explanation: File-specific logs are helpful when investigating when a configuration file or application file was changed.

Screenshot 8:



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git log --pretty=format:"%h | %an | %ad | %s" --date=short
868a6c7 | naveenlingampelli456-cmyk | 2026-10-27 | commit
029add6 | naveenlingampelli456-cmyk | 2026-10-27 | Add urgent task file
2243bf5 | naveenlingampelli456-cmyk | 2026-10-27 | Add file for stash demonstration
dcceb5d | naveenlingampelli456-cmyk | 2026-10-27 | Ignore local password file
01b3f22 | naveenlingampelli456-cmyk | 2026-10-27 | Merge pull request #1 from naveenlingampelli456-cmyk/feature-branch
7be44ae | naveenlingampelli456-cmyk | 2026-10-27 | pr request
d372ef9 | naveenkumar | 2026-10-27 | Add sample file on feature branch
2b9438d | naveenkumar | 2026-10-27 | created files
40473c5 | naveenkumar | 2026-10-27 | gitignore created
32f1956 | naveenlingampelli456-cmyk | 2026-10-27 | Initial commit

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git log --oneline -- .gitignore
dcceb5d (reset-demo, main) Ignore local password file
40473c5 gitignore created
```

Step 8: View the Changes Introduced by Each Commit

Run:

```
git log -p -3
```

This displays patch details for the three most recent commits, including the changes introduced by each commit.

To view only the names of files changed in recent commits, run:

```
git log --stat -5
```

To view a concise list of files changed by a specific commit, run:

```
git show --name-only --format=fuller COMMIT_HASH
```

Replace COMMIT_HASH with a valid hash from your repository.

Explanation: Patch and file-stat views help developers investigate code changes without opening every file manually.

Step 10: View Commit History by Date

To view commits made within the last seven days, run:

```
git log --since="7 days ago" --oneline
```

To view commits made after a specified date, run:

```
git log --since="2026-10-01" --oneline
```

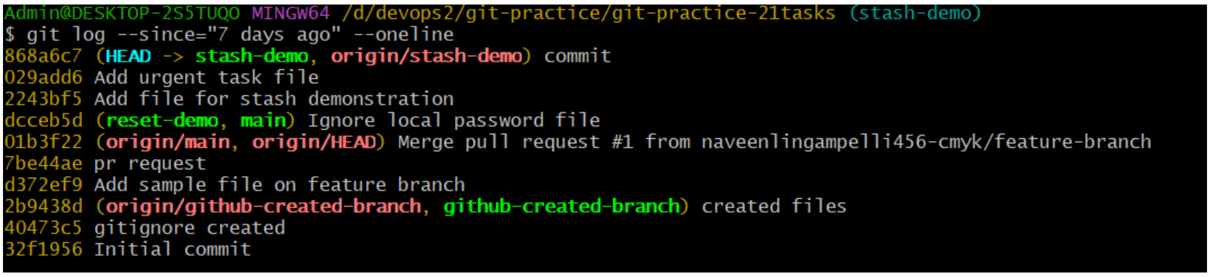
To view commits made before a specified date, run:

```
git log --before="2026-10-27" --oneline
```

These examples use the dates shown in this assignment. Change the dates as needed for your repository history.

Explanation: Date filters are useful when reviewing recent development activity or preparing reports.

Screenshot 10:



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git log --since="7 days ago" --oneline
868a6c7 (HEAD -> stash-demo, origin/stash-demo) commit
029add6 Add urgent task file
2243bf5 Add file for stash demonstration
dcceb5d (reset-demo, main) Ignore local password file
01b3f22 (origin/main, origin/HEAD) Merge pull request #1 from naveenlingampelli456-cmyk/feature-branch
7be44ae pr request
d372ef9 Add sample file on feature branch
2b9438d (origin/github-created-branch, github-created-branch) created files
40473c5 gitignore created
32f1956 Initial commit
```

Step 11: View Commits by Author

To filter the history by author name or email, run:

```
git log --author="naveenlingampelli456" --oneline
```

Replace the filter with the author name or email you want to search for.

Git matches the filter against the author information recorded in commits. A commit may have a different author and committer, so the filter might not match every commit associated with a person in another role.

Explanation: This command helps review contributions by a particular author.

Step 12: Check the Reflog

Run:

```
git reflog
```

The reflog records recent changes to local references, including movements of HEAD. It may show operations such as commits, branch switches, resets, and merges.

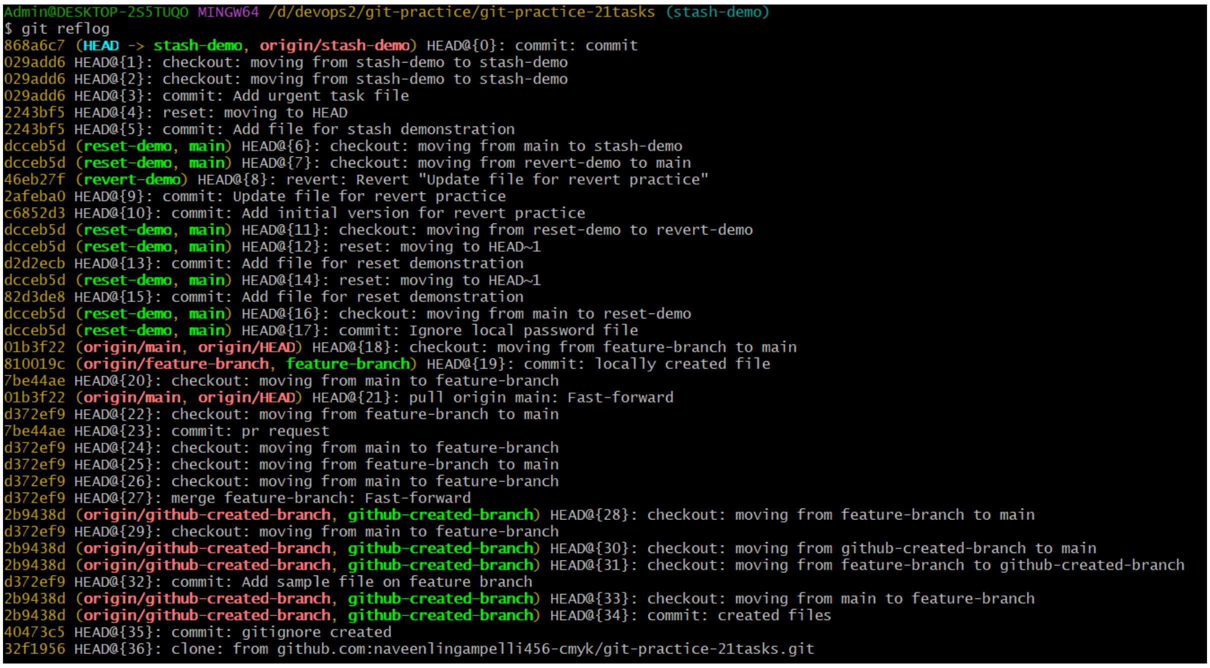
To see a concise reflog for the current branch's HEAD, run:

git reflog --oneline

Important: The reflog is not the same as git log. It records local reference movements and is generally maintained locally. Entries can expire, and reflogs are not a permanent backup.

This command is particularly useful when investigating recent operations such as the reset performed in Task 13.

Screenshot 12:



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git reflog
868a6c7 (HEAD -> stash-demo, origin/stash-demo) HEAD@{0}: commit: commit
029add6 HEAD@{1}: checkout: moving from stash-demo to stash-demo
029add6 HEAD@{2}: checkout: moving from stash-demo to stash-demo
029add6 HEAD@{3}: commit: Add urgent task file
2243bf5 HEAD@{4}: reset: moving to HEAD
2243bf5 HEAD@{5}: commit: Add file for stash demonstration
dcceb5d (reset-demo, main) HEAD@{6}: checkout: moving from main to stash-demo
dcceb5d (reset-demo, main) HEAD@{7}: checkout: moving from revert-demo to main
46eb27f (revert-demo) HEAD@{8}: revert: Revert "Update file for revert practice"
2afeba0 HEAD@{9}: commit: Update file for revert practice
c6852d3 HEAD@{10}: commit: Add initial version for revert practice
dcceb5d (reset-demo, main) HEAD@{11}: checkout: moving from reset-demo to revert-demo
dcceb5d (reset-demo, main) HEAD@{12}: reset: moving to HEAD~1
d2d2ecb HEAD@{13}: commit: Add file for reset demonstration
dcceb5d (reset-demo, main) HEAD@{14}: reset: moving to HEAD~1
82d3de8 HEAD@{15}: commit: Add file for reset demonstration
dcceb5d (reset-demo, main) HEAD@{16}: checkout: moving from main to reset-demo
dcceb5d (reset-demo, main) HEAD@{17}: commit: Ignore local password file
01b3f22 (origin/main, origin/HEAD) HEAD@{18}: checkout: moving from feature-branch to main
810019c (origin/feature-branch, feature-branch) HEAD@{19}: commit: locally created file
7be44ae HEAD@{20}: checkout: moving from main to feature-branch
01b3f22 (origin/main, origin/HEAD) HEAD@{21}: pull origin main: Fast-forward
d372ef9 HEAD@{22}: checkout: moving from feature-branch to main
7be44ae HEAD@{23}: commit: pr request
d372ef9 HEAD@{24}: checkout: moving from main to feature-branch
d372ef9 HEAD@{25}: checkout: moving from feature-branch to main
d372ef9 HEAD@{26}: checkout: moving from main to feature-branch
d372ef9 HEAD@{27}: merge feature-branch: Fast-forward
2b9438d (origin/github-created-branch, github-created-branch) HEAD@{28}: checkout: moving from feature-branch to main
d372ef9 HEAD@{29}: checkout: moving from main to feature-branch
2b9438d (origin/github-created-branch, github-created-branch) HEAD@{30}: checkout: moving from github-created-branch to main
2b9438d (origin/github-created-branch, github-created-branch) HEAD@{31}: checkout: moving from feature-branch to github-created-branch
d372ef9 HEAD@{32}: commit: Add sample file on feature branch
2b9438d (origin/github-created-branch, github-created-branch) HEAD@{33}: checkout: moving from main to feature-branch
2b9438d (origin/github-created-branch, github-created-branch) HEAD@{34}: commit: created files
40473c5 HEAD@{35}: commit: gitignore created
32f1956 HEAD@{36}: clone: from github.com:naveenlingampelli456-cmyk/git-practice-21tasks.git
```

5. Validation Steps

Use the following checklist to validate the task.

- Navigate to the correct Git repository.
- Run git log to display detailed commit history.
- Run git log --oneline to display concise history.
- Run git log --graph --oneline --all --decorate to inspect branch history.
- Use git show COMMIT_HASH to inspect a specific commit.
- Filter the history by file, author, or date.
- Use git log -p or git log --stat to inspect changes.
- Run git reflog to inspect recent local reference movements.
- Confirm that the commands return meaningful results for your repository.

6. Explanation of Important Commands

Command	Purpose

git log	Displays detailed commit history.
git log --oneline	Displays concise commit history.
git log --decorate	Shows available reference labels.
git log --graph --oneline --all --decorate	Displays a graphical history of available references.
git show COMMIT_HASH	Displays details and changes for a specific commit.
git log -5 --oneline	Displays the five most recent commits.
git log --pretty=format:...	Customizes the commit-history output.
git log -- file-name	Displays history affecting a specified file.
git log -p -3	Displays patches for the three most recent commits.
git log --stat -5	Displays file-change statistics for recent commits.
git log --since="7 days ago"	Filters commits by a starting time.
git log --author="name"	Filters commits by author information.
git reflog	Displays recent local reference movements.
git log –all	Includes history reachable from all available local and remote-tracking references.

7. Real-Time Use Case

In a DevOps environment, Git logs help investigate application problems and track changes across development teams.

For example, suppose an application starts failing after a deployment. A DevOps engineer can:

1. Inspect recent commits using git log --oneline.
2. Identify a potentially related commit.
3. Inspect the changes using git show COMMIT_HASH.
4. Check the history of the affected configuration file.
5. Review branch and merge history to understand how the change reached the current branch.
6. Use git reflog if a recent local reset or branch movement needs investigation.

This workflow helps engineers identify potentially problematic changes and gather evidence before deciding on a safe fix or rollback.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
fatal: not a git repository	The terminal is outside the repository.	Navigate to the repository root and run the command again.
git log shows only a few commits	The current branch has limited reachable history, or the repository is shallow.	Check the current branch and clone configuration; fetch the required history if appropriate.
A commit hash is invalid	The hash was copied incorrectly or does not exist locally.	Copy the hash directly from git log --oneline.
File-specific history is empty	The path is incorrect or no reachable commit changed the file.	Verify the file path and use git log --all -- path/to/file if needed.
Author filter returns no commits	The recorded author name or email does not match the filter.	Inspect author details using git log --format=fuller.
Reflog does not show an old operation	Reflog entries can expire or may not exist for the relevant reference.	Check available commit history and other references; do not assume reflogs retain every operation permanently.
Log output fills the terminal	Git opens a pager to display long output.	Press q to exit the pager.
Recent GitHub commits are missing	Local remote-tracking references may be outdated.	Run git fetch origin, then inspect the history again.
Branch graph appears different from GitHub	Local references may differ from the latest remote state.	Fetch updates and compare local and remote-tracking branches.

9. Best Practices

- Use meaningful commit messages to make the history easy to understand.
- Inspect commits before attempting a rollback or revert.
- Use git log --graph --oneline --all --decorate to understand branch and merge relationships.
- Use file-specific history to investigate configuration or application changes.
- Remember that git log and git reflog serve different purposes.
- Fetch remote updates before relying on remote-tracking history.
- Avoid rewriting shared commit history unless the team has agreed to do so.
- Do not publish sensitive information that may be present in commit messages or historical file contents.

10. Conclusion

In this task, we learned how to inspect Git history using multiple log commands. We explored detailed and concise commit logs, branch graphs, author and date filters, file-specific history, patch details, and reflogs.

These commands help developers understand the evolution of a codebase, investigate changes, troubleshoot issues, and support reliable DevOps workflows.

Task 21: Rename the Git Commit Message

1. Outcome

Successfully learn how to rename a Git commit message using the git commit --amend command.

This task demonstrates how to update the most recent commit message without creating an additional commit. A separate demo branch is used to avoid modifying the main branch.

Environment: Windows 11 with Git Bash

Local Repository: git-practice-21tasks

Repository Path: /d/devops2/git-practice/git-practice-21tasks

Demo Branch: rename-commit-demo

2. Objective

The objective of this task is to understand how to rename the most recent Git commit message.

Commit messages describe the changes introduced by a commit. Clear and meaningful messages make the Git history easier to understand and maintain.

Sometimes a developer commits changes with an incorrect, incomplete, or unclear message. Git provides the git commit --amend command to correct the most recent commit message.

For example:

- Original message: wrong commit message
- Updated message: Rename commit message

3. Prerequisites

The following prerequisites are required:

- Git installed on Windows 11.
- Git Bash available.
- A local Git repository.
- Basic understanding of Git commits and branches.
- A clean working tree before starting the exercise.

Important: This task uses a separate demo branch. Amending a commit changes its commit ID because Git creates a replacement commit. Avoid amending commits that have already been shared with other developers unless history rewriting has been coordinated.

4. Step-by-Step Implementation

Step 1: Open Git Bash and Navigate to the Repository

Open Git Bash and navigate to the repository:

```
cd /d/devops2/git-practice/git-practice-21tasks
```

Verify the current directory:

```
pwd
```

Check the working-tree status:

```
git status
```

Ensure any unrelated changes are committed or safely stored before proceeding.

Step 2: Create a Separate Demo Branch

Create and switch to a new branch:

```
git switch -c rename-commit-demo
```

Verify the current branch:

```
git branch --show-current
```

Expected output:

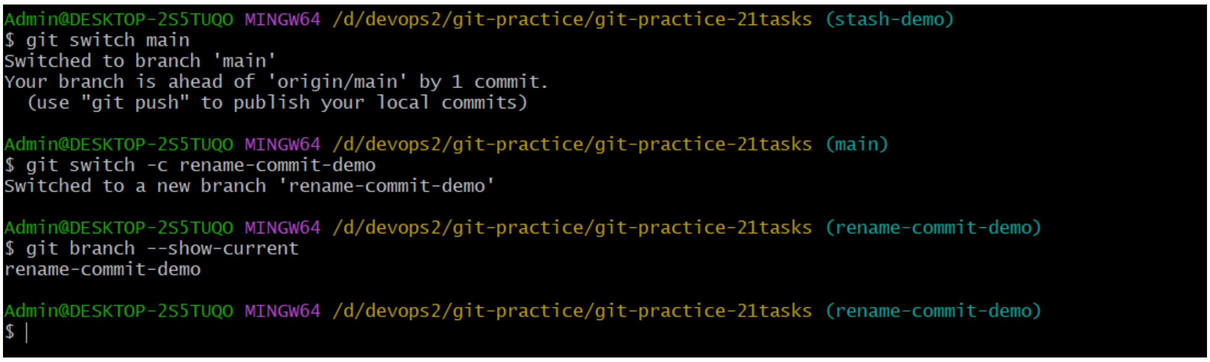
```
rename-commit-demo
```

This keeps the demonstration separate from main.

If the branch already exists from an earlier attempt, use:

```
git switch rename-commit-demo
```

Screenshot 2:



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (stash-demo)
$ git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (main)
$ git switch -c rename-commit-demo
Switched to a new branch 'rename-commit-demo'

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (rename-commit-demo)
$ git branch --show-current
rename-commit-demo

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (rename-commit-demo)
$ |
```

Step 3: Create a Demo File

Create a sample file:

```
echo "Testing commit message rename" > rename-demo.txt
```

Verify the file contents:

```
cat rename-demo.txt
```

Expected output:

Testing commit message rename

Check the repository status:

git status

The new file should appear as untracked.

Step 4: Stage and Commit the File with an Incorrect Message

Stage the demo file:

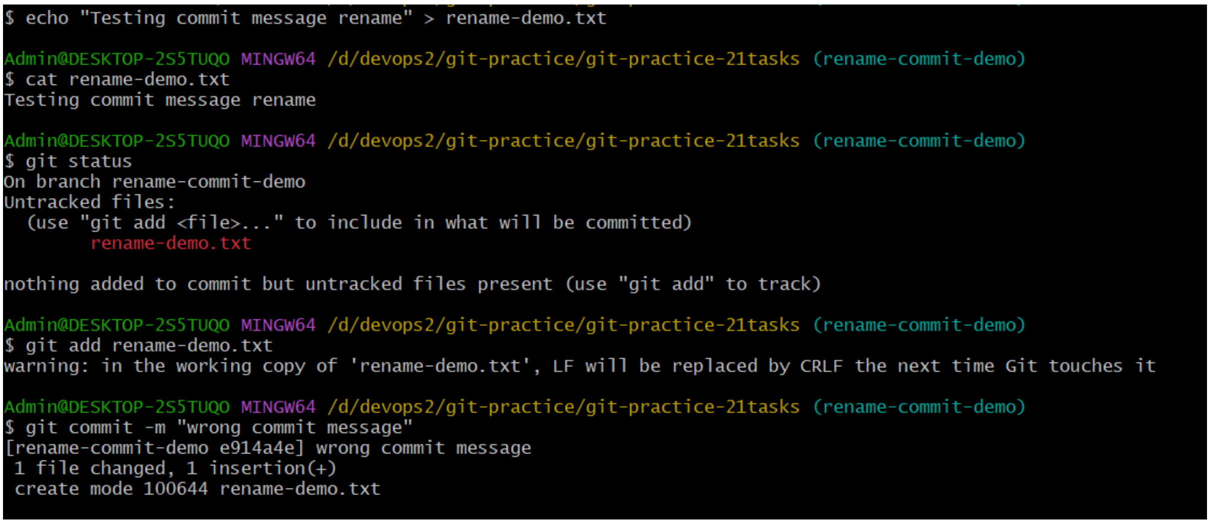
git add rename-demo.txt

Create the initial commit with an intentionally unclear message:

git commit -m "wrong commit message"

Git should report that the commit was created.

Screenshot 4:



```
$ echo "Testing commit message rename" > rename-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (rename-commit-demo)
$ cat rename-demo.txt
Testing commit message rename

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (rename-commit-demo)
$ git status
On branch rename-commit-demo
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        rename-demo.txt

nothing added to commit but untracked files present (use "git add" to track)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (rename-commit-demo)
$ git add rename-demo.txt
warning: in the working copy of 'rename-demo.txt', LF will be replaced by CRLF the next time Git touches it

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (rename-commit-demo)
$ git commit -m "wrong commit message"
[rename-commit-demo e914a4e] wrong commit message
 1 file changed, 1 insertion(+)
 create mode 100644 rename-demo.txt
```

Step 5: Verify the Original Commit Message

Run:

git log -1 --oneline

Example output:

a1b2c3d wrong commit message

The hash shown above is illustrative; your actual commit hash will be different.

This command displays the most recent commit and confirms the original message.

Step 6: Rename the Most Recent Commit Message

Run:

git commit --amend -m "Rename commit message"

This replaces the most recent commit with an amended commit containing the updated message.

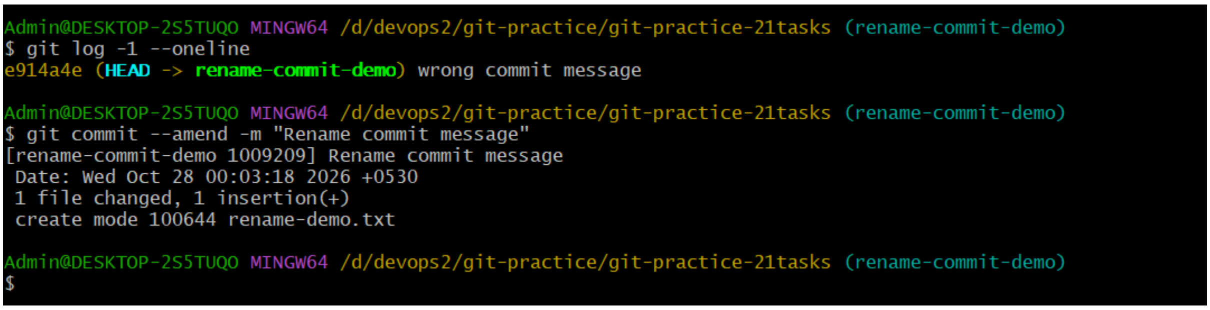
Expected output will indicate that the commit was amended. The file changes remain part of the amended commit.

Explanation:

- git commit creates a commit.
- --amend replaces the most recent commit with an updated version.
- -m supplies the new commit message.

The amended commit will normally have a different commit hash, even when only the message changes.

Screenshot 6:



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (rename-commit-demo)
$ git log -1 --oneline
e914a4e (HEAD -> rename-commit-demo) wrong commit message

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (rename-commit-demo)
$ git commit --amend -m "Rename commit message"
[rename-commit-demo 1009209] Rename commit message
 Date: Wed Oct 28 00:03:18 2026 +0530
 1 file changed, 1 insertion(+)
 create mode 100644 rename-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (rename-commit-demo)
$
```

Step 7: Verify the Updated Commit Message

Run:

```
git log -1 --oneline
```

Expected output:

```
d4e5f6a Rename commit message
```

The hash is an example. Your actual hash will differ.

Confirm that the old message wrong commit message has been replaced by Rename commit message for the latest commit.

You can also inspect the full commit details:

```
git show --stat HEAD
```

Step 8: Verify the Repository Status

Run:

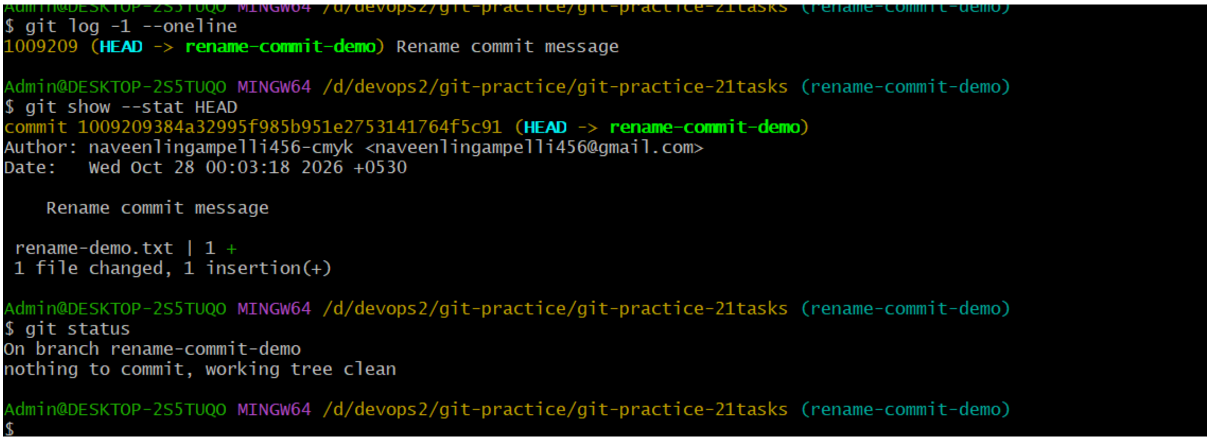
```
git status
```

Expected result:

```
On branch rename-commit-demo

nothing to commit, working tree clean
```

The exact output depends on whether other changes exist in the repository.

A clean working tree confirms that no uncommitted changes remain from this exercise.



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (rename-commit-demo)
$ git log -1 --oneline
1009209 (HEAD -> rename-commit-demo) Rename commit message

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (rename-commit-demo)
$ git show --stat HEAD
commit 1009209384a32995f985b951e2753141764f5c91 (HEAD -> rename-commit-demo)
Author: naveenlingampelli456-cmyk <naveenlingampelli456@gmail.com>
Date:   Wed Oct 28 00:03:18 2026 +0530

    Rename commit message

 rename-demo.txt | 1 +
 1 file changed, 1 insertion(+)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (rename-commit-demo)
$ git status
On branch rename-commit-demo
nothing to commit, working tree clean

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (rename-commit-demo)
$
```

5. Validation Steps

Use the following checklist to validate the task.

- Navigate to the correct Git repository.
- Confirm that the working tree is clean before starting.
- Create and switch to rename-commit-demo.
- Create and commit rename-demo.txt.
- Verify the original commit message.
- Run git commit --amend -m "Rename commit message".
- Verify the updated message with git log -1 --oneline.
- Confirm that the working tree is clean.
- Confirm that the demonstration did not modify the main branch.

6. Explanation of Important Commands

Command	**Purpose**
git switch -c rename-commit-demo	Creates and switches to a new branch.
git branch --show-current	Displays the current branch name.
git status	Displays the working-tree and staging-area status.
git add rename-demo.txt	Stages the demo file.
git commit -m "wrong commit message"	Creates the initial commit.
git log -1 --oneline	Displays the latest commit in a concise format.
git commit --amend -m "Rename commit message"	Replaces the latest commit with an amended commit containing the new message.

git show --stat HEAD	Displays the latest commit details and file-change statistics.

7. Real-Time Use Case

In a DevOps team, developers work with Git daily and use commit messages to document changes.

Suppose a developer commits a configuration update with the message:

changes

The message does not clearly explain what was changed.

The developer can correct it before sharing the commit:

```
git commit --amend -m "Update application configuration"
```

The updated message makes the repository history more understandable during code reviews, troubleshooting, and release preparation.

Important: Amend only the latest commit. If the commit has already been pushed and other developers are using it, coordinate before rewriting the shared history.

8. Common Errors and Troubleshooting

Error or Issue	**Possible Cause**	**Resolution**
fatal: not a git repository	Git Bash is in the wrong directory.	Navigate to the repository root and retry.
Branch already exists	The demo branch was created earlier.	Run git switch rename-commit-demo.
Nothing to amend	No previous commit exists on the current branch, or the expected commit is not the current HEAD.	Run git log -1 --oneline and confirm the branch and commit history.
Wrong commit message still appears	The amend command did not run successfully, or another commit is now at HEAD.	Run git log -1 --oneline and repeat the amend only after confirming the intended commit.
Unexpected changes appear in git status	Other files were modified before or during the exercise.	Review the changes and avoid committing unrelated files.
Push is rejected after amending a previously pushed commit	The amended commit has a different hash from the commit on the remote branch.	Coordinate with the team. Use a safe history-update procedure only when authorized.
Git opens an editor unexpectedly	The amend command was run without a -m argument or another editor-based option was used.	Enter the desired message in the editor, save, and close it.

9. Best Practices

- Write short, descriptive commit messages.

- Use git log -1 --oneline to verify the current message before and after amending.
- Use a separate branch for learning and testing.
- Avoid rewriting commits that have already been shared with other developers.
- Never use git commit --amend as a substitute for creating a new commit when you intend to preserve the existing commit as a separate historical event.
- Review git status before running Git commands that modify history.

10. Conclusion

In this task, we learned how to rename the most recent Git commit message using git commit --amend -m.

We created a separate demo branch, committed a sample file with an unclear message, amended the commit with a meaningful message, and verified the result using git log.

This technique helps maintain a clean, readable Git history when the latest commit has not yet been shared with others.

Task 22: Merge Multiple Git Commits into a Single Commit

1. Outcome

Learn how to combine multiple Git commits into one commit using **interactive rebase** and the squash command.

In this task, three separate commits will be created on a dedicated demo branch and then combined into one commit. The result will be verified using Git log commands.

Environment: Windows 11 with Git Bash

Local Repository: git-practice-21tasks

Repository Path: /d/devops2/git-practice/git-practice-21tasks

Demo Branch: squash-commits-demo

Technique: Interactive rebase with squash

2. Objective

The objective of this task is to understand how to combine multiple commits into a single commit while maintaining the changes introduced by those commits.

During development, developers may create several small commits while working on one feature. Before merging the feature branch into the main branch, they may want to combine those commits into a single, meaningful commit.

For example, three commits:

- Add initial demo file
- Update demo file
- Complete demo file

Can be combined into one commit:

- Add completed demo file

This process is commonly called **squashing commits**.

3. Prerequisites

- Git installed on Windows 11.
- Git Bash available.
- A local Git repository with existing commits.
- Basic understanding of Git commits and branches.
- A clean working tree before starting the exercise.

Important: Interactive rebase rewrites commit history and creates new commit hashes. This task uses a separate demo branch to avoid modifying main or rewriting shared commits.

4. Step-by-Step Implementation

Step 1: Open Git Bash and Navigate to the Repository

Open Git Bash and navigate to your local repository:

```
cd /d/devops2/git-practice/git-practice-21tasks
```

Verify the current directory:

```
pwd
```

Check the repository status:

```
git status
```

Make sure there are no unrelated uncommitted changes before starting.

Step 2: Create a Separate Demo Branch

Create and switch to a new branch:

```
git switch -c squash-commits-demo
```

Verify the current branch:

```
git branch --show-current
```

Expected output:

```
squash-commits-demo
```

If this branch already exists from an earlier attempt, switch to it instead:

```
git switch squash-commits-demo
```

Step 3: Create the First Commit

Create a sample file:

echo "First change" > squash-demo.txt

Stage the file:

git add squash-demo.txt

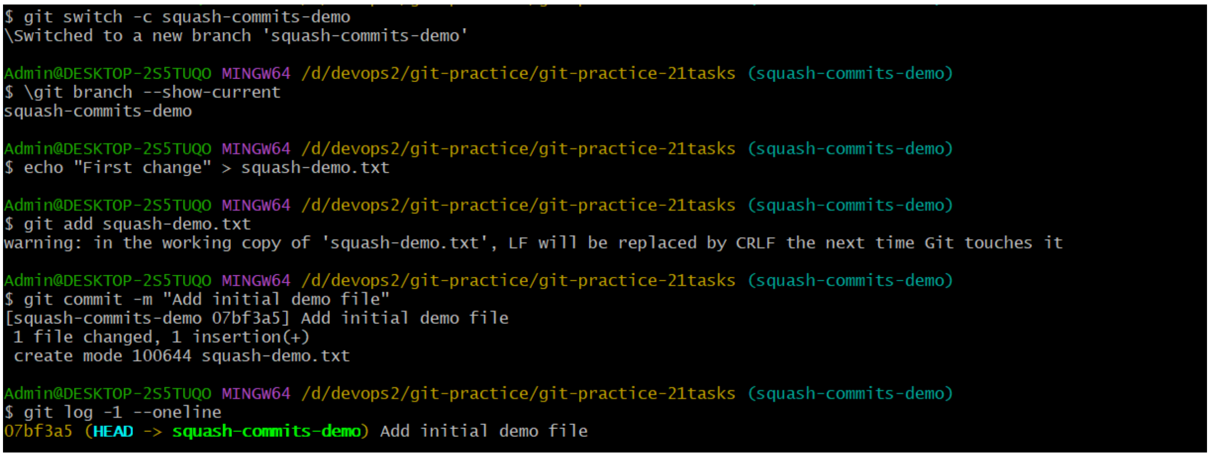
Create the first commit:

git commit -m "Add initial demo file"

Verify the latest commit:

git log -1 --oneline

Screenshot 3:



```
$ git switch -c squash-commits-demo
\Switched to a new branch 'squash-commits-demo'

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
$ \git branch --show-current
squash-commits-demo

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
$ echo "First change" > squash-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
$ git add squash-demo.txt
warning: in the working copy of 'squash-demo.txt', LF will be replaced by CRLF the next time Git touches it

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
$ git commit -m "Add initial demo file"
[squash-commits-demo 07bf3a5] Add initial demo file
 1 file changed, 1 insertion(+)
 create mode 100644 squash-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
$ git log -1 --oneline
07bf3a5 (HEAD -> squash-commits-demo) Add initial demo file
```

Step 4: Create the Second Commit

Append another line to the file:

echo "Second change" >> squash-demo.txt

Stage the changes:

git add squash-demo.txt

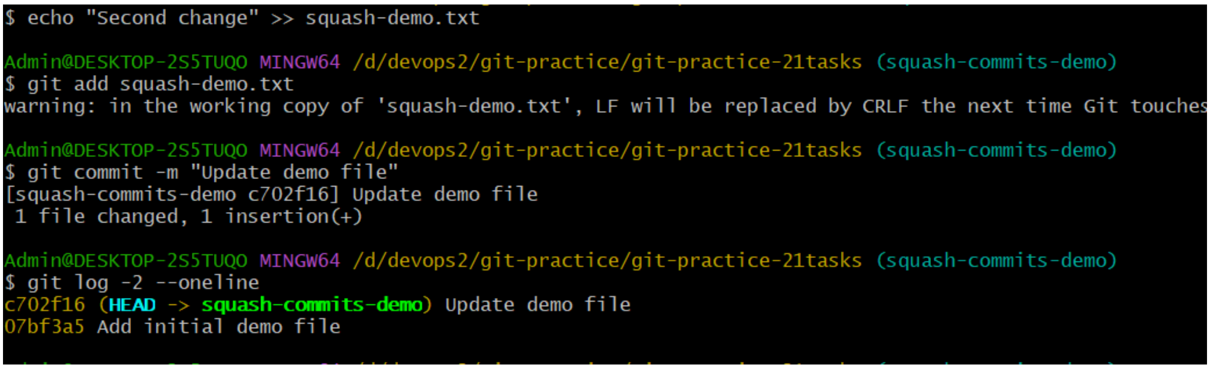
Create the second commit:

git commit -m "Update demo file"

Verify the latest commits:

git log -2 --oneline

Screenshot 4:



```
$ echo "Second change" >> squash-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
$ git add squash-demo.txt
warning: in the working copy of 'squash-demo.txt', LF will be replaced by CRLF the next time Git touches

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
$ git commit -m "Update demo file"
[squash-commits-demo c702f16] Update demo file
 1 file changed, 1 insertion(+)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
$ git log -2 --oneline
c702f16 (HEAD -> squash-commits-demo) Update demo file
07bf3a5 Add initial demo file
```

Step 5: Create the Third Commit

Append a third line to the file:

echo "Third change" >> squash-demo.txt

Stage the changes:

git add squash-demo.txt

Create the third commit:

git commit -m "Complete demo file"

Verify the three commits:

git log -3 --oneline

Expected commit messages, from newest to oldest:

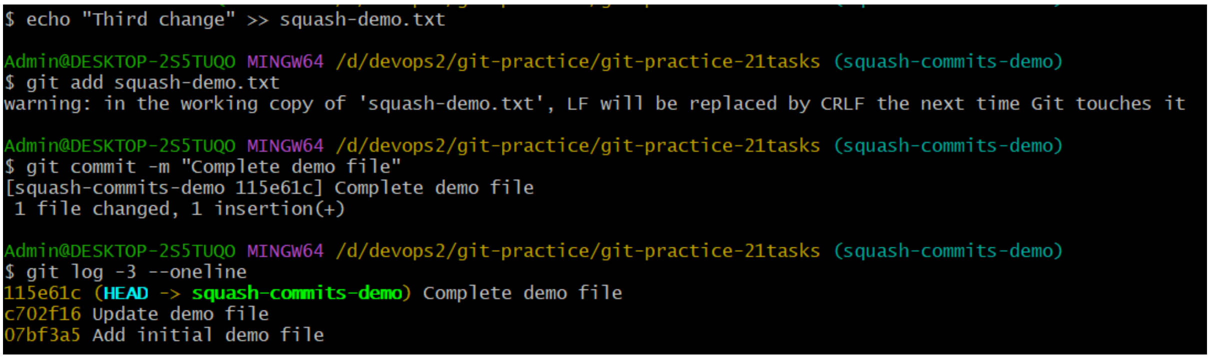
<hash-3> Complete demo file

<hash-2> Update demo file

<hash-1> Add initial demo file

The hashes will be generated by Git and will differ on your system.

Screenshot 5:



```
$ echo "Third change" >> squash-demo.txt

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
$ git add squash-demo.txt
warning: in the working copy of 'squash-demo.txt', LF will be replaced by CRLF the next time Git touches it

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
$ git commit -m "Complete demo file"
[squash-commits-demo 115e61c] Complete demo file
 1 file changed, 1 insertion(+)

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
$ git log -3 --oneline
115e61c (HEAD -> squash-commits-demo) Complete demo file
c702f16 Update demo file
07bf3a5 Add initial demo file
```

Step 6: Start Interactive Rebase

Run:

git rebase -i HEAD~3

This opens an editor showing the last three commits.

The initial list should contain entries similar to:

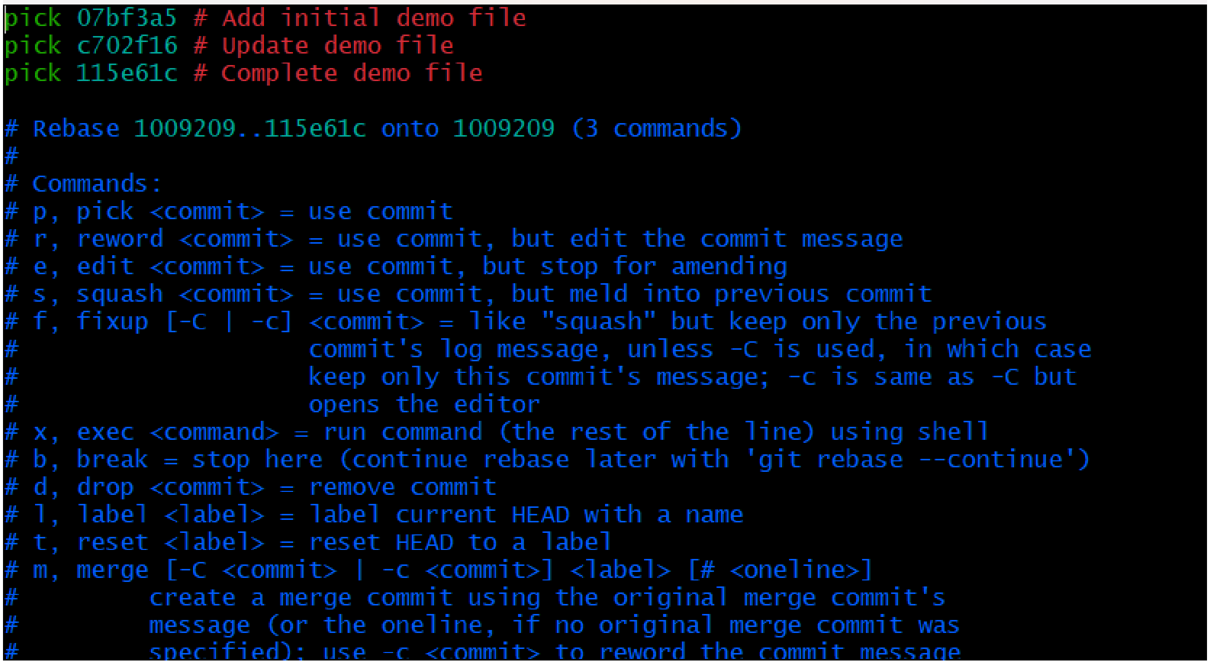
pick <hash-1> Add initial demo file

pick <hash-2> Update demo file

pick <hash-3> Complete demo file

The commits are listed from oldest to newest.

Screenshot 6:



```
pick 07bf3a5 # Add initial demo file
pick c702f16 # Update demo file
pick 115e61c # Complete demo file

# Rebase 1009209..115e61c onto 1009209 (3 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                    commit's log message, unless -C is used, in which case
#                    keep only this commit's message; -c is same as -C but
#                    opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
#         create a merge commit using the original merge commit's
#         message (or the oneline, if no original merge commit was
#         specified); use -c <commit> to reword the commit message
```

Step 7: Change the Commands to Squash the Commits

In the editor, keep the first commit as pick and change the next two commands to squash or s.

The final list should look similar to:

pick <hash-1> Add initial demo file

squash <hash-2> Update demo file

squash <hash-3> Complete demo file

Save the file and close the editor.

If Git opens Vim:

1. Press i to enter insert mode.
2. Edit the two lines to use squash.
3. Press Esc.
4. Type :wq and press Enter to save and exit.

If Git opens another editor, use its normal save-and-close procedure.

Explanation:

- pick keeps the first commit as the base commit.
- squash combines the selected commit with the previous commit and allows their messages to be combined.

Step 8: Enter the Combined Commit Message

After the rebase instructions are saved, Git may open another editor to combine the commit messages.

Replace the existing messages with one clear message, for example:

Add completed demo file

Save and close the editor.

Git will replay the changes and create one replacement commit containing the changes from all three original commits.

If Git reports a conflict, resolve the conflict as described in the troubleshooting section before continuing.

Step 9: Verify the Squashed Commit

Run:

```
git log -3 --oneline
```

The newest entry should now be the single combined commit:

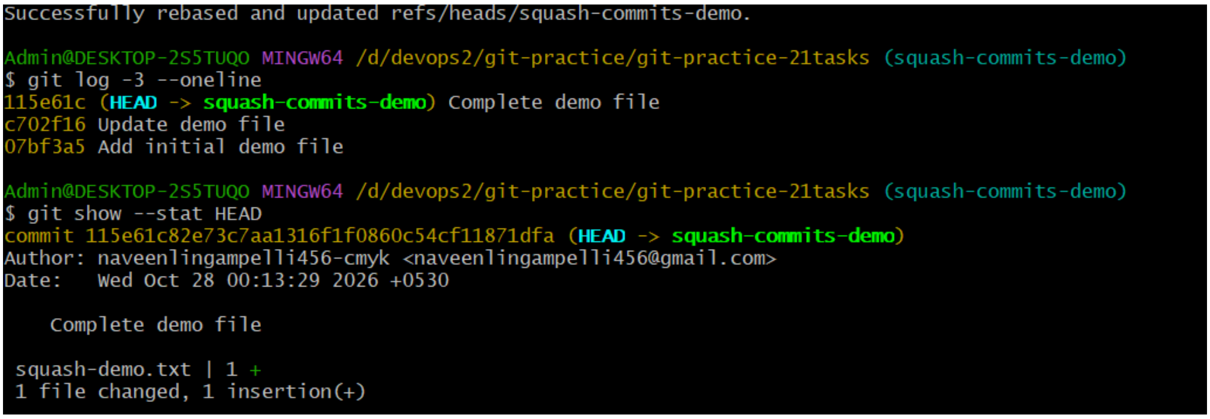
```
<new-hash> Add completed demo file
```

Older commits from before the demo branch will still appear beneath it. That is expected.

To inspect the changes included in the combined commit, run:

```
git show --stat HEAD
```

Screenshot 9:



```
Successfully rebased and updated refs/heads/squash-commits-demo.

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
$ git log -3 --oneline
115e61c (HEAD -> squash-commits-demo) Complete demo file
c702f16 Update demo file
07bf3a5 Add initial demo file

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
$ git show --stat HEAD
commit 115e61c82e73c7aa1316f1f0860c54cf11871dfa (HEAD -> squash-commits-demo)
Author: naveenlingampelli456-cmyk <naveenlingampelli456@gmail.com>
Date:   Wed Oct 28 00:13:29 2026 +0530

    Complete demo file

 squash-demo.txt | 1 +
 1 file changed, 1 insertion(+)
```

Step 10: Verify the File Contents

Run:

cat squash-demo.txt

Expected output:

First change

Second change

Third change

All three lines should remain in the file after the commits are squashed.

Check the repository status:

git status

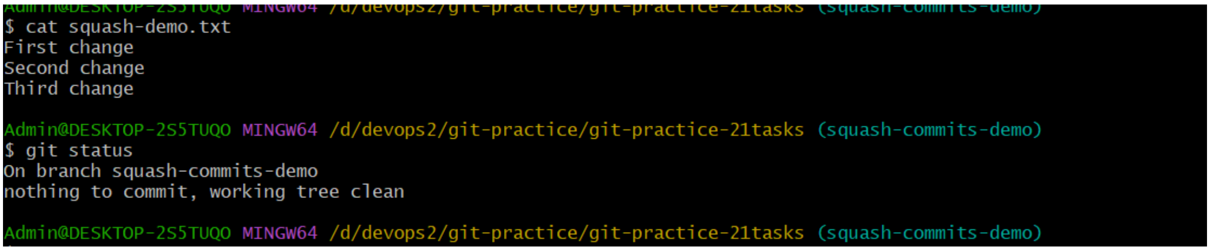
Expected result:

On branch squash-commits-demo

nothing to commit, working tree clean

The exact output may differ if other changes exist.

Screenshot 10:



```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
$ cat squash-demo.txt
First change
Second change
Third change

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
$ git status
On branch squash-commits-demo
nothing to commit, working tree clean

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice/git-practice-21tasks (squash-commits-demo)
```

5. Validation Steps

Use the following checklist to verify the task:

- Confirm that the repository working tree is clean before starting.
- Create and switch to squash-commits-demo.
- Create three separate commits.
- Verify all three commits with git log -3 --oneline.
- Run git rebase -i HEAD~3.
- Keep the oldest of the three commits as pick.
- Change the next two commits to squash.
- Save the combined commit message.
- Verify that the three demo commits have become one commit.
- Confirm that all three lines remain in squash-demo.txt.
- Confirm that the working tree is clean.
- Confirm that the main branch was not modified by the exercise.

6. Explanation of Important Commands

Command	Purpose
git switch -c squash-commits-demo	Creates and switches to a demo branch.
git add squash-demo.txt	Stages changes to the demo file.
git commit -m "message"	Creates a commit with the specified message.
git log -3 –oneline	Displays the three most recent commits concisely.
git rebase -i HEAD~3	Opens an interactive rebase for the last three commits.
Pick	Keeps a commit as a separate commit during the rebase sequence.
Squash	Combines a commit with the preceding commit and allows commit messages to be combined.
git show --stat HEAD	Displays the latest commit and its file-change statistics.
git status	Displays the working-tree and staging-area status.

7. Real-Time Use Case

In a DevOps team, a developer may create several commits while developing one feature:

Add deployment script

Fix deployment script

Update deployment documentation

Before submitting a pull request, the developer may combine these commits into one meaningful commit:

Add deployment automation

This can make the feature branch easier to review and maintain. Teams may also configure pull requests to squash commits during merging, depending on their repository policies.

Squashing is useful when multiple small commits represent one logical change and the team wants a simpler commit history.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
fatal: not a git repository	Git Bash is outside the repository.	Navigate to the repository root and retry.
fatal: invalid upstream 'HEAD~3'	The current branch does not have enough reachable commits.	Ensure three demo commits exist, then rerun the command.

Rebase editor opens but the changes are not saved	The editor was closed without saving.	Reopen the rebase operation if it is still active, edit the instructions, and save correctly.
Rebase reports a conflict	Changes from different commits cannot be applied automatically.	Resolve the conflicted files, stage them with git add, then run git rebase --continue.
Wrong commit message appears	The combined commit message was not edited as intended.	If the rebase completed, use git commit --amend -m "Add completed demo file" to rename the latest combined commit.
Three demo commits still appear	The interactive rebase was cancelled or the squash commands were not applied.	Check the current branch and rerun the interactive rebase after confirming the three intended commits.
git status shows unexpected changes	Other files were changed during the exercise.	Review the files and avoid including unrelated changes in the rebase.
Push is rejected after squashing previously pushed commits	Rebase changed commit IDs on the remote branch.	Coordinate with the team and follow the repository's approved history-update process.

If you need to cancel an in-progress rebase and return to the state before it started, run:

```
git rebase --abort
```

Use this only while a rebase is in progress. If the rebase has already completed, do not assume that git rebase --abort can undo it.

9. Best Practices

- Use a separate branch to practice interactive rebase.
- Review the commit list carefully before squashing.
- Preserve meaningful changes and choose a descriptive combined message.
- Do not squash commits that other developers depend on without coordination.
- Avoid rewriting shared branch history unless the team has agreed to it.
- Verify the resulting history and file contents after the operation.
- Consider using your repository's pull-request squash-merge option when it matches the team's workflow.

10. Conclusion

In this task, we learned how to merge multiple Git commits into a single commit using interactive rebase and the squash instruction.

We created three demo commits, combined them into one commit, verified the updated commit history, and confirmed that the file retained all its changes.

This technique helps developers maintain a cleaner, more understandable Git history when several commits belong to one logical feature or change.